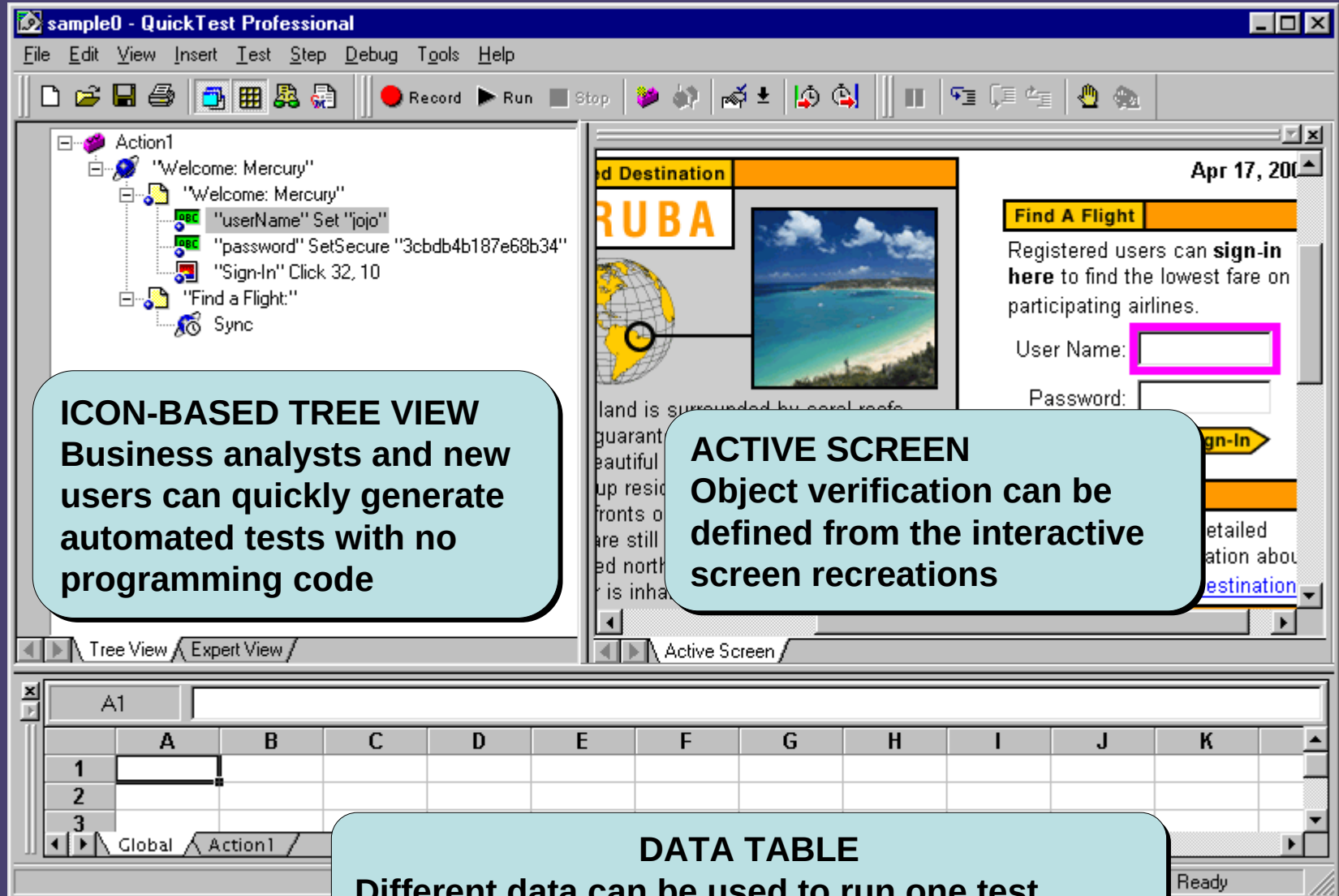


Fundamentals of QuickTest Professional 6

TRAINING OBJECTIVES

- Introduction
- Test Planning
- Record and Playback
- How QuickTest Identifies Objects
- Synchronizing Tests
- Standard and Text Checkpoints
- Creating Tests with Multiple Actions
- Data Driven Tests
- Output and Correlation
- Verifying Database Content

Why Use QuickTest? (1)



ICON-BASED TREE VIEW
Business analysts and new users can quickly generate automated tests with no programming code

ACTIVE SCREEN
Object verification can be defined from the interactive screen recreations

DATA TABLE
Different data can be used to run one test (instead of manually running tests repeatedly)

Benefits of the Graphical Interface

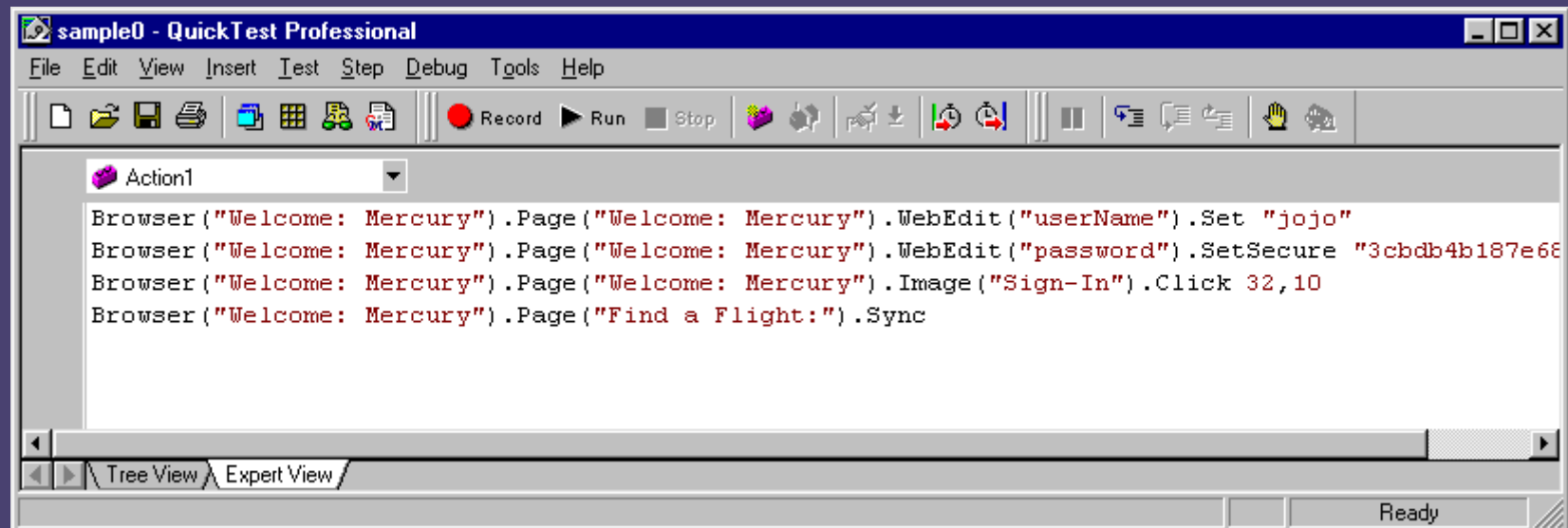
Point and click interface provides ease of use for:

- Recording your test
- Adding verification
- Creating parameters
- Using one test for many iterations
- Adding business conditions

Why Use QuickTest? (2)

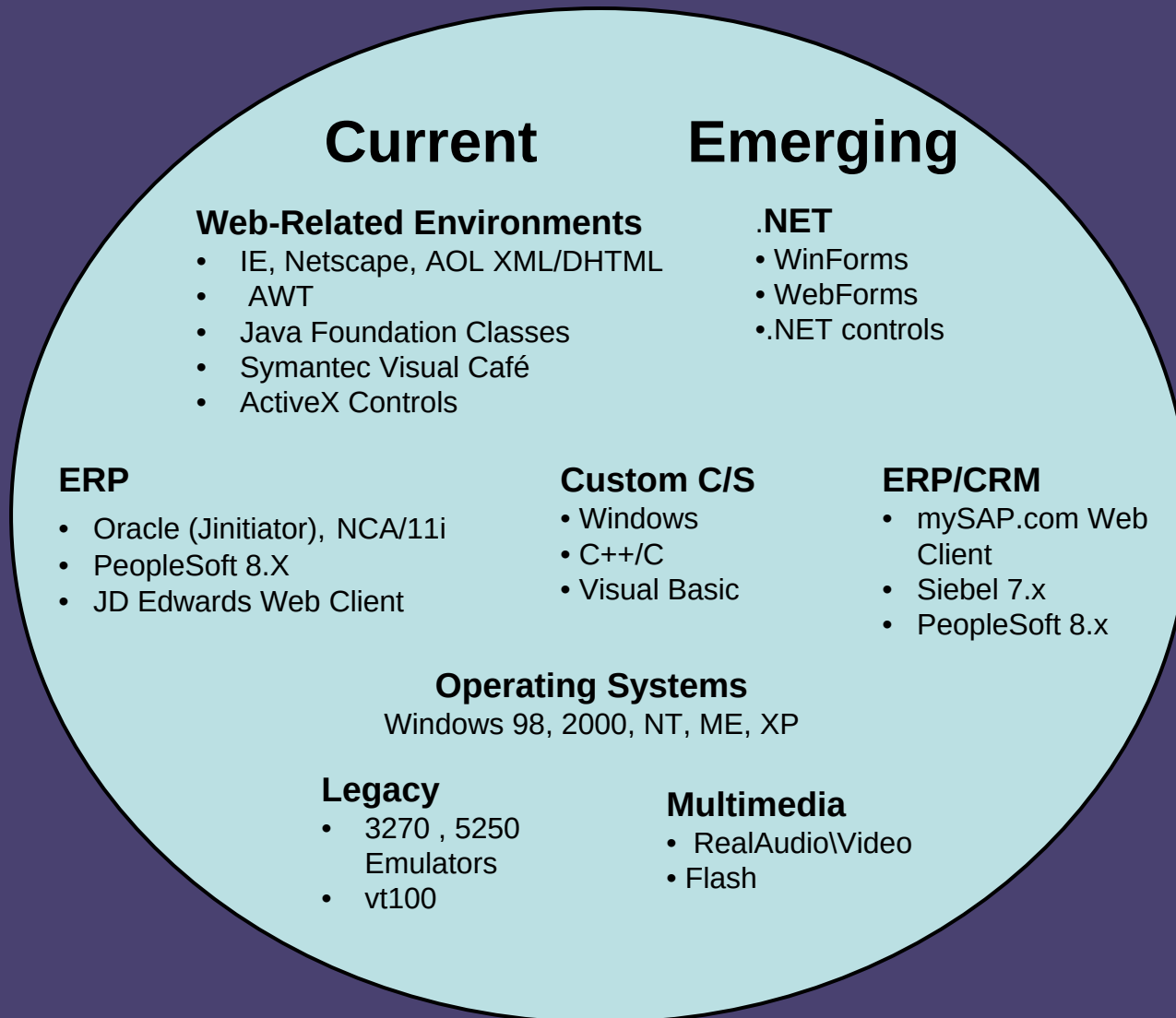
Powerful scripting capabilities for robust testing:

- For users with programming background
- For users who will work with dynamic values, external DLLs, VBScript's FileSystemObject and user-defined functions



Working with the Expert View is taught in the Advanced QuickTest Professional course.

QuickTest Supported Environments



Which Test Cases to Automate?

- ✓ Tests that need to be run for every build of the Web site (*sanity level*)
- ✓ Tests that use multiple data values for the same actions (*data driven tests*)
- ✓ Identical tests that need to be executed using different browsers
- ✓ Mission-critical pages
- ✓ Transaction with pages which won't change in short term



**More repetitive execution?
Better candidate for automation.**

Which Test Cases *Not* to Automate?

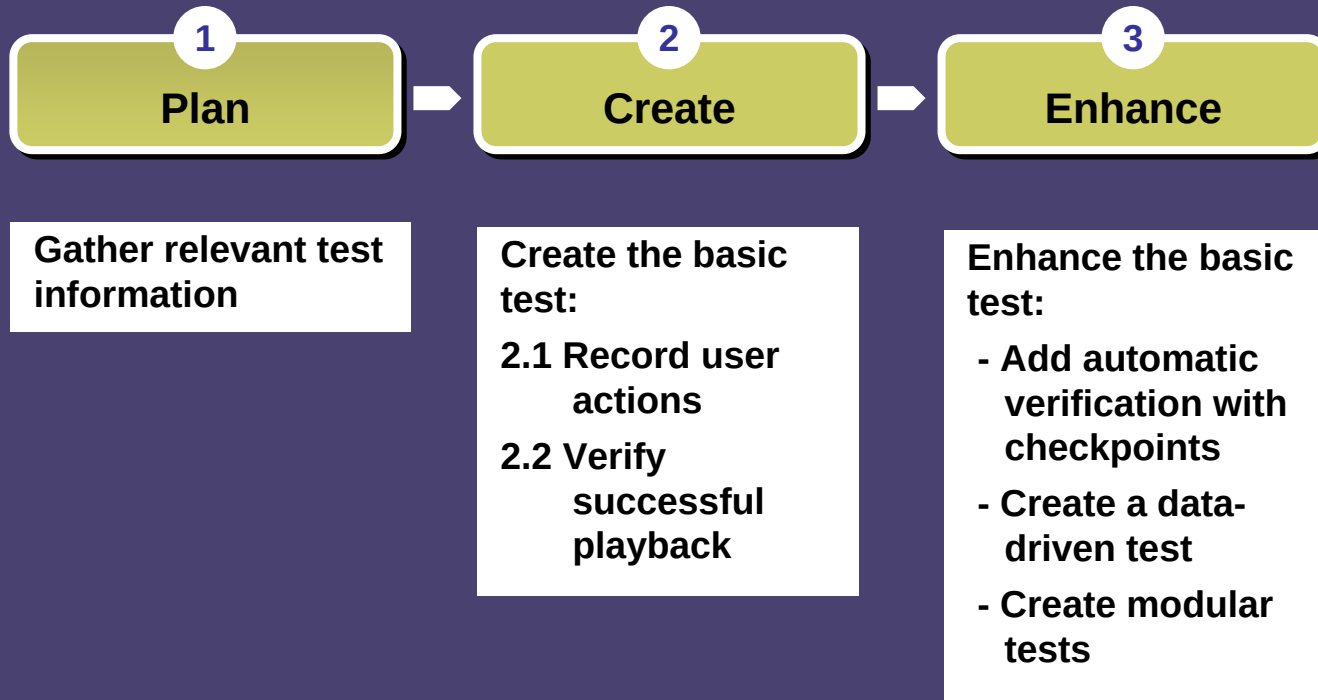
- ❌ Usability testing
 - "How easy is the Web site to use?"
- ❌ One-time testing
- ❌ Ad hoc/random testing
 - based on intuition and knowledge of application



Improvisation required?
***Poor* candidate for automation.**

QuickTest Workflow

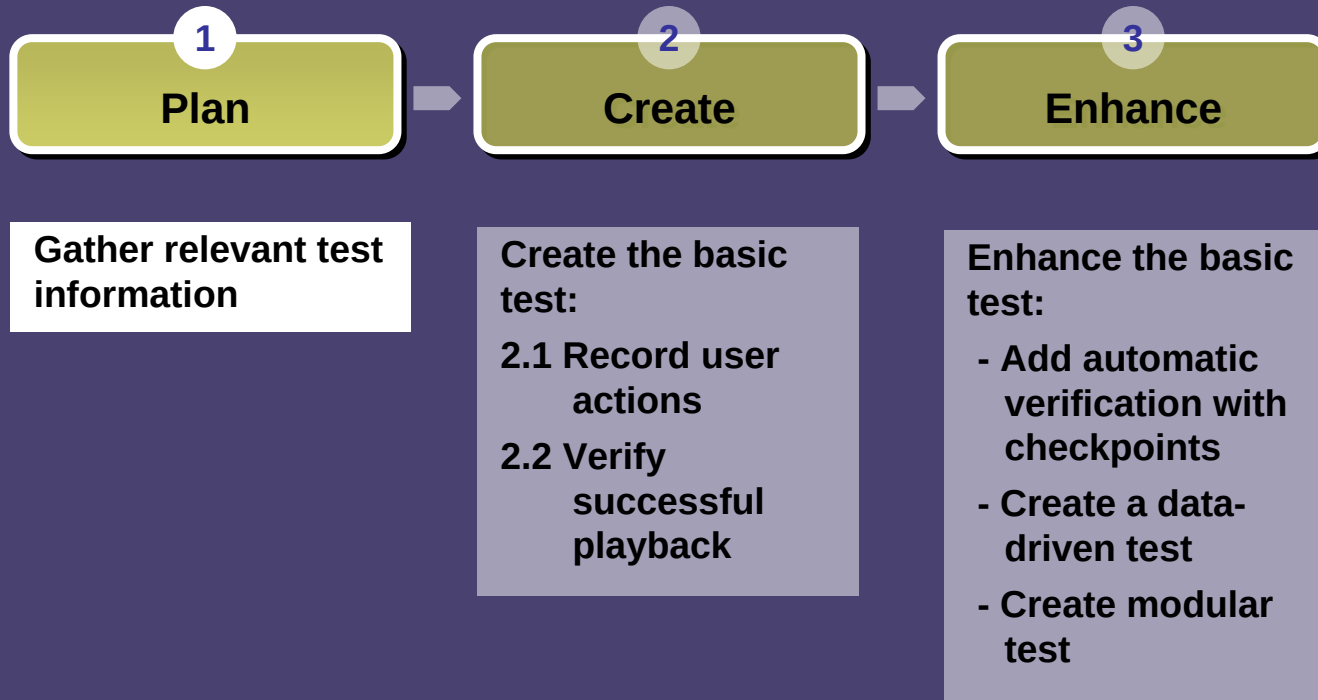
Automated



Test Planning

QuickTest Workflow

Automated



Planning Review: Document Business Process Properties

Obtain necessary information to create tests:

- User actions
- Expected results of each user action
- Alternate paths and optional screens
- Valid data to use
- Possible exceptions

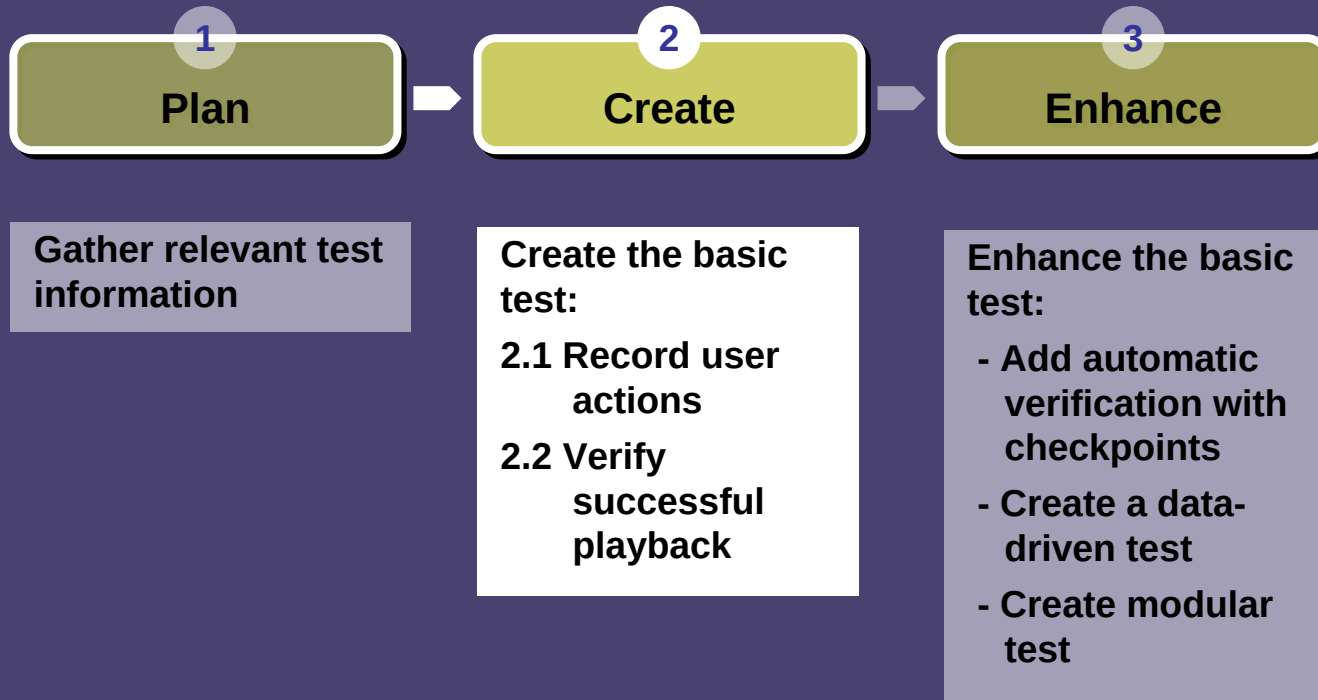
Prepare for Automated Testing

- Check that the application under test is stable and ready for testing
- Go through the test case steps on the application to verify that all steps are listed in the correct order
- Check that the provided data to use is valid for recording the basic test
- Check that QuickTest and the add-ins, if any, are installed and running without errors

Record and Playback

QuickTest Workflow

Automated



Start a New Test in QuickTest

Allows you to select the add-ins QuickTest will use depending on the objects you will test



Before you record:

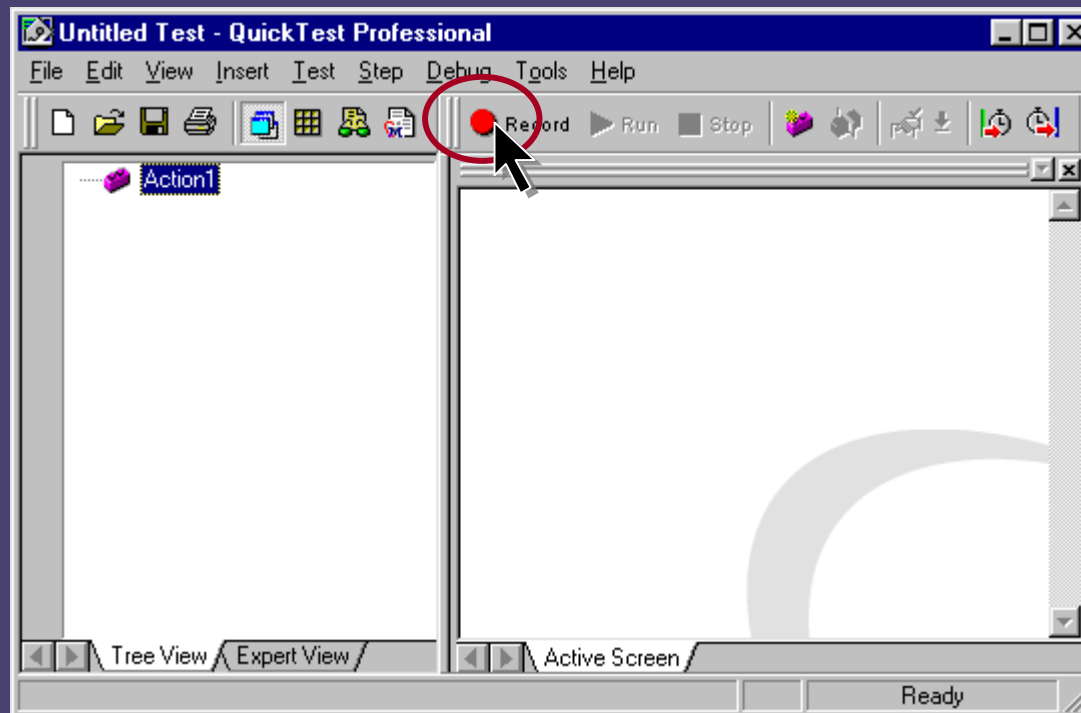


- Learn the application under test
 - User actions of the transaction to be recorded
 - Valid data to use
- Close all the applications not required for the test

2.1 Record User Actions



Follow the user actions listed in the test case.
Make sure you have the correct sample data for a
complete and successful transaction.

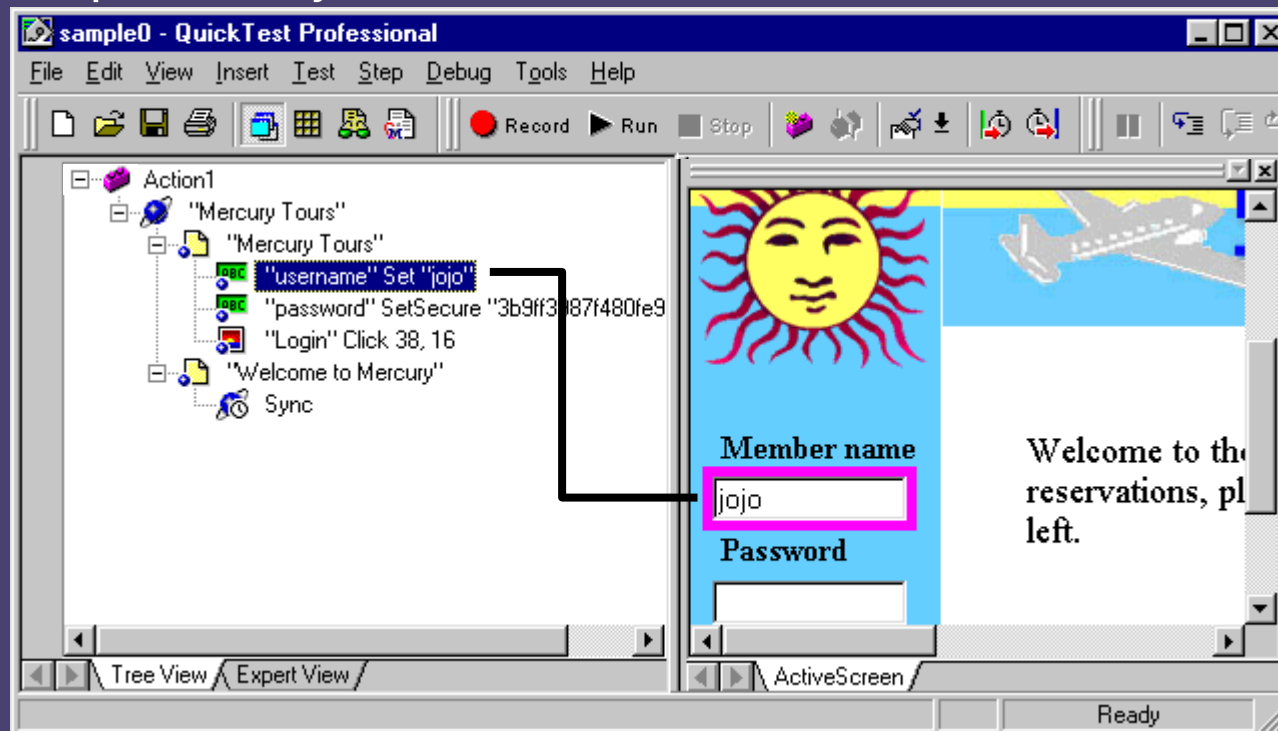


QuickTest window for a new blank test

Recorded User Actions - Basic Test

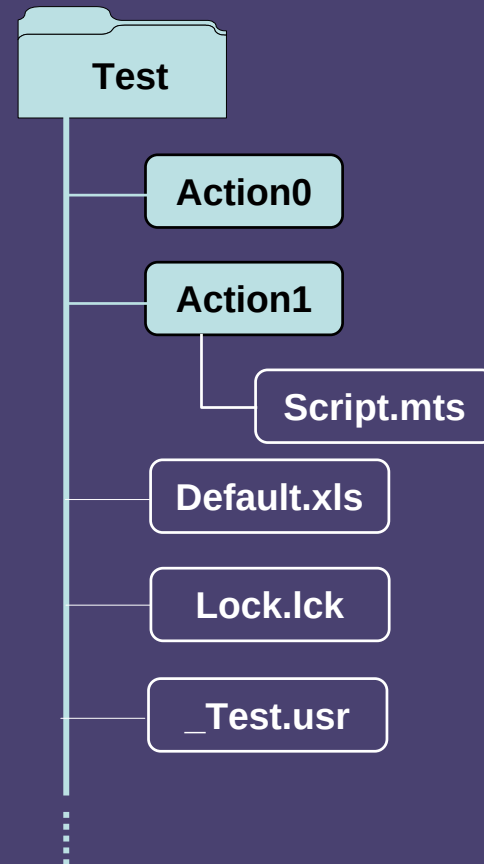


Each selected step in the Tree View highlights the captured object in the ActiveScreen



Test Structure

- QuickTest creates a directory structure for each saved test
 - Action1 contains log of user actions
 - Action0 invokes Action1
 - Lock file allows only one user to access an open test
 - Other internal QuickTest files



2.2 Verify Correct Playback



Check that no error messages appear

1 Use the 'Run' button to begin playback from the first step

2 Select the location of the test results

- new results folder
- temporary folder

Run Results Destination

Write run results to

☐ New run results folder

☒ Temporary run results folder (overwriting older temporary results)

C:\TEMP\TempResults

OK Cancel Help

Viewing the Test Results

The screenshot shows the 'sample0 [TempResults] - Test Results' window. The left pane displays a tree view of the test results:

- Test sample0 Summary
 - Runtime Data
 - sample0 Iteration 1 (Row 1)
 - Action1 Summary
 - Welcome: Mercury
 - Welcome: Mercury
 - "userName":Set "jojo"
 - "password":SetSecure "3cddb4b187e68b"
 - "Sign-In":Click
 - Find a Flight:
 - Sync

The right pane shows the details for the selected step:

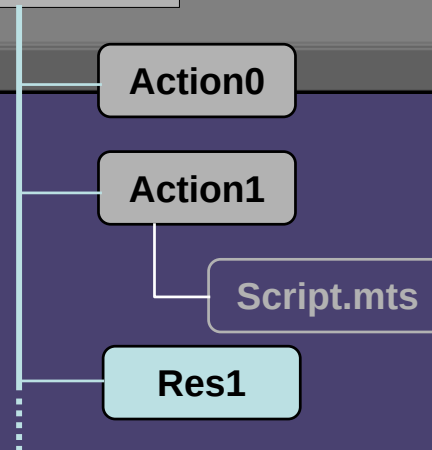
Step Name:
"userName":Set "jojo"

Step Done

Test	Details	Result	Time
"userName":Set "jojo"	Set "jojo"	Done	13:29:27

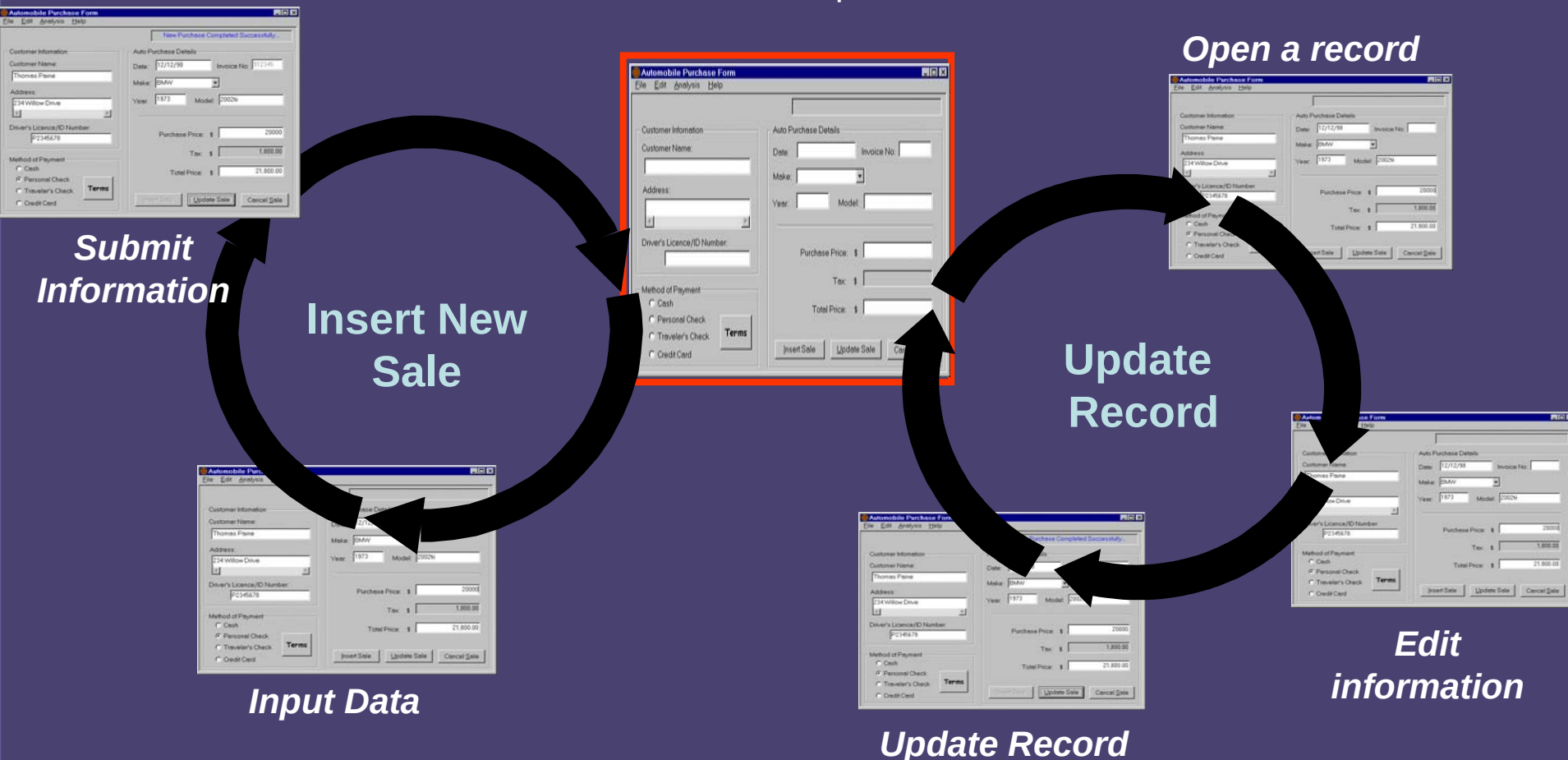
At the bottom of the right pane, there is a status bar with the text 'Ready'.

Test results can be stored in their own location (for documentation) or in a temporary location that is overwritten each time the test is run (for debugging).



Best Practice: Setting Initial and End Conditions

- Identify identical start and end state
 - Allows test case to iterate successfully
 - Enables test case independence



Problem: Playback Displays Error



Why is the test giving me run error on 'OutOfStock' step?

Why Test Results Show a Warning

The screenshot shows the 'optional [debug] - Mercury Test Results' window. The left pane displays a test execution tree. A red box highlights a section of the tree under 'optional Iteration 1 (Row 1)' containing the following steps:

- Replay Error : email
- Description : email
- Error ignored : email
- Replay Error : Yes
- Description : Yes
- Error ignored : Yes

The right pane shows a table with the following data:

Object	Details	Result	Time
email	Optional step - Proceeding with replay	Information	10:20:50

Below the table, an 'Out of Stock' dialog box is displayed with the message: 'Sorry. This item is not in stock. If you would like to be notified of the arrival of this item enter your email id otherwise press cancel.' The email field contains 'mercury@hotmail.com' and 'Yes' and 'Cancel' buttons are at the bottom.

For Help, press F1

Upon playback the test is not able to identify the object 'OutOfStock' and issues a warning that is subsequently ignored.

Low-Level Recording

To capture unsupported objects in QuickTest:

- Use the low-level recording mode to record your actions and keyboard input based on window coordinates

To record in low-level mode:

- select Test > Low Level Recording

or,

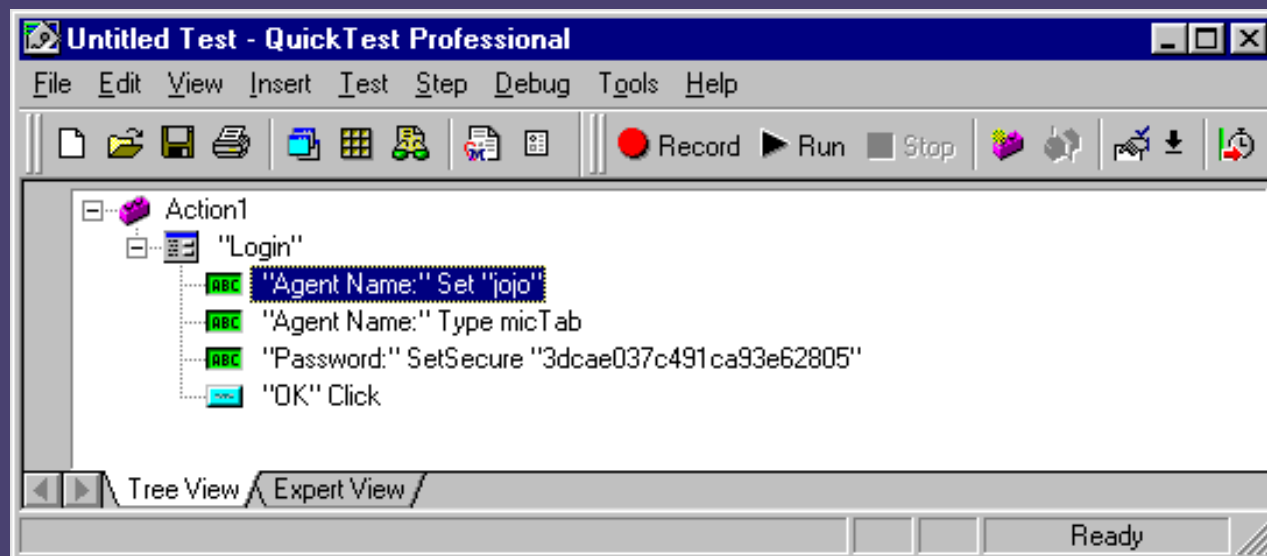
- press **Ctrl+Shift+F3**

while recording.

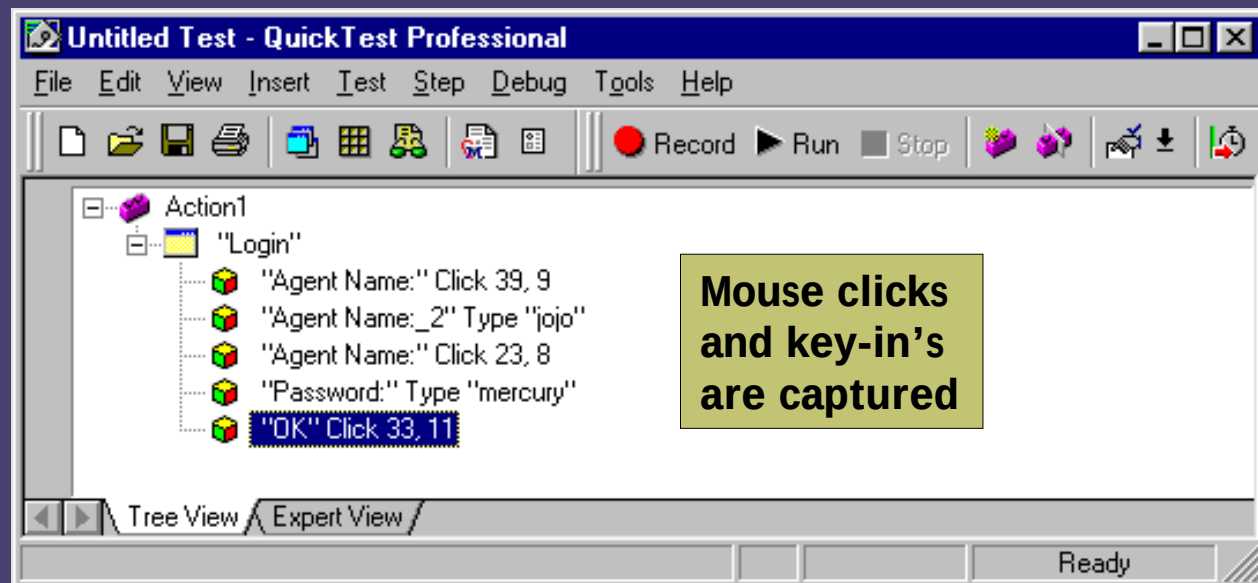
Samples of Standard and Low-level Recording



**Standard
Recording**



**Low-level
Recording**



How QuickTest Identifies Web Objects



Definition

object properties

A set of characteristics that define an object's appearance, values, state and identity in an application.

How QuickTest Identifies Objects During Recording

QuickTest creates a unique identity for an object by:

1. Recognizing the object type ("class")
2. Learning the object's properties
3. Assigning a "Logical name" to the learned object

QuickTest Learns the Object Properties

Get Web object properties from the HTML

```

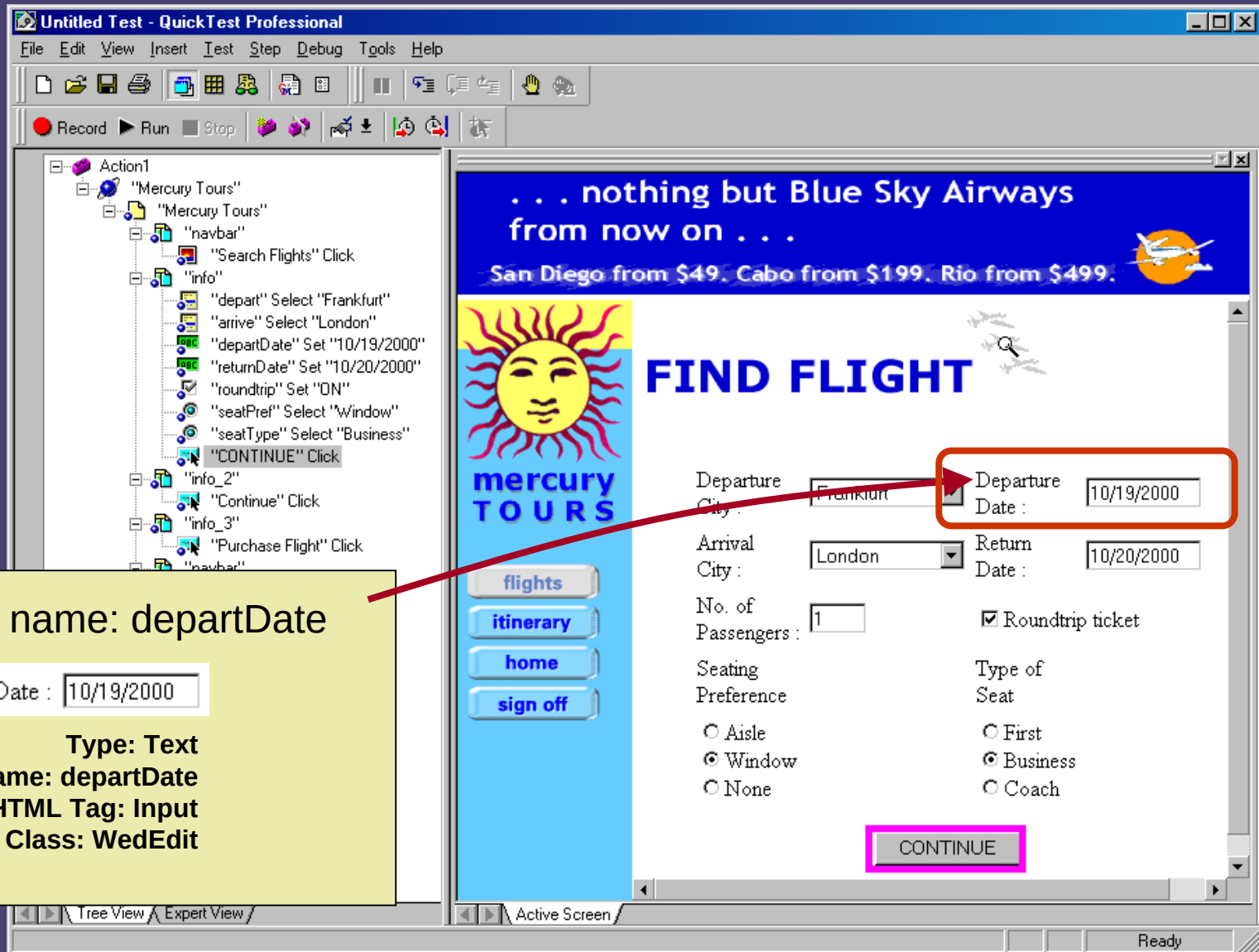
<BODY BGCOLOR="white"><img src=/MercuryWebTours/images/splash_findflight.gif width=395 height=90>
<blockquote>
<FORM METHOD="POST" ACTION="reservations.pl" ENCTYPE="application/x-www-form-urlencoded" ONSUBMIT="" NAME="DestForm">
<TABLE CELLSPACING="5" BORDER="0">
<TR><TD ALIGN="left">Departure City :</TD>
<TD><SELECT NAME="depart">
      <OPTION SELECTED VALUE="Denver">Denver
      <OPTION VALUE="Frankfurt">Frankfurt
      <OPTION VALUE="London">London
      <OPTION VALUE="Los Angeles">Los Angeles </SELECT> </TD>
<TD ALIGN="left">Departure Date :</TD>
<TD><INPUT TYPE="text" NAME="departDate" VALUE="10/19/2000" SIZE=10 MAXLENGTH=10> </TD></TR>
<TR><TD ALIGN="left">Arrival City :</TD>
<TD><SELECT NAME="arrive">
      <OPTION SELECTED VALUE="Denver">Denver
      <OPTION VALUE="Frankfurt">Frankfurt
      <OPTION VALUE="London">London
      <OPTION VALUE="Los Angeles">Los Angeles </SELECT></TD>
<TD ALIGN="left">Return Date :</TD> <TD><INPUT TYPE="text" NAME="returnDate" VALUE="10/20/2000" SIZE=10 MAXLENGTH=10> </TD></TR>
<TR><TD ALIGN="left">No. of Passengers : </TD>
<TD><INPUT TYPE="text" NAME="numPassengers" VALUE="1" SIZE=3 MAXLENGTH=8></TD>
<TD COLSPAN="2"><INPUT TYPE="checkbox" NAME="roundtrip" VALUE="on">Roundtrip ticket </TD></TR>
<TR><TD>Seating Preference</TD> <TD> <TD>Type of Seat</TD></TR>
<TR><TD><INPUT TYPE="radio" NAME="seatPref" VALUE="Aisle">Aisle<BR>
<INPUT TYPE="radio" NAME="seatPref" VALUE="Window" CHECKED>Window<BR>
<INPUT TYPE="radio" NAME="seatPref" VALUE="None">None<BR></TD> <TD>
<TD>
<INPUT TYPE="radio" NAME="seatType" VALUE="First">First<BR>
<INPUT TYPE="radio" NAME="seatType" VALUE="Business" CHECKED>Business<BR>
<INPUT TYPE="radio" NAME="seatType" VALUE="Coach">Coach<BR></TD></TR>
<TR><TD HEIGHT="10"></TD>
<TR><TD ALIGN="center" COLSPAN="4"><INPUT TYPE="submit" NAME="findFlights" VALUE="CONTINUE"></TD></TR></TABLE>

```

**Name: departDate
Type: Text
HTML Tag: Input**

Departure Date : 10/19/2000

Sample Object as Recognized by QuickTest



Definition

test object

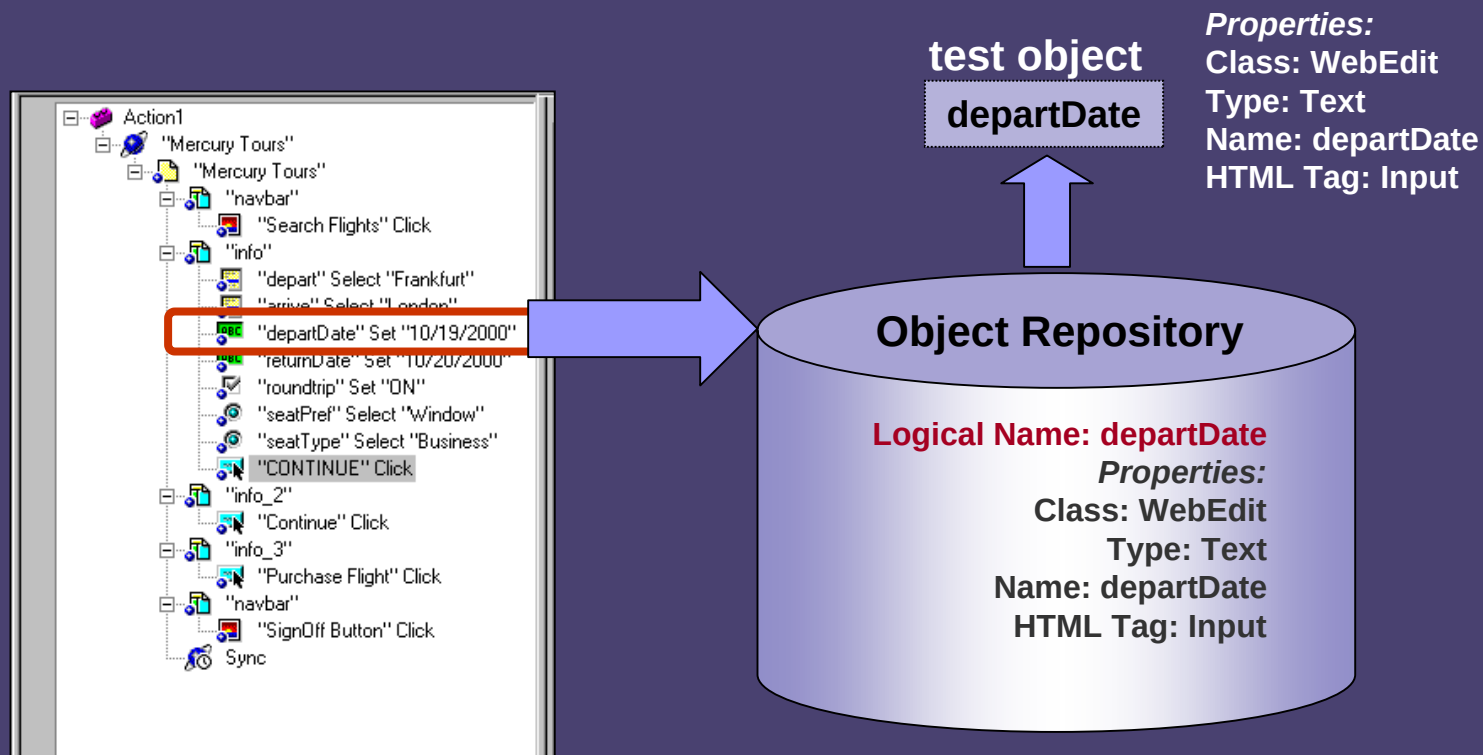
A test object is an object which QuickTest creates for the test to represent an actual object in your Web site or application.

run-time object

A run-time object is an actual object in your Web site or application on which methods are performed during the test run.

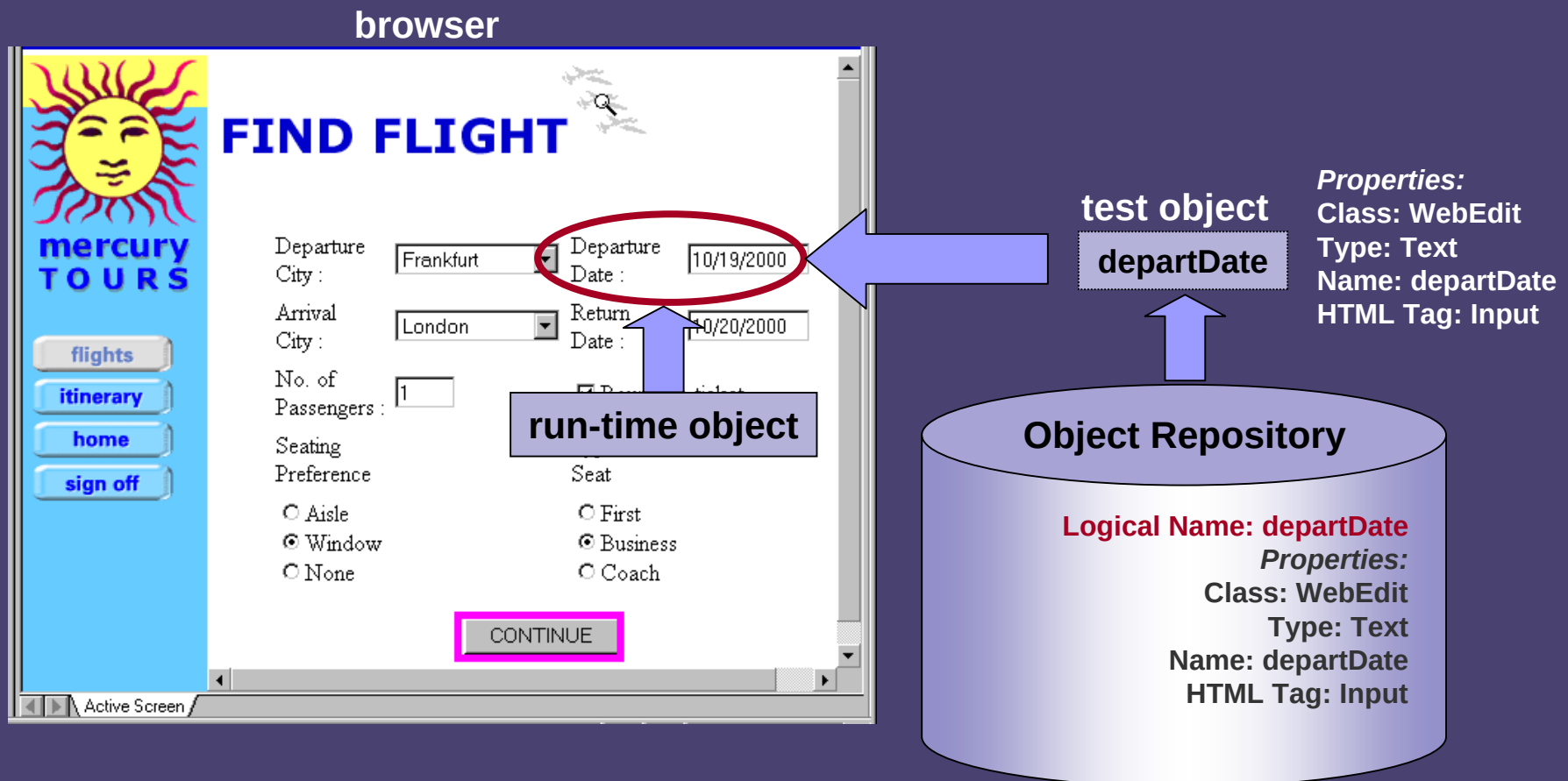
The Test Object

QuickTest creates a test object based on the class, and gives it the properties of the recorded Web object.



The Run-time Object

- QuickTest searches the Web page in the browser for an object with properties identical to the test object
- If it finds a matching object, QuickTest performs the user action called for by the script on that object



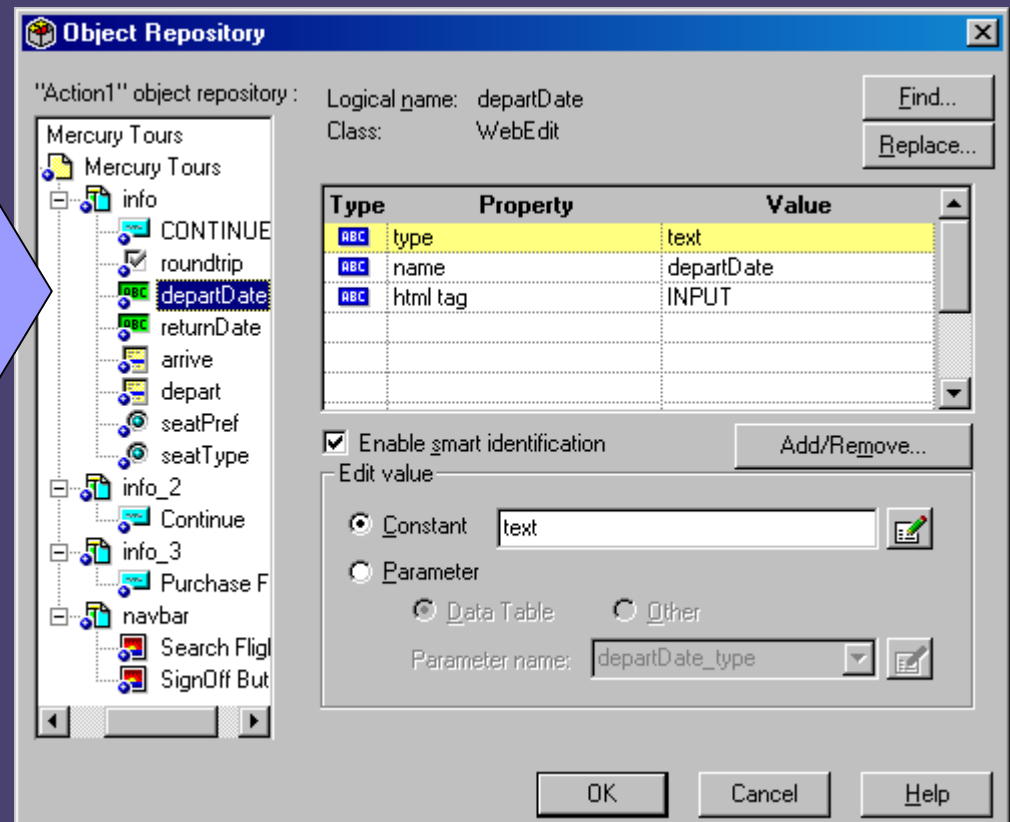
Object Repository

Test Object information is stored in the Object Repository

Logical name: **departDate**

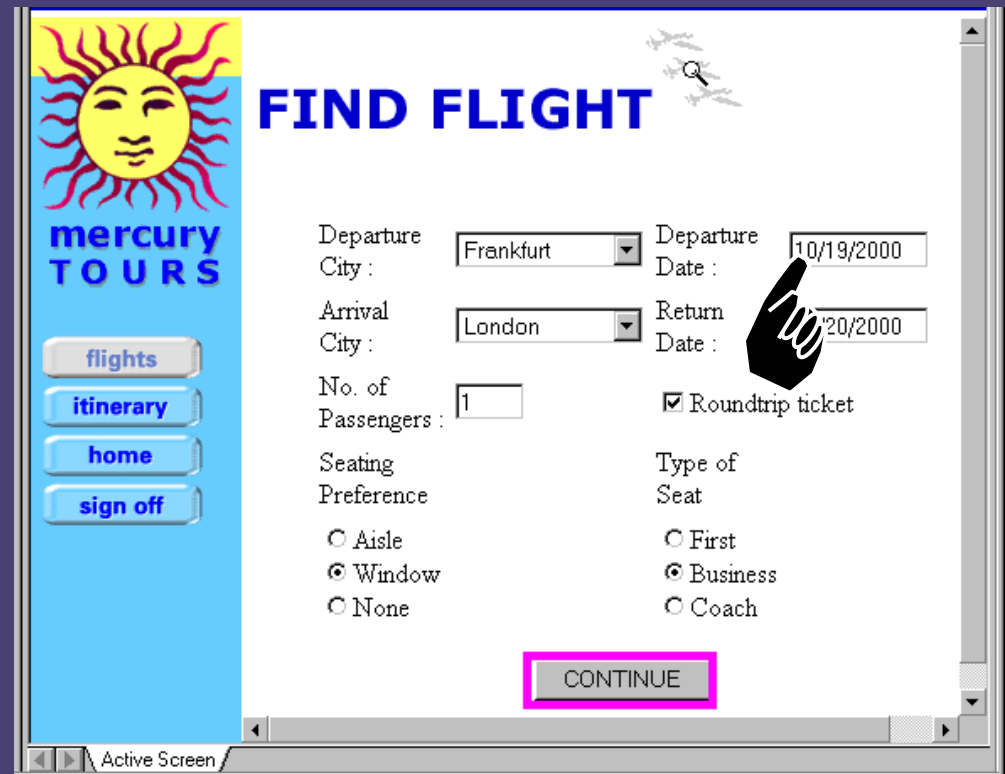
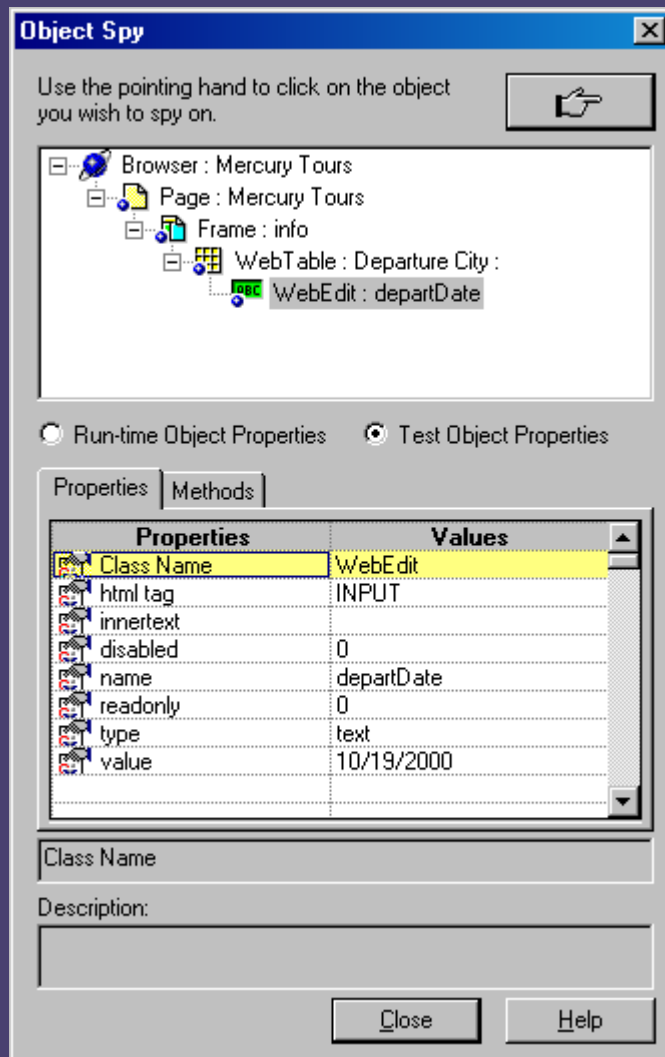
Departure Date : 10/19/2000

Type: Text
Name: departDate
HTML Tag: Input
Class: WebEdit



Object Spy

Run-time Object information can be seen with the Object Spy



Smart Identification—Example

- QuickTest could not identify the image object, "Featured Destination"

The screenshot shows the 'Correlate2 [Res1] - Test Results' window. The left pane displays a tree view of test results, including 'Test Correlate2 Summary', 'Run-Time Data', and 'Correlate2 Iteration 1 (Row 1)'. The right pane shows the details for the 'Featured Destination' object, which was identified successfully.

Step Name: "Featured Destination"- Smart Identification

Step Done

Object	Details	Result	Time
"Featured Destination"- Smart Identification	The smart identification mechanism identified a unique object with the following properties: image type = Plain Image html id = name = Image class = visible = 1 width = 273 micclass = Image html tag = IMG	Done	12/16/2002 - 17:39:43

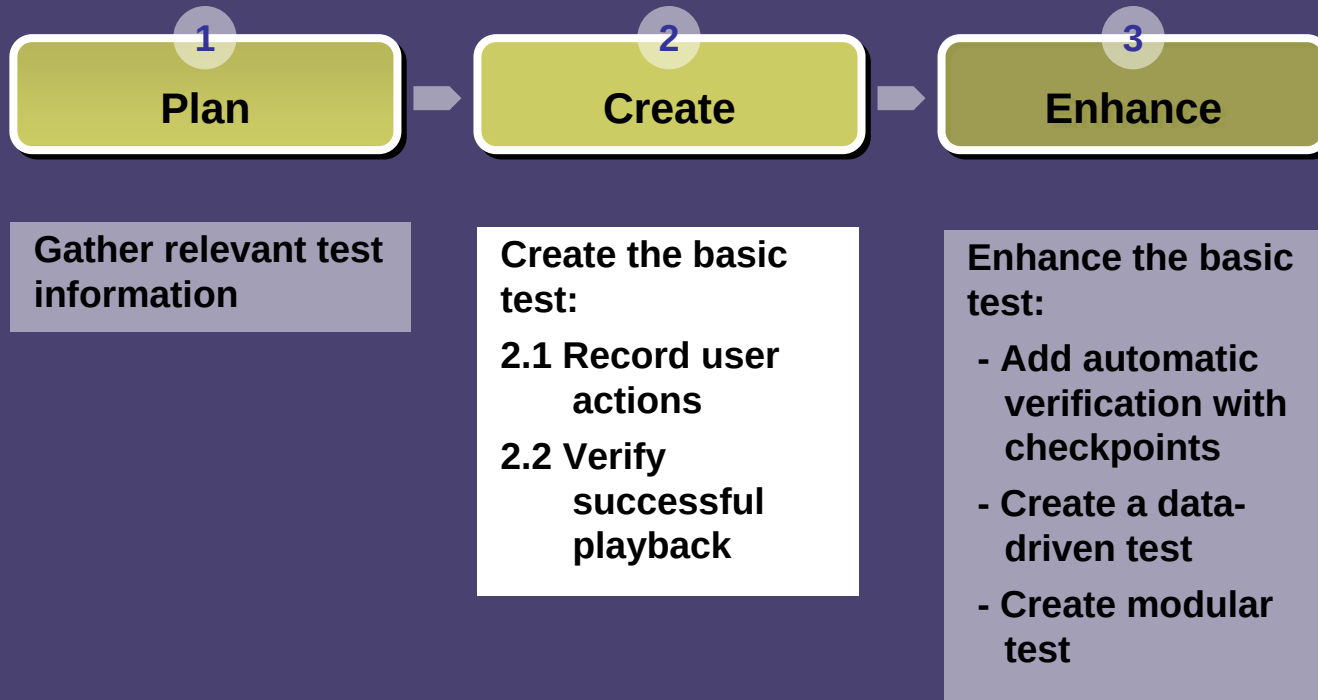
For Help, press F1

Ready

Synchronizing Tests

QuickTest Workflow

Automated



Problem: Some Objects Require More Processing Time

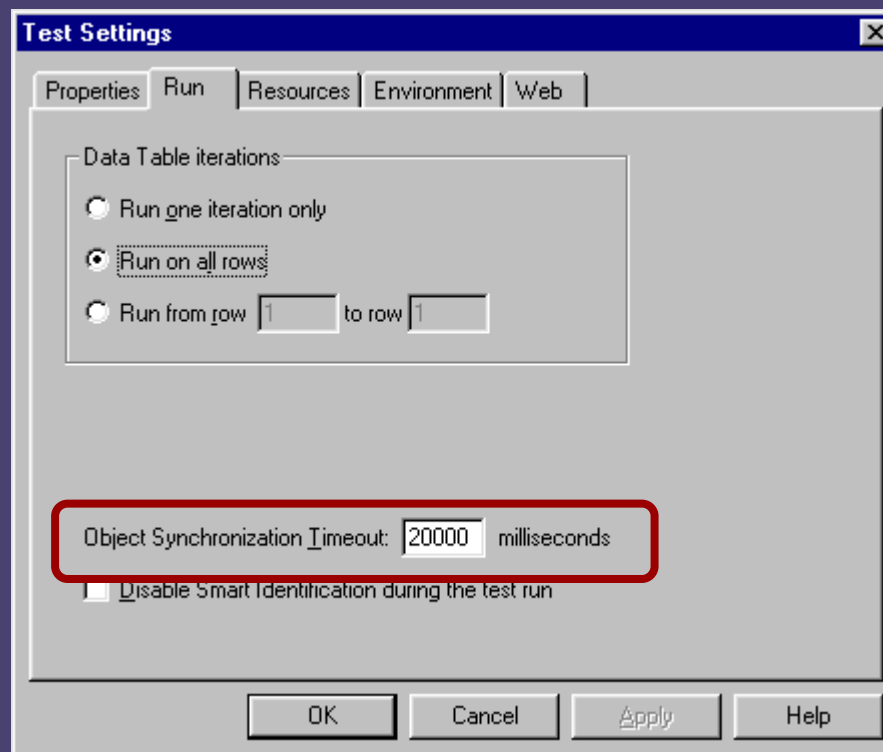
- On one hand, some objects may take several seconds longer than other objects due to processing time, for example:
 - For a progress bar to reach 100%
 - For a status message to appear
 - For a button to become enabled
 - For a window or pop-up message to open
- On the other hand, QuickTest runs each step in the same length of time
- An “Object not enabled” message appears if QuickTest runs a step and progresses to the next while the previous step is not yet complete

Solution: Synchronize the Test

- What is synchronization?
- Why synchronize?
- When to synchronize?

Global Synchronization Value for All Objects

- Edit the Global synchronization value
- Instructs QuickTest to wait for all objects for a specific amount of time.



Synchronization Step for a Specific Object



What property of a window can I use if the script will wait for it to appear?

How about 'text' property?
QuickTest will wait until a window with the specified window title will appear before moving to the next step.

IMPORTANT! Identify the object property that will synchronize the step

Add Synchronization Point [X]

Class:	Window
Logical name:	Flight
Property name:	abs_x
Property value:	text
Timeout (in milliseconds):	visible
	width
	window id
	x
	y

OK

Standard and Text Checkpoints

QuickTest Workflow

Automated



Visual Cues Verify a Process

MERCURY TOURS

[SIGN-OFF](#) [ITINERARY](#) [PROFILE](#) [SUPPORT](#) [CONTACT](#)

[Home](#) [Flights](#) [Hotels](#) [Car Rentals](#) [Cruises](#) [Destinations](#) [Vacations](#)

HTML VERSION
[Use Java Version](#)

SAVINGS!
Rent A Car

[CLICK HERE](#)

FLIGHT CONFIRMATION

Your itinerary has been booked!

Please print a copy of this screen for your records. Thank you for choosing Mercury Tours. **PRINT IT!**

Flight Confirmation # 2002-10-01208 2002-10-01 11:04:38.0

Departing
San Francisco to Paris
12/02/2002 @ 7:13 w/ Blue Skies Airlines 641
First
\$615 each

Passengers
2 passengers

Billed To
1325 Borregas Ave.
Sunnyvale CA, 94089, 215
AX 987654321

Delivery Address (N/A for Ticketless Travel)
1325 Borregas Ave.
Sunnyvale CA, 94089, 215

Total Taxes: \$101 USD
Total Price (including taxes): \$1331 USD

[BACK TO FLIGHTS](#) [BACK TO HOME](#) [LOG OUT](#)

Active Screen

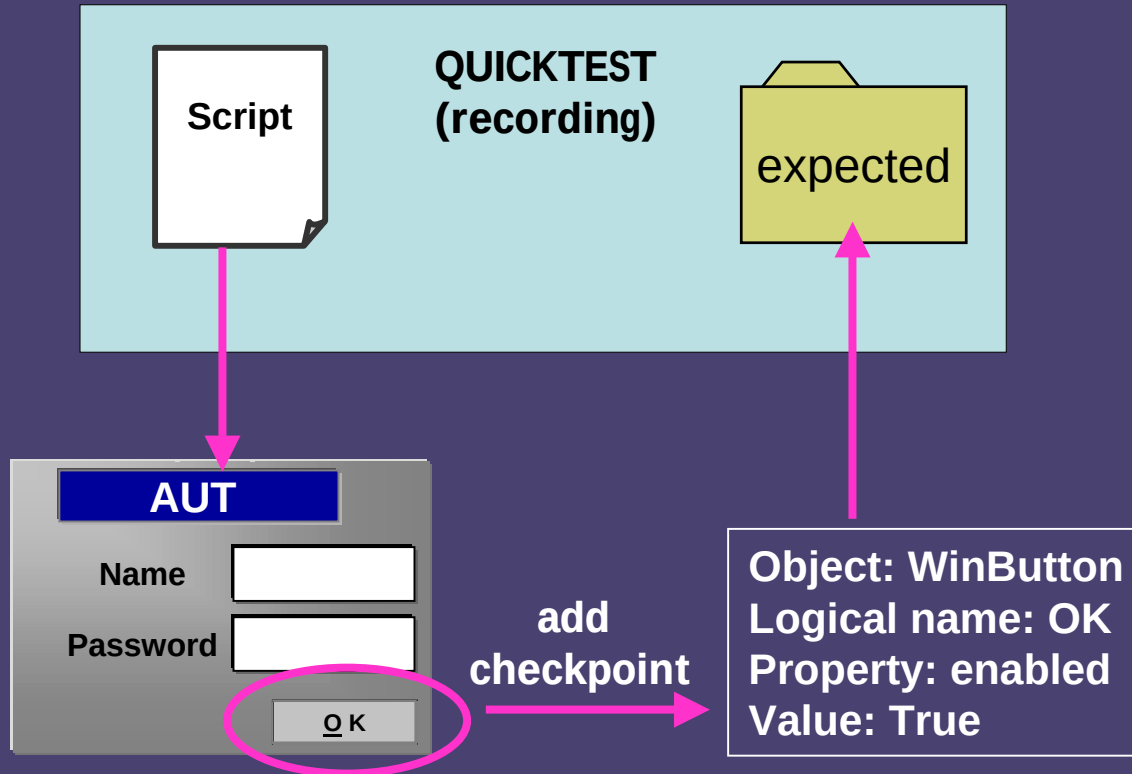
Definition

checkpoint

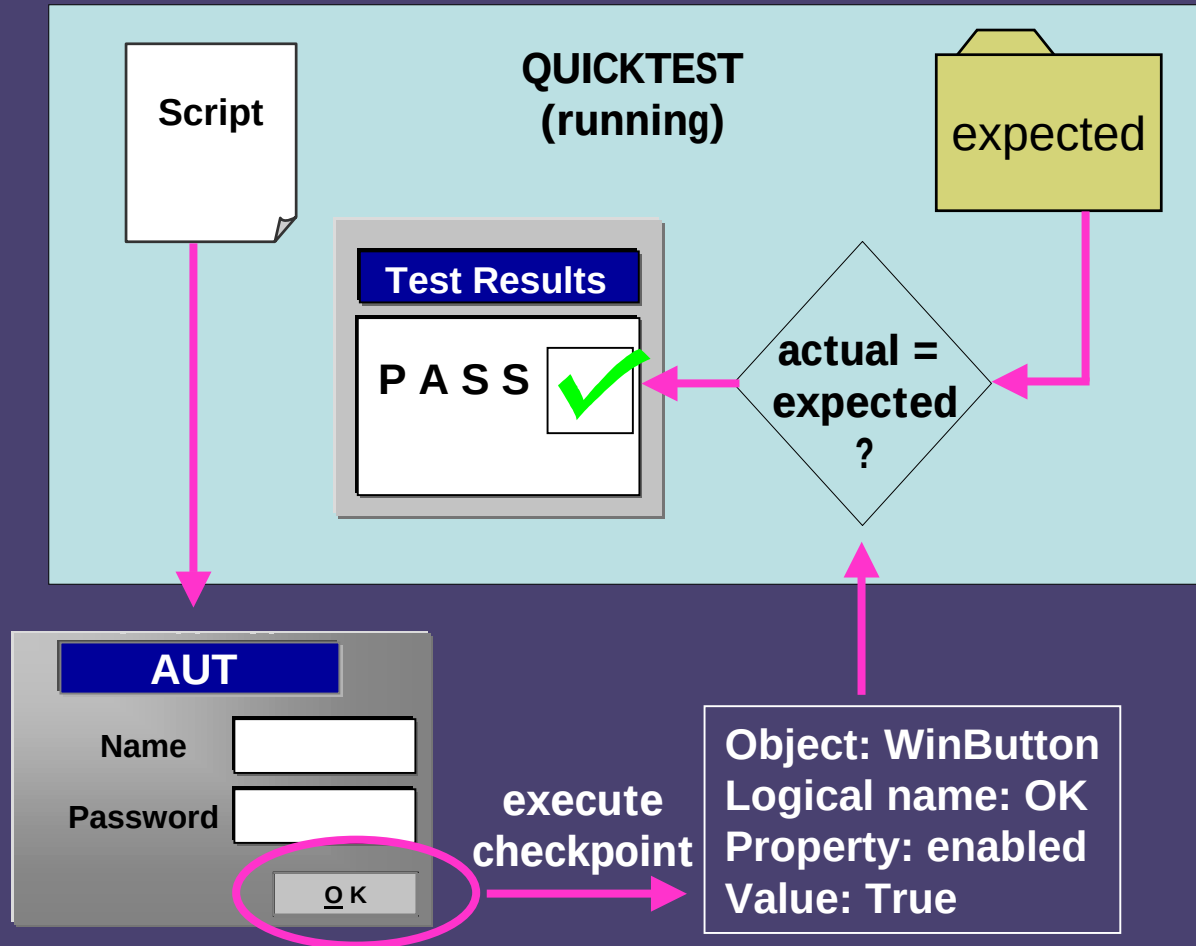
A checkpoint is a specialized step in a QuickTest script that compares 2 values and reports the result.

If the 2 values are the same, the checkpoint passes; if not, the checkpoint fails.

How Checkpoints Work—Recording



How Checkpoints Work—Running



How to Add a Standard Checkpoint

1. Select Insert > Standard Checkpoint.
2. Click on object in the ActiveScreen.
3. Accept default object, or pick another object from the dialog – click OK.
4. Select the property you want to check and click OK.

FLIGHT FINDER

Use our Flight Finder to search for the lowest fare on participating airlines. Once you've booked your flight, don't forget to visit the Mercury Tours Hotel Find and reserve lodging in your destination city.

Flight Details

Type: ☐ Round Trip ☒ One Way

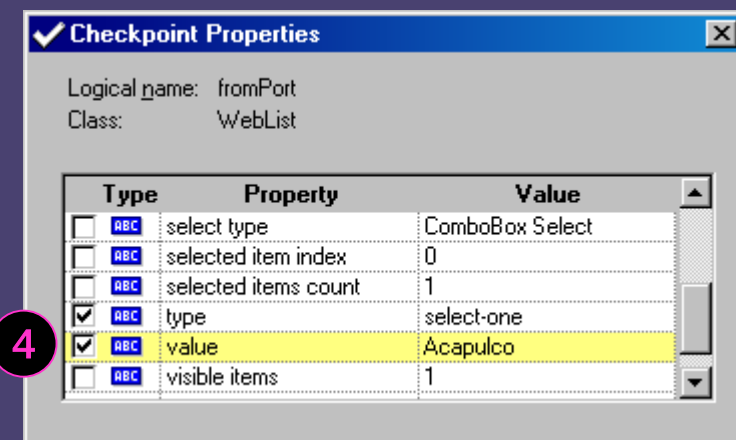
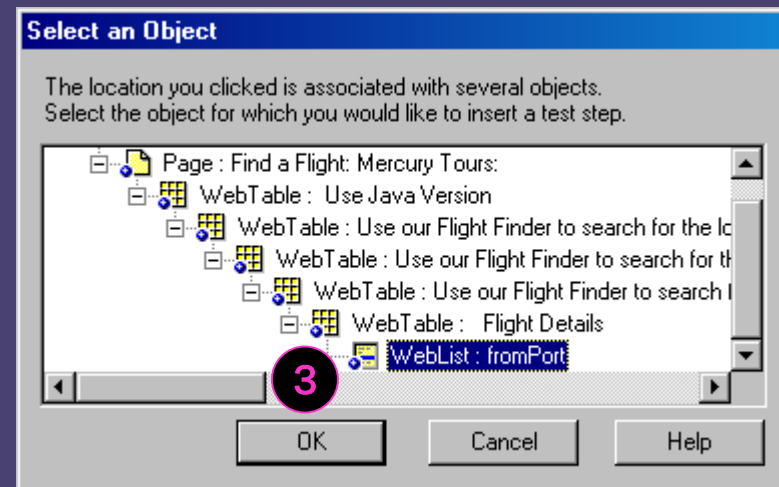
Passengers:

Departing From: 2

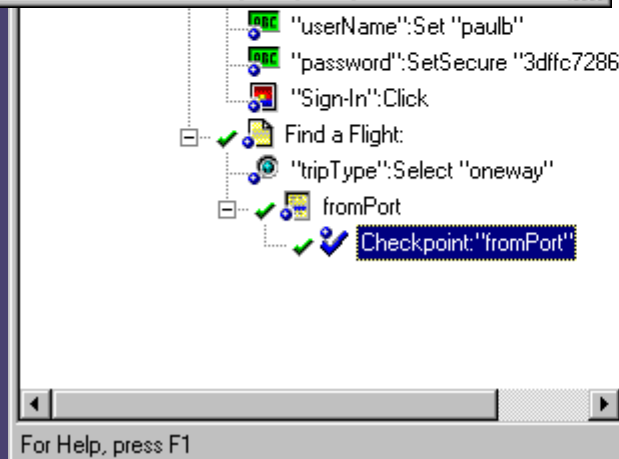
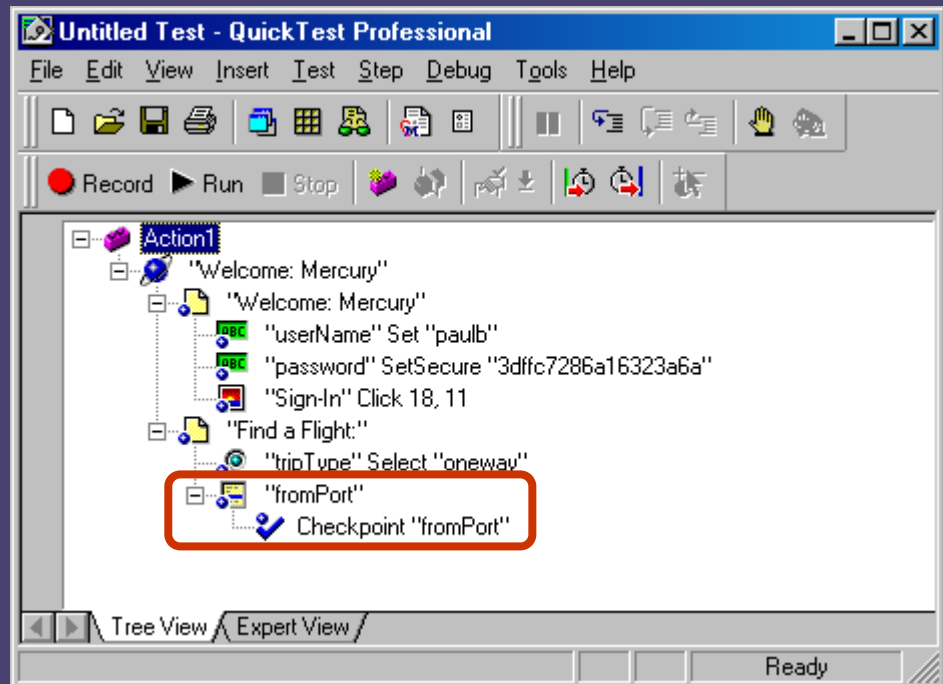
On:

Arriving In:

Returning:



Checkpoint in Script and Test Results



Checking fromPort: Passed

Details

fromPort Results	
Property Name	Property Value
html tag	SELECT
items count	10
name	fromPort
type	select-one
value	Acapulco

Other Commonly-Used Checkpoints

- Table checkpoint
 - Allows you to verify table content in rows and columns
 - Can be accessed from the Standard Checkpoints command
- Text checkpoint
 - Allows you to verify strings of text on a page
 - Is defined by the text surrounding the string to verify (right and left boundary) and the number of occurrences
- Automatic page checkpoints
 - Allows you to add checkpoints per page automatically during recording
 - Accessibility checkpoint checks conformance of the Web page to W3 standards
 - Automatic page checkpoints check for broken links, number of images, source code and links

Regular Expressions

Use a regular expression when you want to check for a value within a range of values

- A period (`.`) matches any single character
 - `welcome.` = welcomes or welcomed
 - `welcome.` ✗ welcoming
- An asterisk (`*`) matches zero to any number of occurrences of the preceding character
 - `a*b` = aab, aaaaab, b
- A plus (`+`) matches one to any number of occurrences of the preceding character
 - `a+b` = aab, aaaaab, ab
- A question mark (`?`) matches zero to one occurrence of the preceding character
 - `a?b` = b, ab
- Brackets [] match a single character within range
 - `197[0-9]` = 1971, 1977, 1970
 - `[A-Z][a-z]` = Ca, Mi, Oz

Web and Java Objects

- Standard checkpoints are based upon object properties

FLIGHT FINDER ✈️

Use our Flight Finder to search for the lowest fare on participating airlines. Once you find the flight you want, don't forget to visit the Mercury Tours website for more information on our vacation city.

☒ Round Trip ☐ One Way

2

San Francisco

Sep 13 View Calendar

Zurich

Sep 14 View Calendar

☒ Economy class
☐ Business class
☐ First class

No Preference

Checkpoint Properties

Logical name: toPort
 Class: WebList

Type	Property	Value
☒	name	toPort
☐	select type	ComboBox Select
☐	selected item index	9
☐	selected items count	1
☒	type	select-one
☒	value	Zurich

Edit value

☒ Constant Zurich

☐ Parameter

☒ Data Table ☐ Other

Parameter name: toPort_value

Checkpoint Properties

Logical name: View Calendar
 Class: JButton

Type	Property	Value
☒	label	View Calendar
☐	lightweight	false
☐	list_item_name	
☐	opaque	true
☐	path	Button;showCalendar;Applet
☐	popup	0

Edit value

☒ Constant View Calendar

☐ Parameter

☒ Data Table ☐ Other

Parameter name: View_Calendar_label

CONTINUE

ActiveX and Windows Objects

Checkpoint Properties

Logical name: MaskedTextBox
Class: ActiveX

Type	Property	Value
☒	FormattedText	12/12/04
☐	height	19
☐	HideSelection	True
☐	hWnd	197640
☐	Mask	99/99/99
☐	MaxLength	8

Edit value

☒ Constant 12/12/04

☐ Parameter

☒ Data Table ☐ Other

Parameter name: MaskedTextBox_FormattedText

Flight Reservation

File Edit Analysis Help

Flight Schedule:

Date of Flight: 12/12/04

Fly From: Los Angeles

Fly To: Denver

Order Information:

Name: Wayne Wax

Departure Time:

Arrival Time:

Class:

☐ First ☐ Business ☐ Economy

Flights ...

Insert Order **Update Order** Delete Order

Checkpoint Properties

Logical name: Update Order
Class: WinButton

Type	Property	Value
☒	enabled	True
☐	focused	False
☐	height	23
☒	text	&Update Order
☐	width	72
☐	x	247

Edit value

☒ Constant &Update Order

☐ Parameter

☒ Data Table ☐ Other

Parameter name: Update_Order_text

Creating Tests with Multiple Actions

Test Case Requires One User with Many Orders

TEST - New Order

Action 1

Login

Search for flights

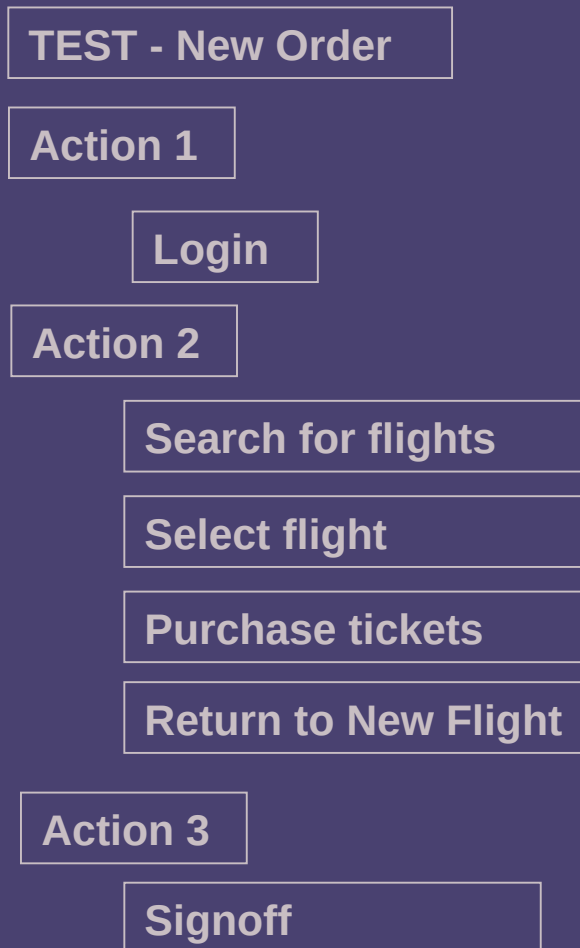
Select flight

Purchase tickets

Signoff

- How will you design your test to meet the requirements in QuickTest?
- What conditions should your test case have in order to fulfill one user with many orders?

When To Use Multiple Actions



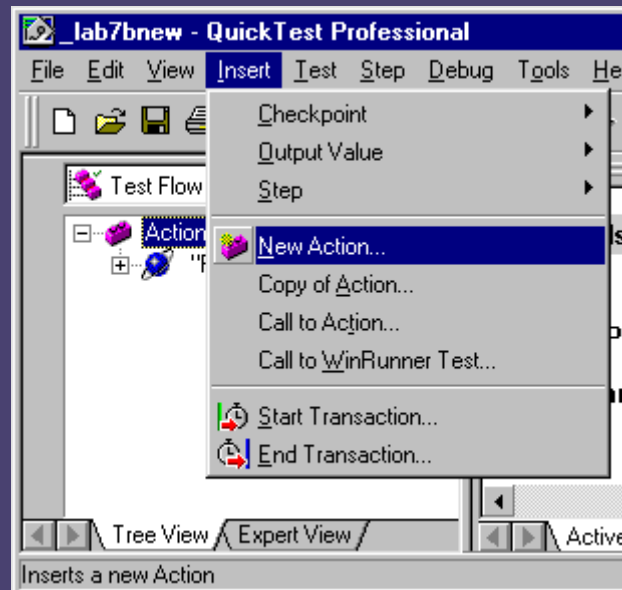
Use multiple actions in a test when a sub-process:

- Requires iterations within a complete transaction
- Is repeated within the test
- Is called by another test

Create Multiple Actions in a Test



Insert new actions during recording



Following your test case plan, insert a new action during recording.

TEST - New Order

Action 1

Login

Action 2

Search for flights

Select flight

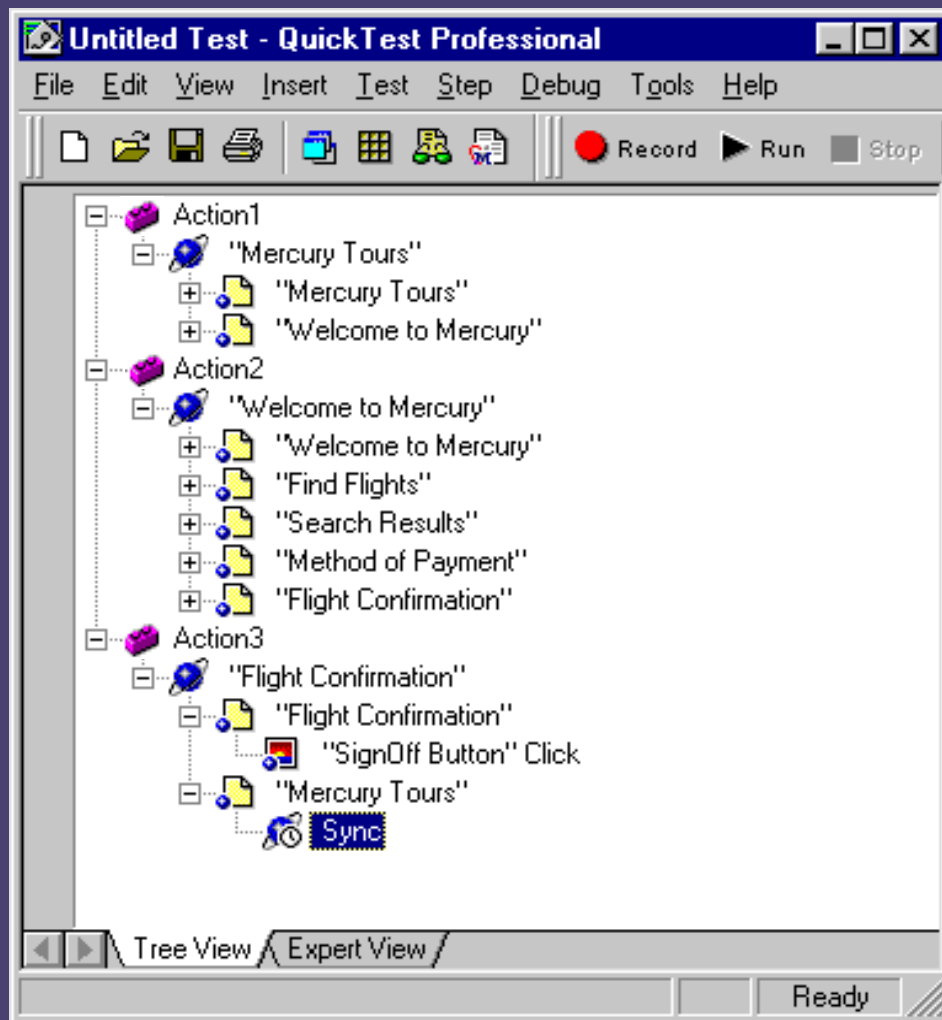
Purchase tickets

Return to New Flight

Action 3

Signoff

Multiple Actions in a Test



TEST - New Order

Action 1

Login

Action 2

Search for flights

Select flight

Purchase tickets

Return to New Flight

Action 3

Signoff

Correct Initial and End Conditions

Ensures proper playback for each iteration

Always end at this page to start clicking on the 'flights' button on next iteration

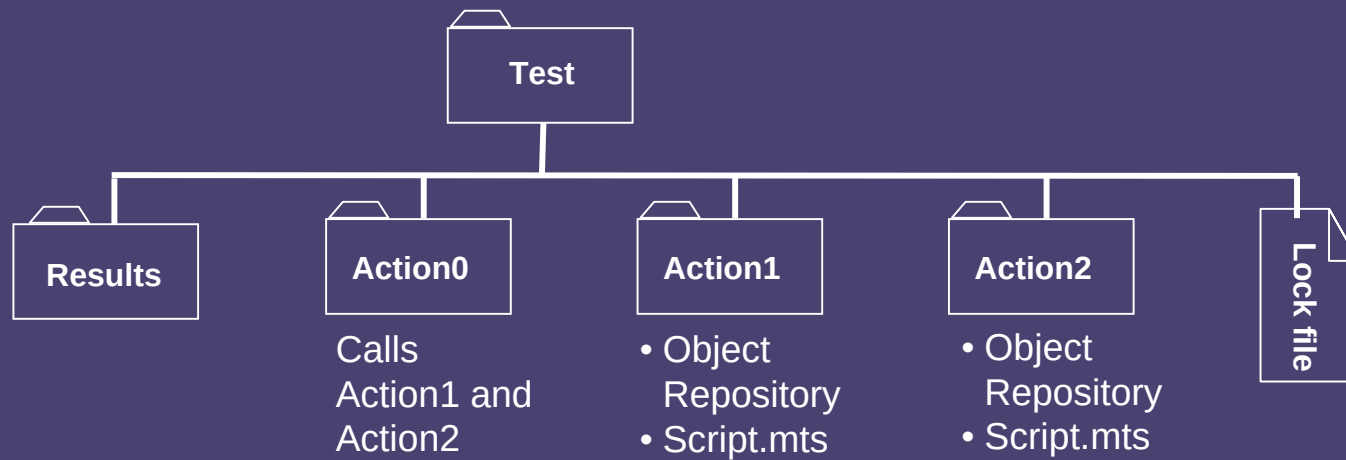
Always end at this page to start flights or itinerary transactions



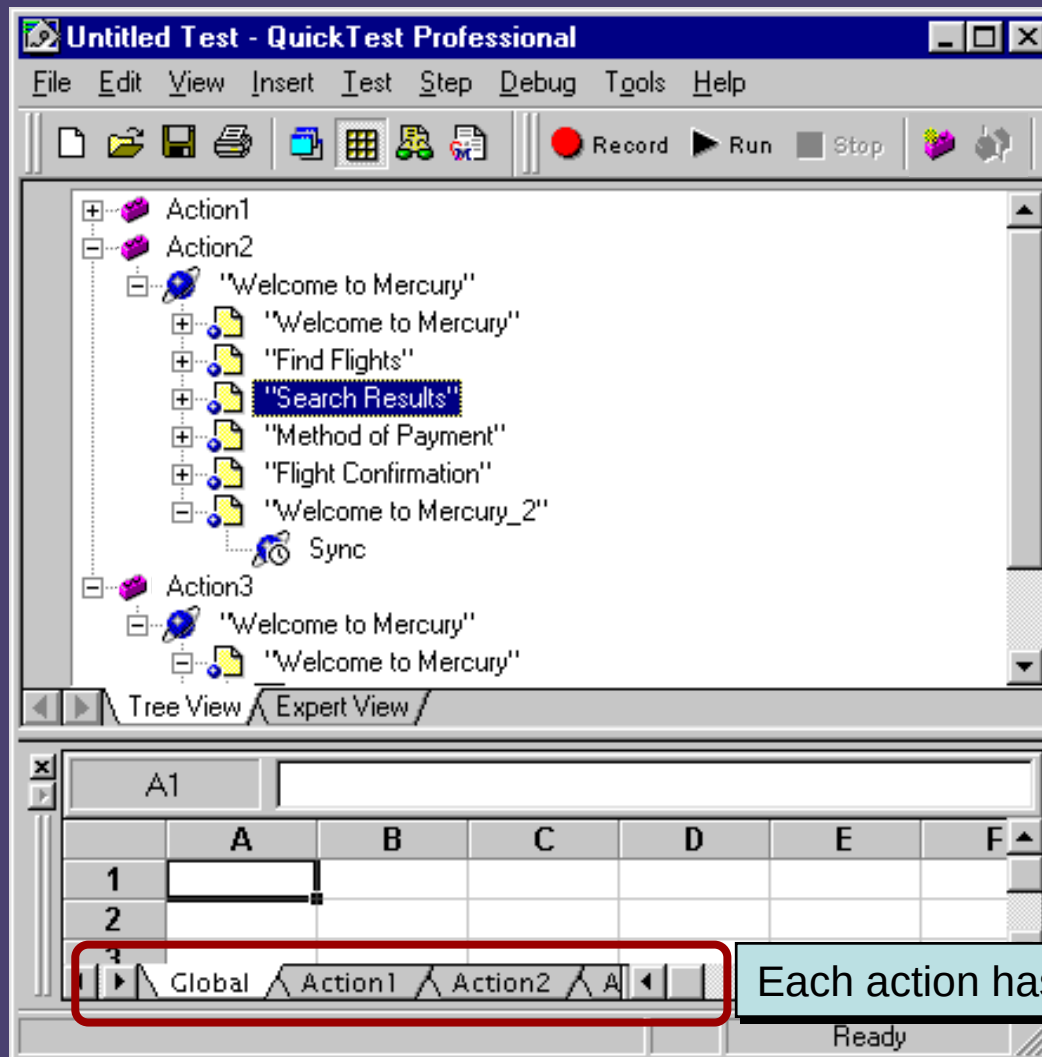
QuickTest Components Affected by Multiple Actions

Object Repository	One Object Repository per action
Data Table	Use the local Action data table
Run Settings	Set run setting for each action

One Object Repository Per Action



Use the Local Data Table (1)

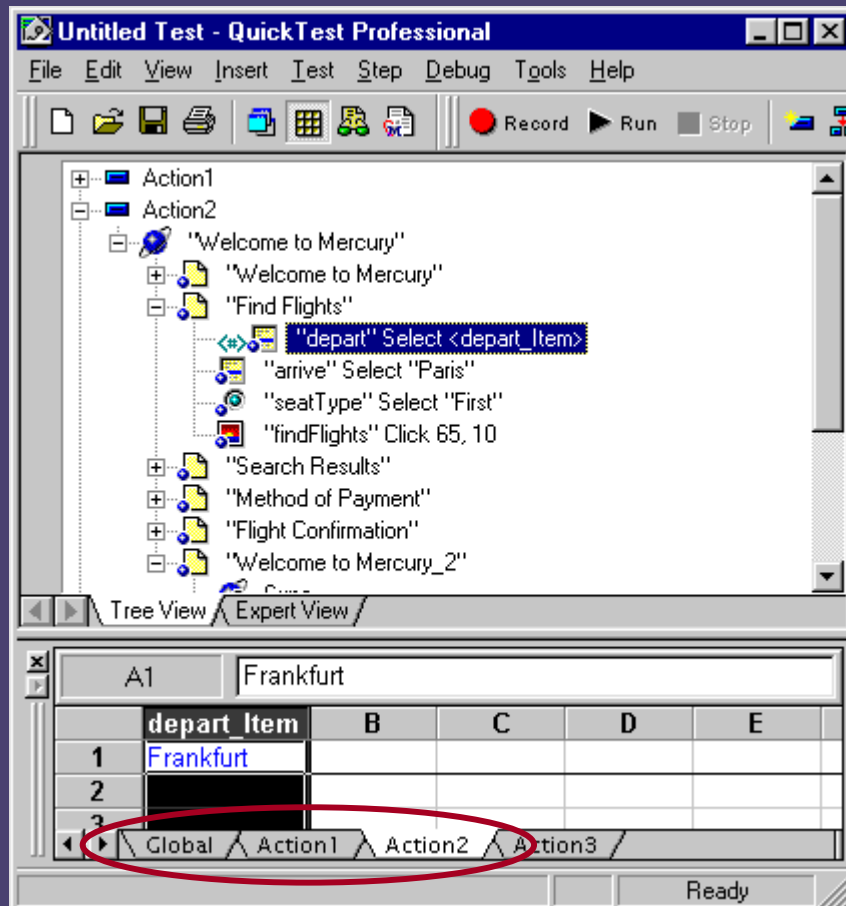


- Global data table iterates data on all actions
- Local data table iterates data only within the corresponding action

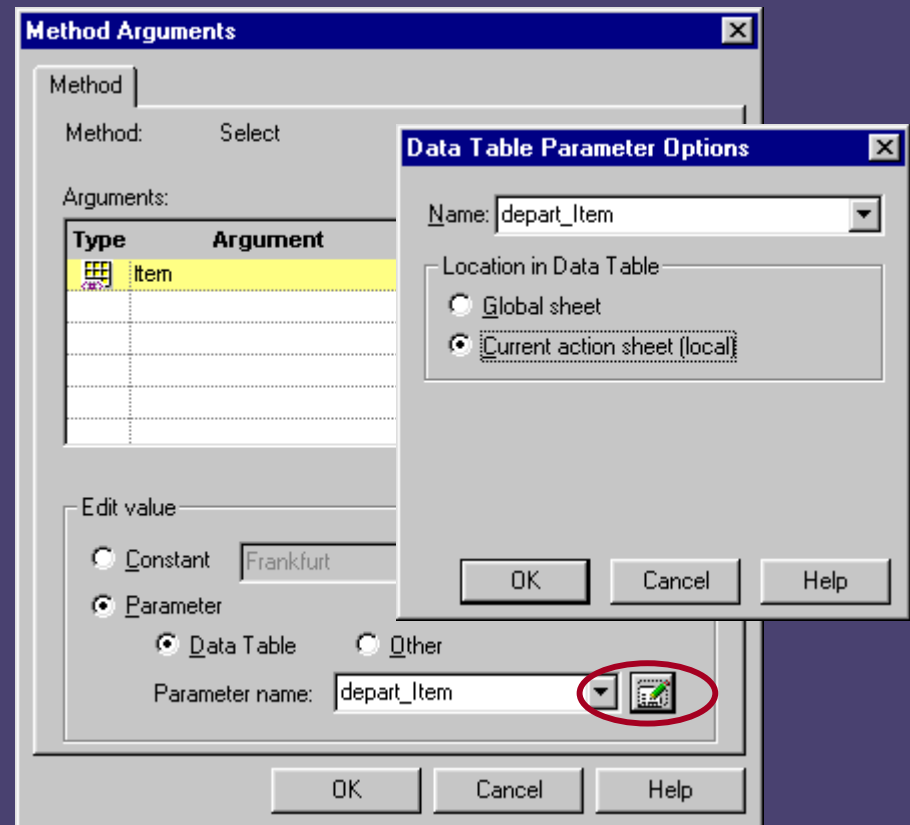
**MORE ABOUT THIS
TOPIC IN THE NEXT
LESSON**

Each action has its own local Data table

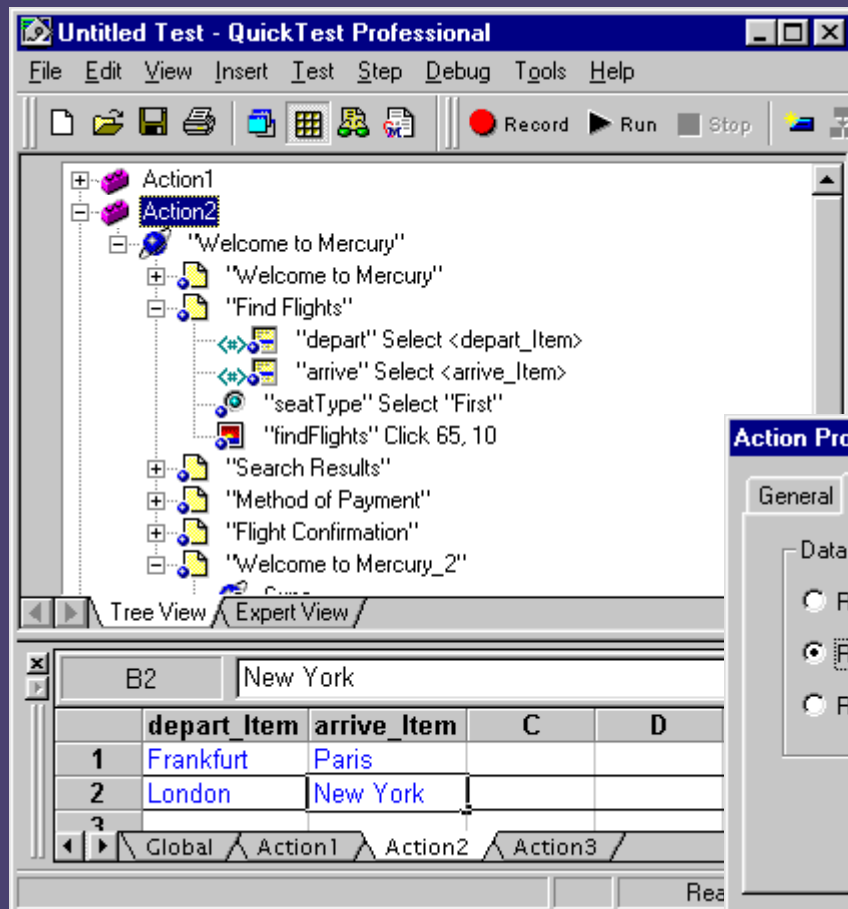
Use the Local Data Table (2)



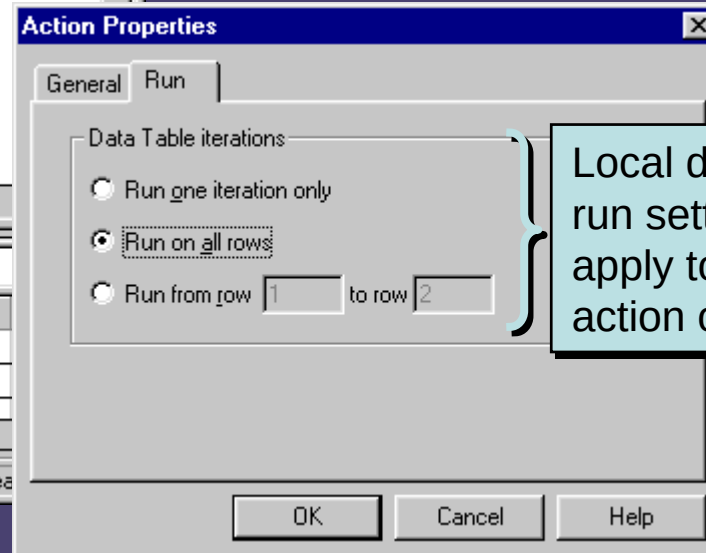
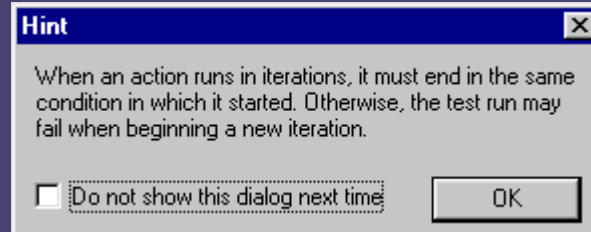
Use the local Data table when you want to iterate a specific action in your test



Set Run Settings for Each Action

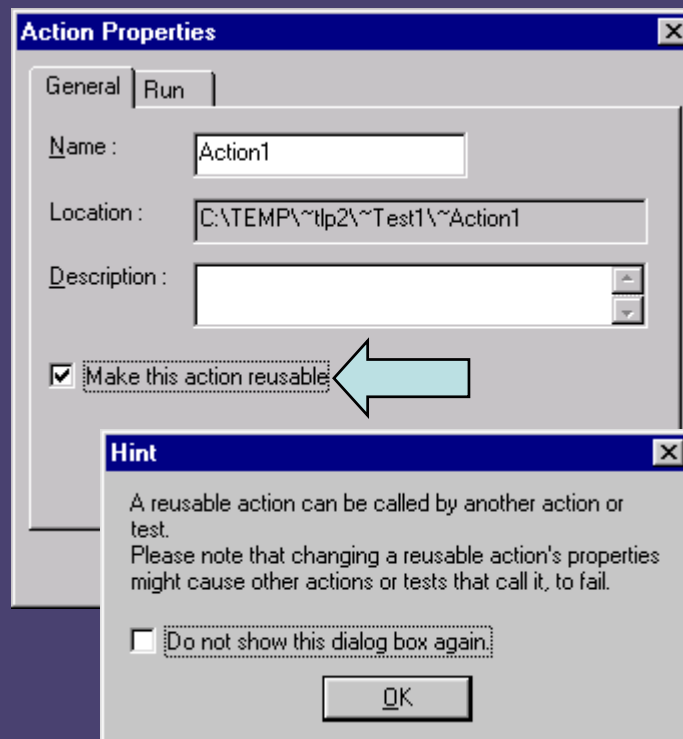
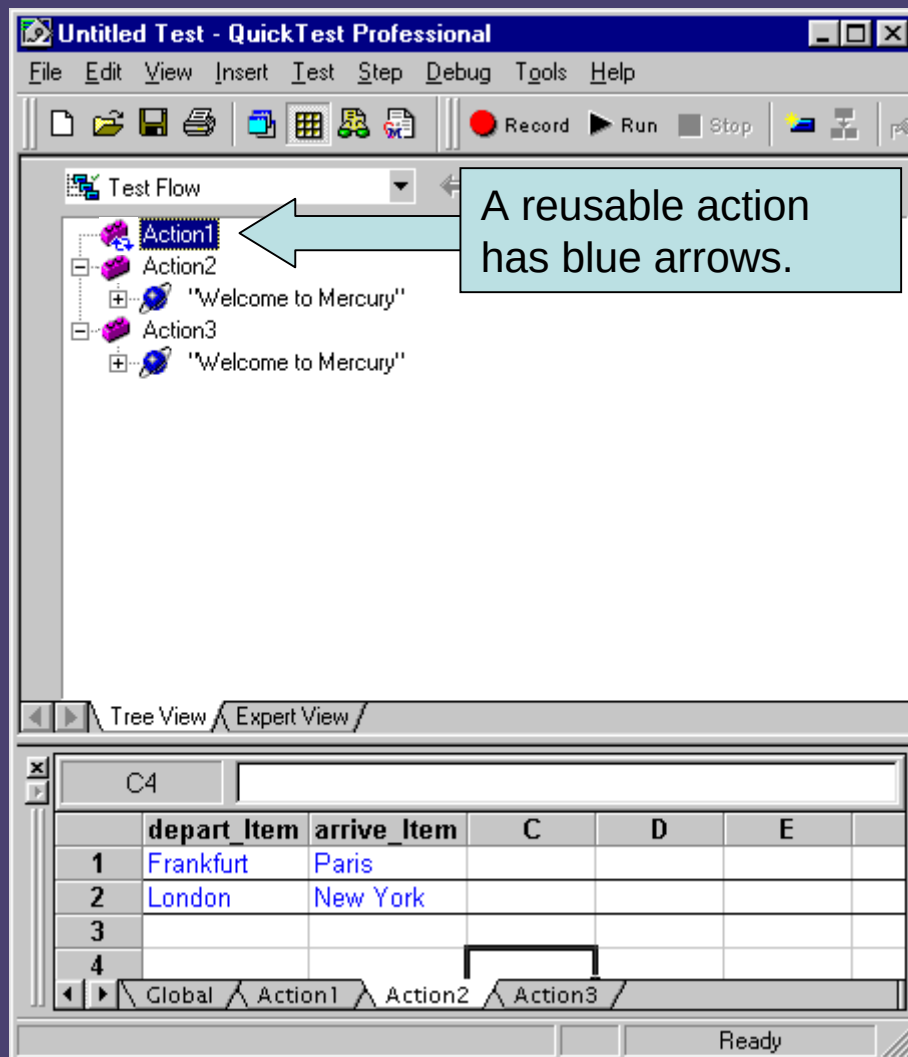


Remember initial and end conditions

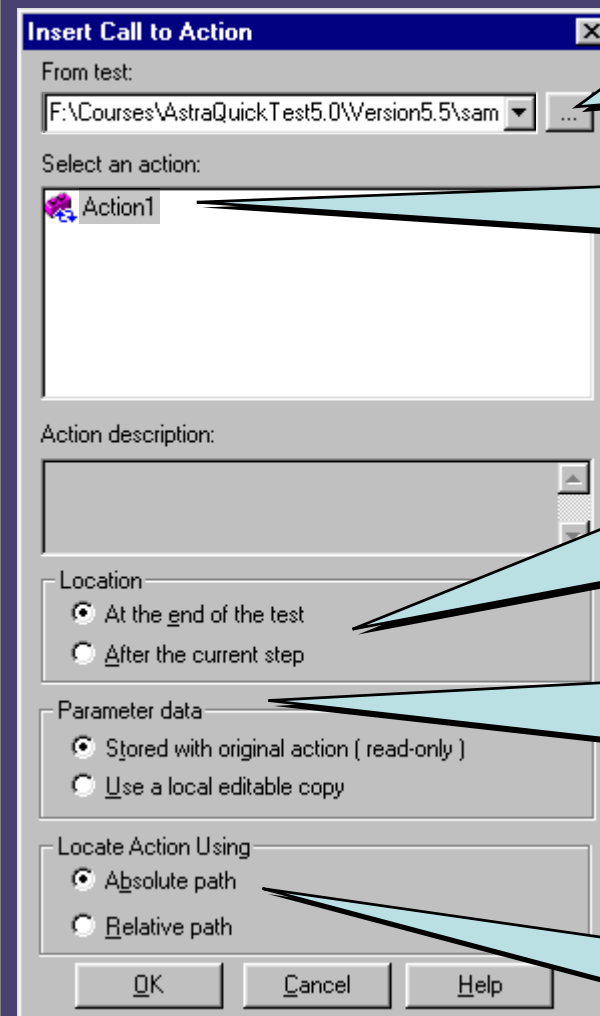
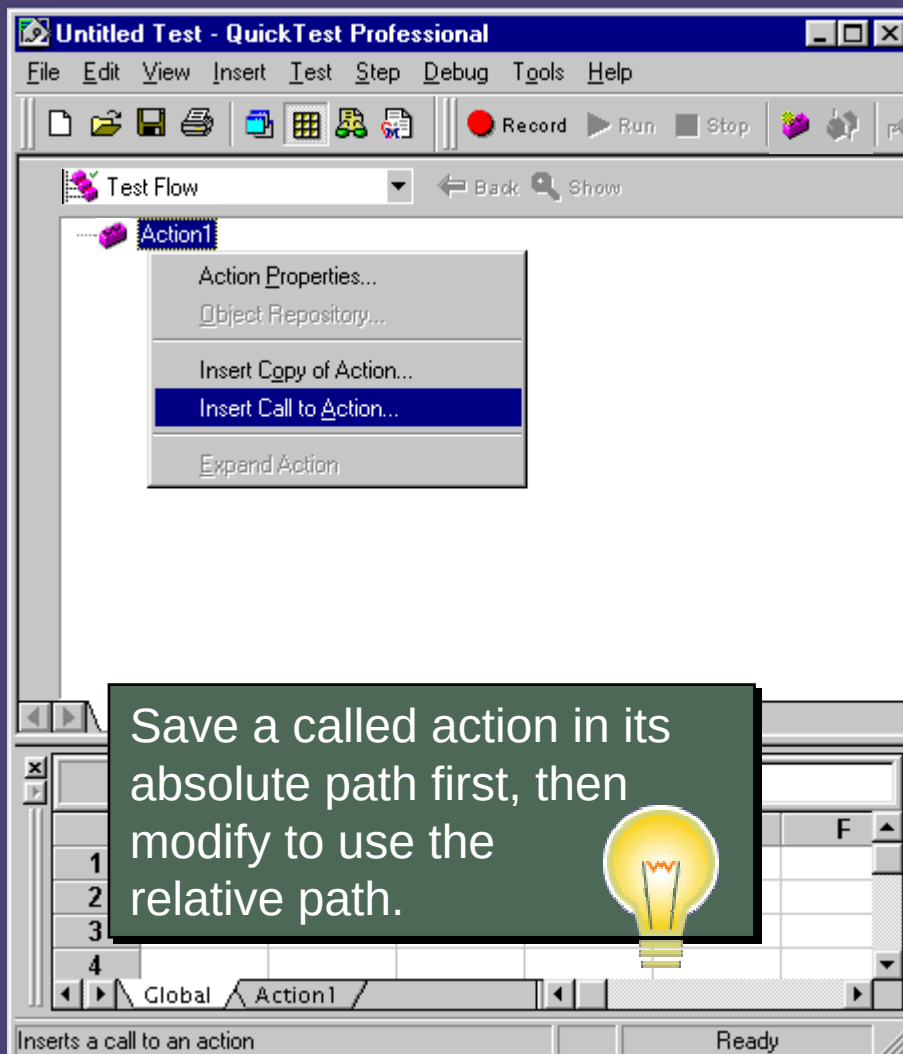


Local data table run settings will apply to specific action only

Set an Action as Reusable



Call an External Action (1)



Browse to the test location

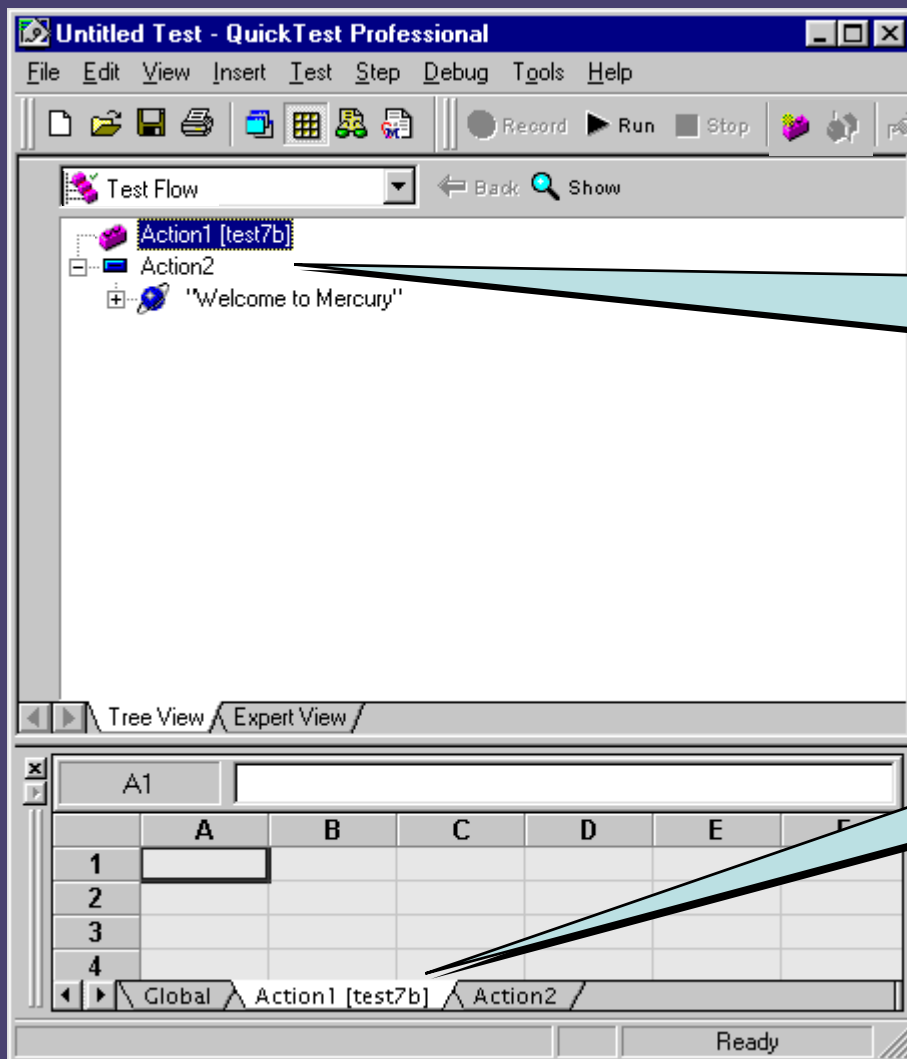
List all reusable actions

Specify where the action will be inserted

Specify how you want to use any stored data

Specify the path of the called test

Call an External Action (2)



You can insert a new action and start recording on the next steps in your test.

Call to action (read only) parameters – Local data table is grayed out

View Test Results

Any checkpoints or data used from the called action will be reported in the calling test's results.

The screenshot shows a software window titled "sample0 [Res1] - Test Results". The window has a menu bar with "File", "View", "Tools", and "Help". Below the menu is a toolbar with icons for file operations and help. The main area is divided into two panes. The left pane shows a hierarchical tree of test results. The right pane displays a summary table for "Test Iteration 1".

Test Iteration 1 Summary:

Iteration Passed

Object	Details	Result	Time
Action1 Summary		Passed	14:7:28
Action2 Summary		Done	14:7:35
Action3 Summary		Done	14:7:43

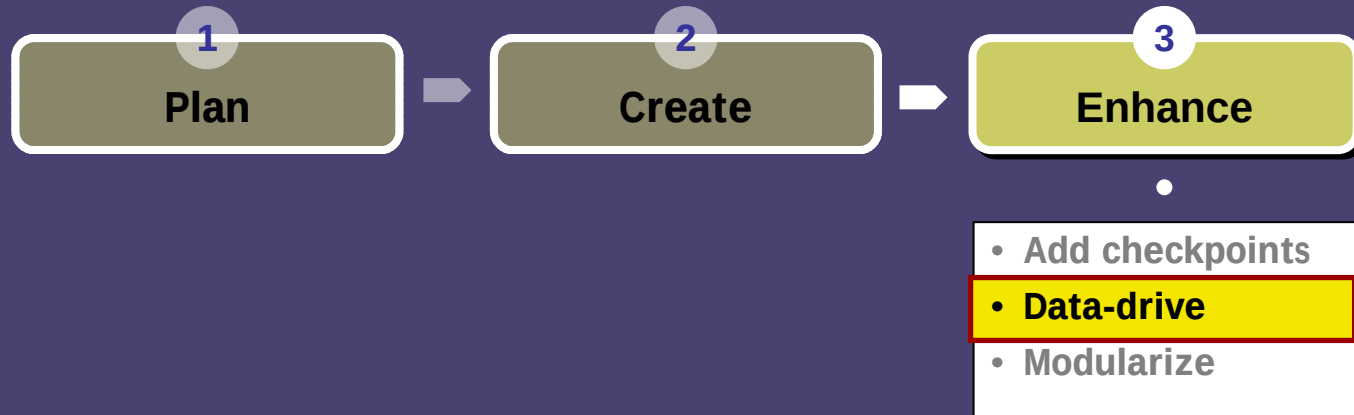
The left pane tree structure is as follows:

- Test sample0 Summary
 - Runtime Data
 - sample0 Iteration 1 (Row 1)
 - Action1 Summary
 - Action1 [2] Summary
 - Welcome: Mercury
 - Welcome: Mercury
 - logc userName
 - logc "userName":Set
 - logc "password":SetSecu
 - "Sign-In":Click
 - Find a Flight:
 - Action2 Summary
 - Action3 Summary

For Help, press F1

Data-Driving a Test

QuickTest Workflow



Definition

data-driven test

A test that runs one set of user actions with multiple input values.

Data driving allows one script to test AUT functionality with many sets of data.

Data Driving Means Less Test Development

- Create a new test for each set of data?

Test: Create Order 1		
	Description	Exp
Step 1	Select File; New	All
Step 2	Choose Fly From: Denver	De
Step 3	Choose Fly To: Los Angeles	Los

Test: Create Order 2		
	Description	Exp
Step 1	Select File; New	All
Step 2	Choose Fly From: Paris	Par
Step 3	Choose Fly To: London	Lon

Test: Create Order 3		
	Description	Exp
Step 1	Select File; New	All
Step 2	Choose Fly From: Frankfurt	Fre
Step 3	Choose Fly To: Sydney	Syd

- Create one data-driven test that uses a data source

Test: Create Order		
	Description	Exp
Note:	Repeat all steps for each row	
Step 1	Select File; New	All
Step 2	Choose Fly From: <fly-from>	<fly
Step 3	Choose Fly To: <fly-to>	<fly

fly-from	fly-to	date
Denver	Los Angeles	12/12/03
Paris	London	07/04/04
Frankfurt	Sydney	25/12/03

Benefits of Data-Driven Tests

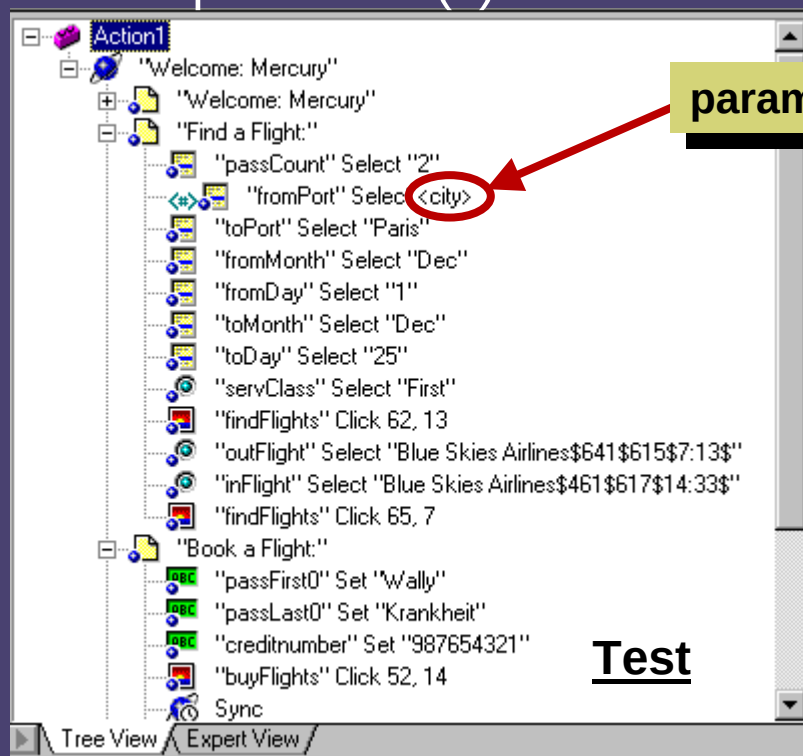
- Increased testing coverage
- Improves ROI in the testing tools
- Test boundary conditions
- Test good data and bad data
- Test different paths through the application code
- Test different tables in the database
- Test all windows/UI features in the application

Elements of a Data-Driven Test

- Recorded test script
- Parameter(s)
 - A parameter is a variable which replaces a recorded value in the script.
 - At run time, QuickTest substitutes a value from a data source for the parameter.
- Data source
 - The location of data that QuickTest will use to supply the parameter.
 - In QuickTest, the data source is called the Data table.
 - The Data table is a spreadsheet file that contains input values for a data driven test.

How a Data Driven Test Works

- Parameter(s) replace recorded input data
- Data table references tell QuickTest where to get data for the parameter(s)



parameter

The screenshot shows the 'Data Table' window with a table containing city names. The word 'city' is circled in red in the header row, and a red arrow points from the 'parameter' label to it. The table has columns A, B, and C, and rows 1 through 7.

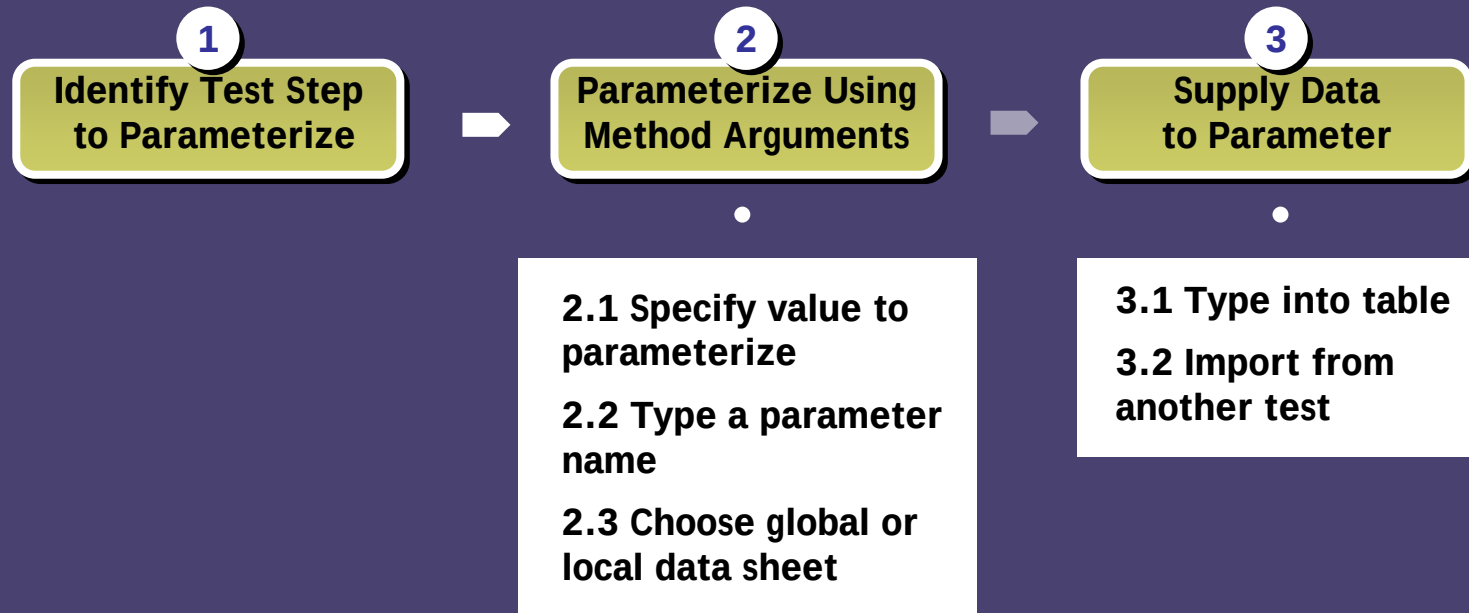
	A	B	C
1	Atlanta		
2	Berlin		
3	Cairo		
4	Lima		
5	Sydney		
6	Tokyo		
7			

Data Table

2 Ways to Data-Drive a Test

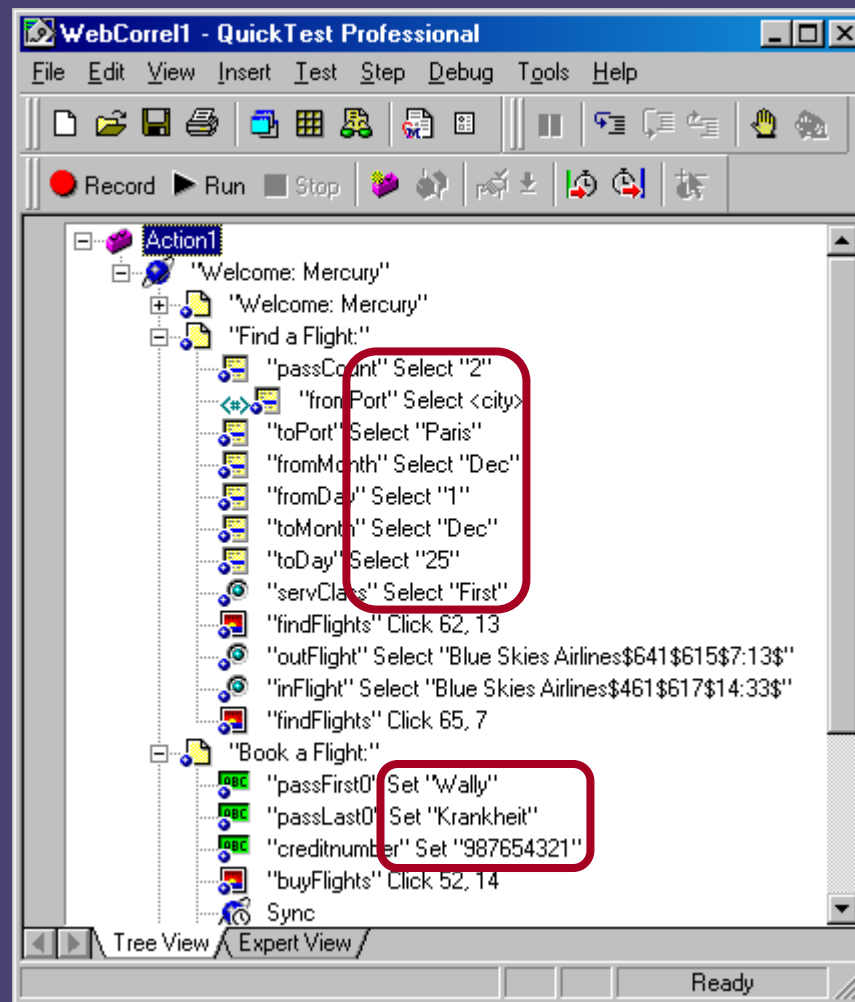
- From a dialog box — easier for one test step. Use:
 - Method Arguments dialog box
 - Object Properties dialog box
 - Object Repository
- Data Driver and Data Driver Wizard — easier for multiple steps

Parameterize One Input Step



Determine Recorded Values to Parameterize

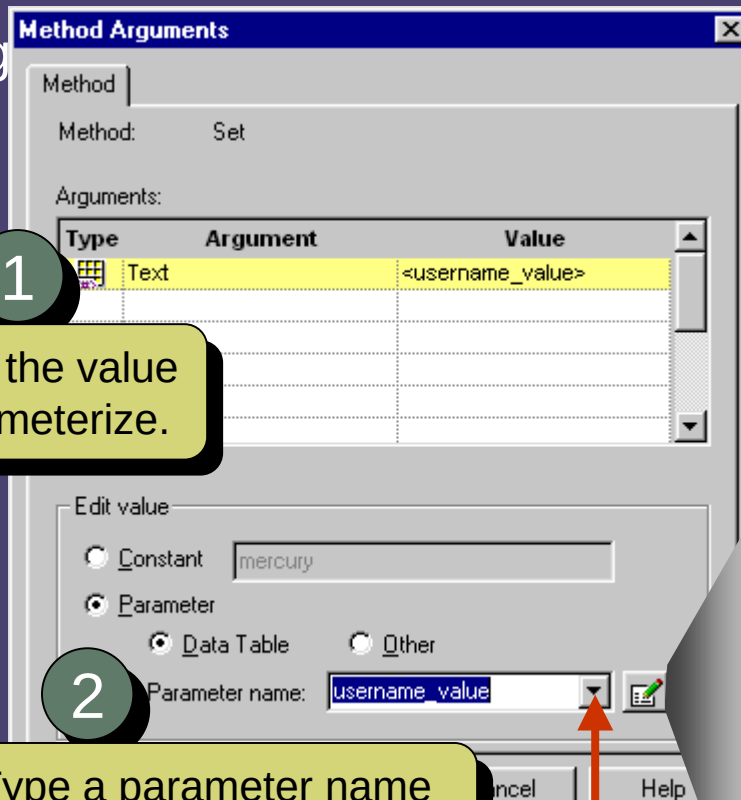
- "Select" represents selecting from a list, or clicking on an item
- "Set" represents typing something into a box from the keyboard



Select a Value and Options

- Select the input value to parameterize from the Tree View

- Right-click on the value to parameterize



The 'Method Arguments' dialog box is shown. The 'Method' is 'Set'. The 'Arguments' table has one row with 'Text' type and '<username_value>' value. The 'Edit value' section has 'Parameter' selected, with 'Data Table' and 'Other' sub-options. The 'Parameter name' is 'username_value'.

Type	Argument	Value
Text		<username_value>

Method: Set

Arguments:

Edit value:

☐ Constant mercury

☒ Parameter

☒ Data Table ☐ Other

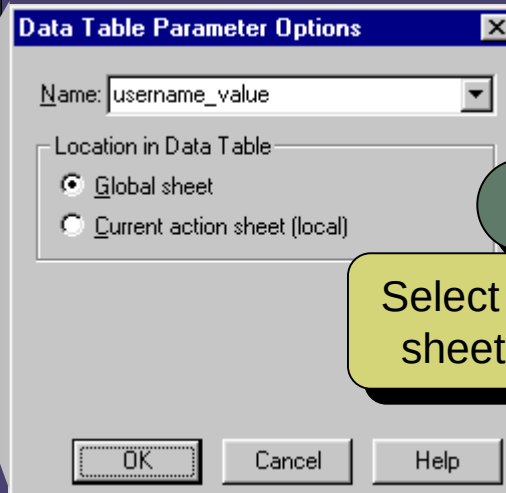
Parameter name: username_value

1

Specify the value to parameterize.

2

Type a parameter name or select one from the list



The 'Data Table Parameter Options' dialog box is shown. The 'Name' is 'username_value'. The 'Location in Data Table' section has 'Global sheet' selected.

Name: username_value

Location in Data Table:

☒ Global sheet

☐ Current action sheet (local)

OK Cancel Help

3

Select the data sheet to use.

3. Supply Data to the Parameter

Some of the easiest ways to supply data:

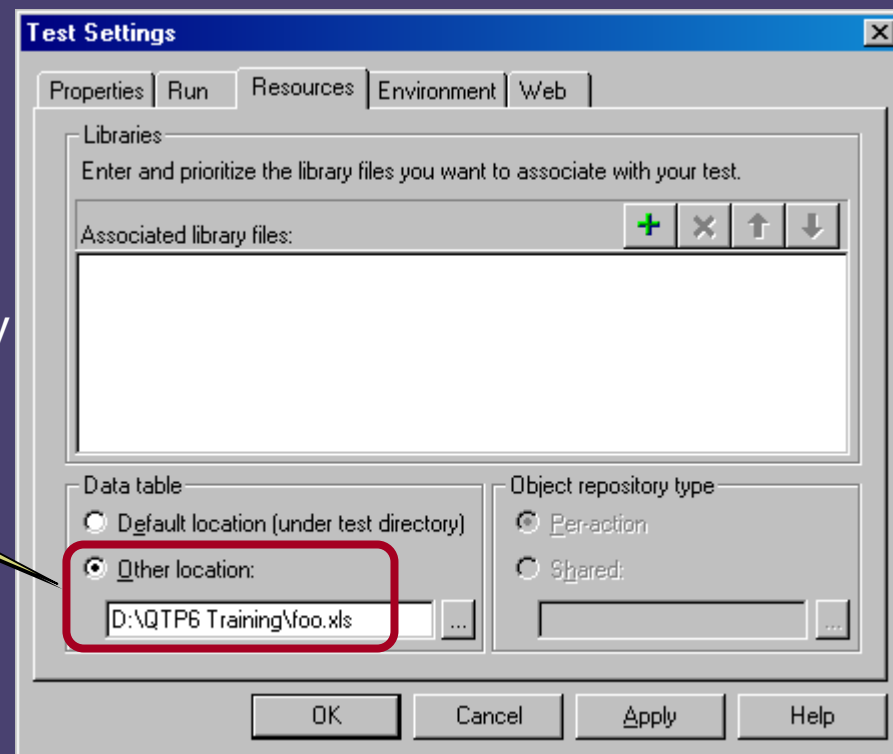
- **Manually place data into the data table**

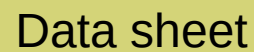
- type in data (Refer to the test case for data to use)
- copy data from an Excel file

- **Import data from another file**

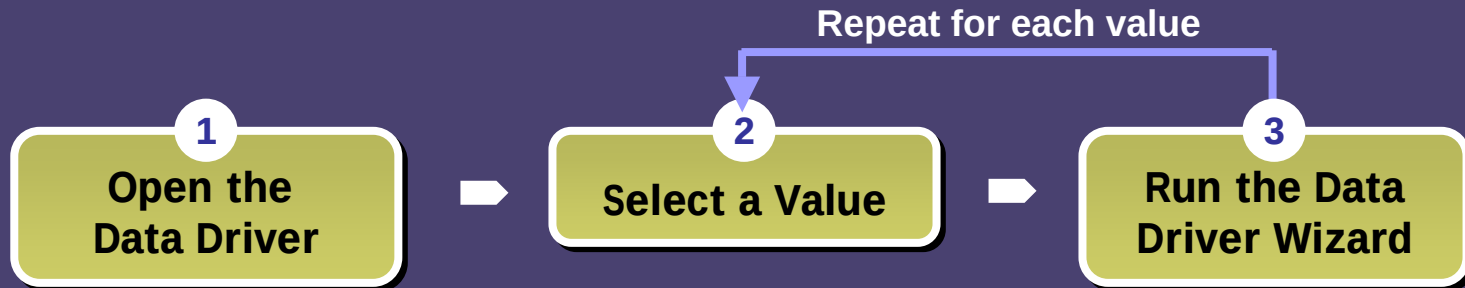
- another test's data table (if using one set of data for many tests)

Import



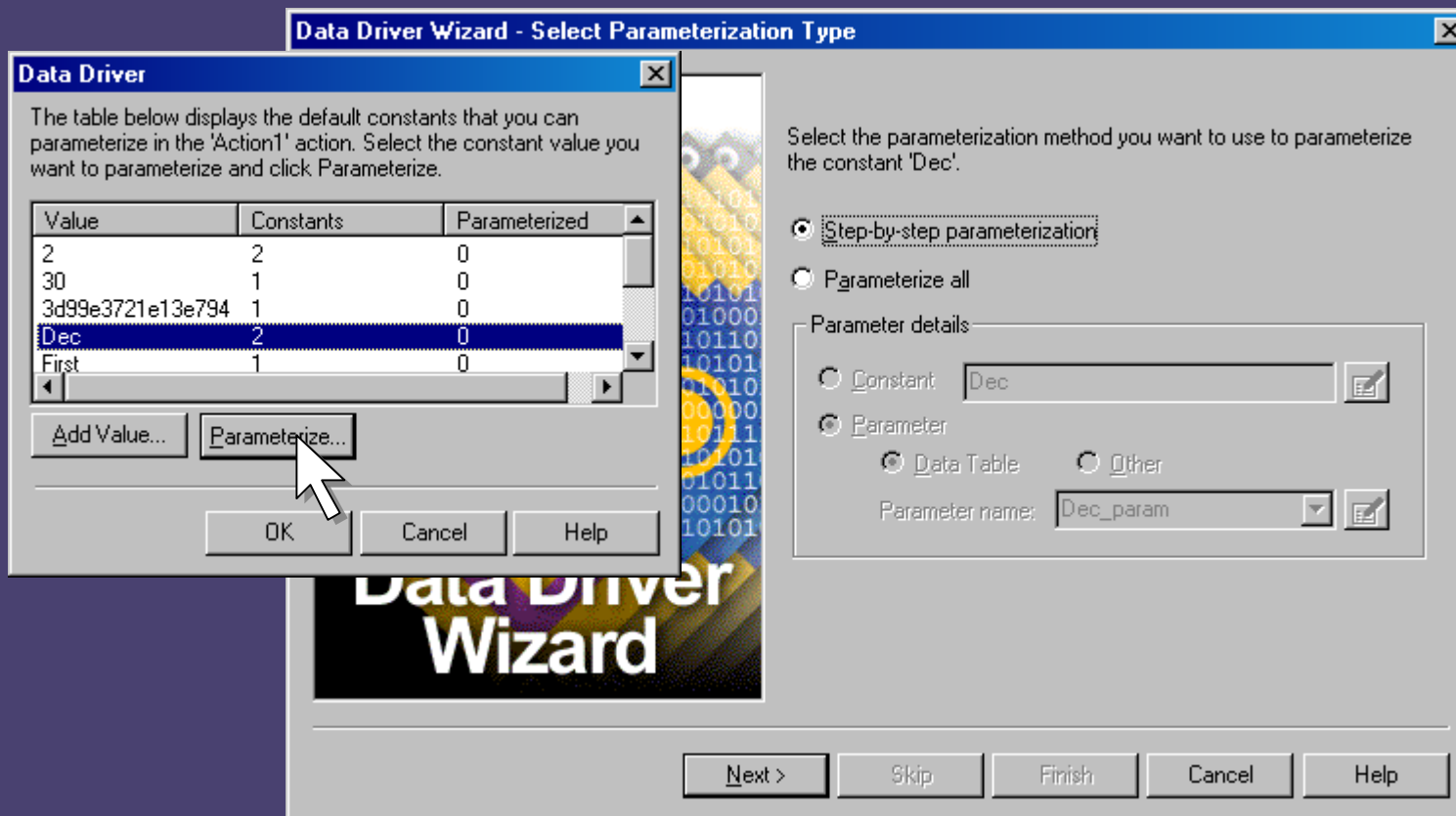
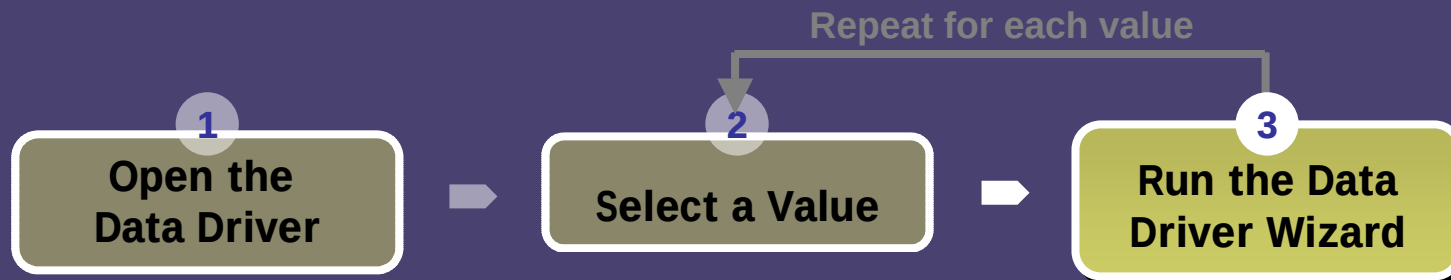


Parameterize Multiple Steps Using the Data Driver

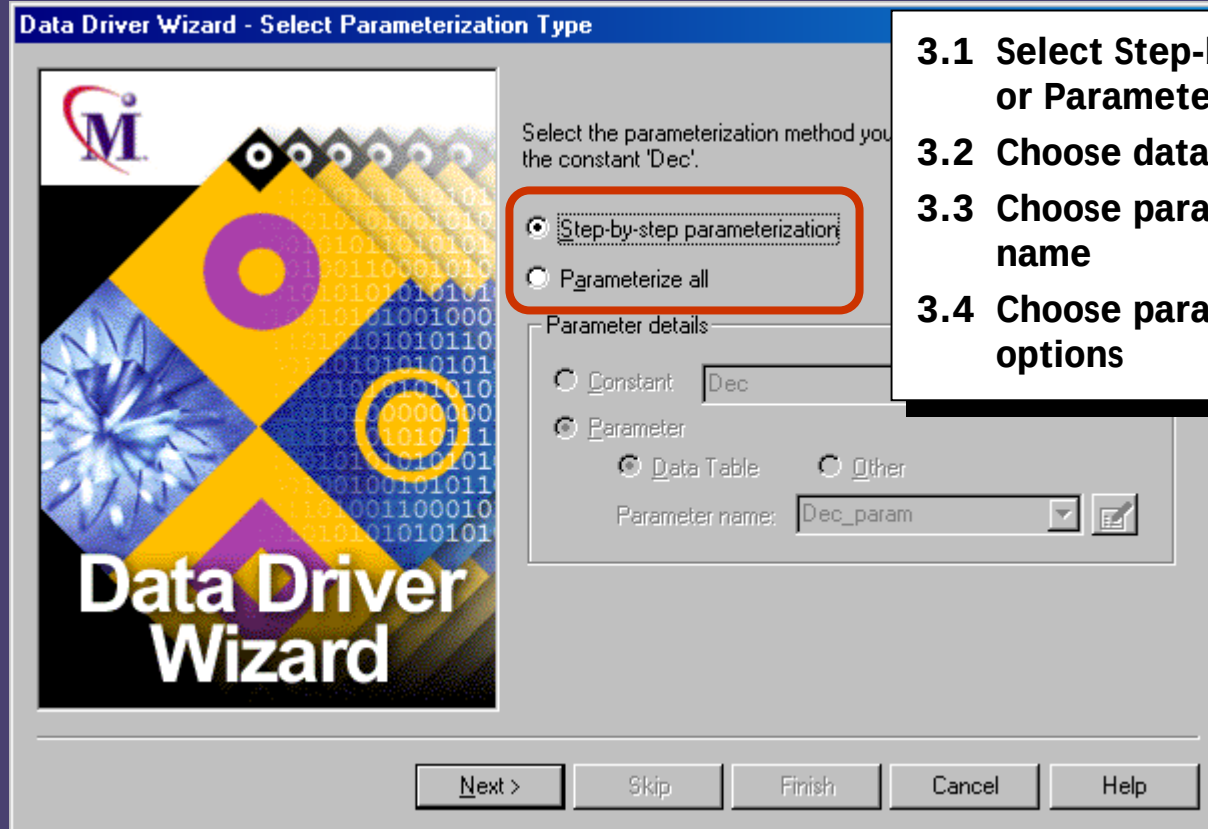
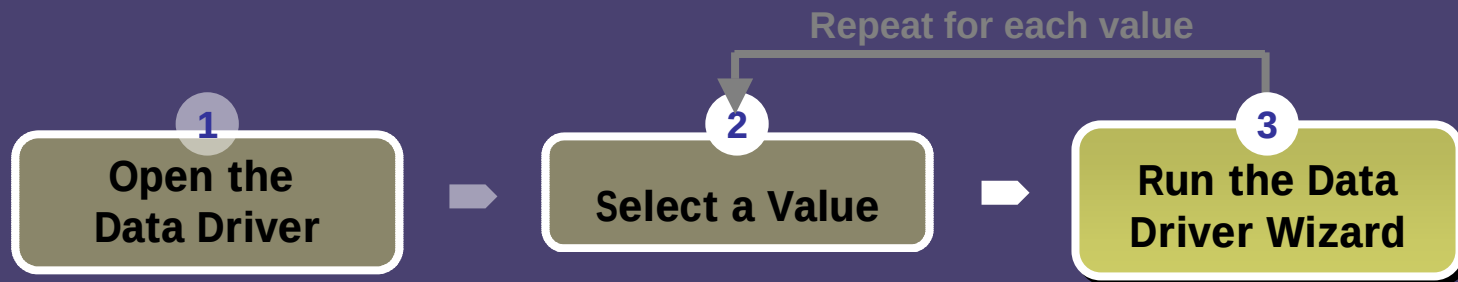


- 3.1 Select Step-by-step or Parameterize all
- 3.2 Choose data source
- 3.3 Choose parameter name
- 3.4 Choose parameter options

Open the Data Driver Wizard

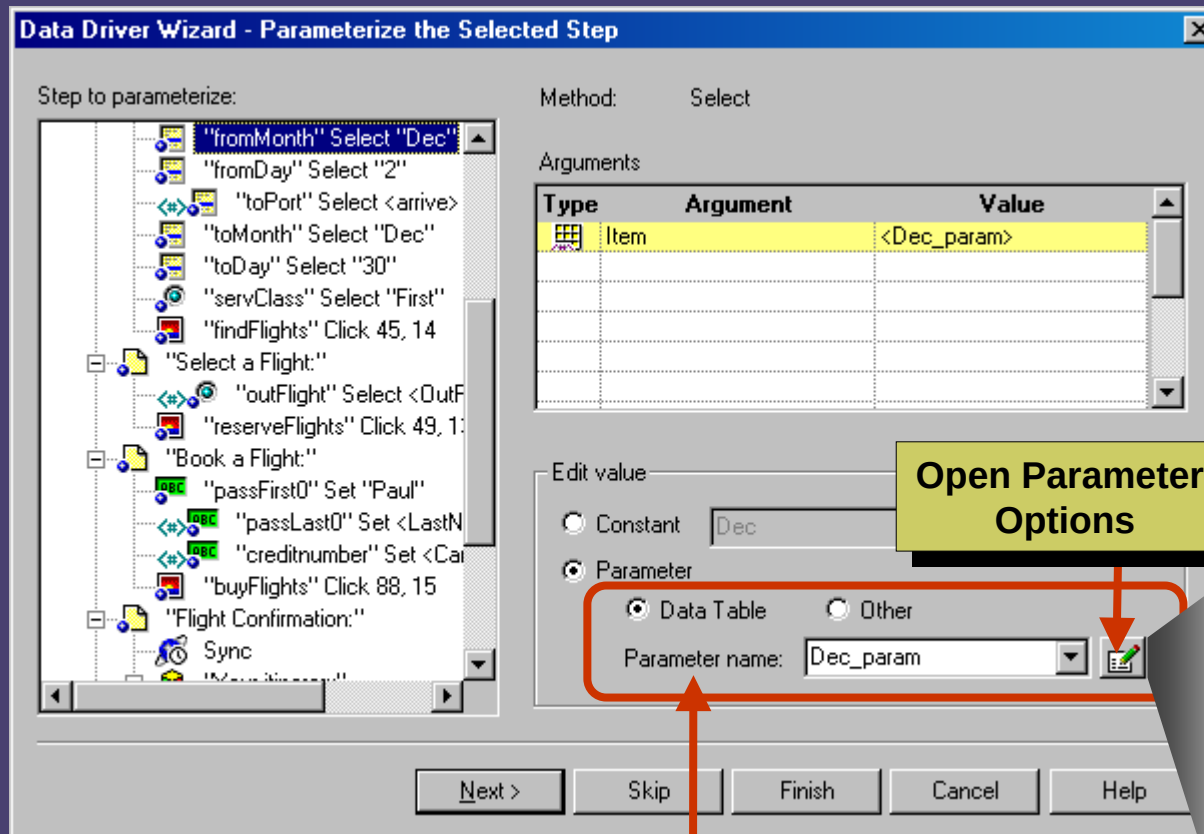


Choose Step-by-step or Parameterize All

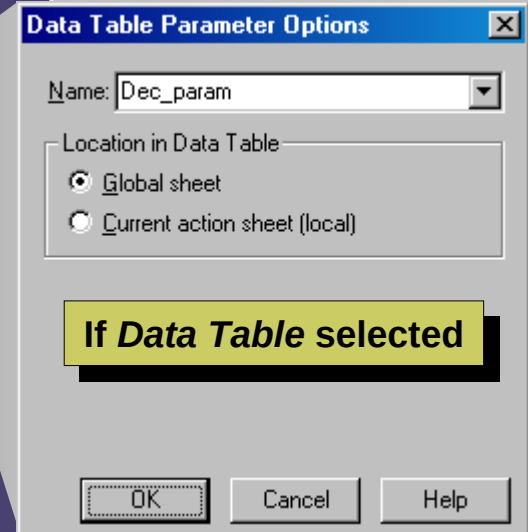
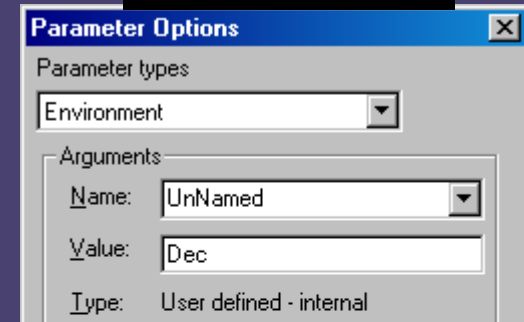


- 3.1 Select Step-by-step or Parameterize all
- 3.2 Choose data source
- 3.3 Choose parameter name
- 3.4 Choose parameter options

Step-by-step Parameterization



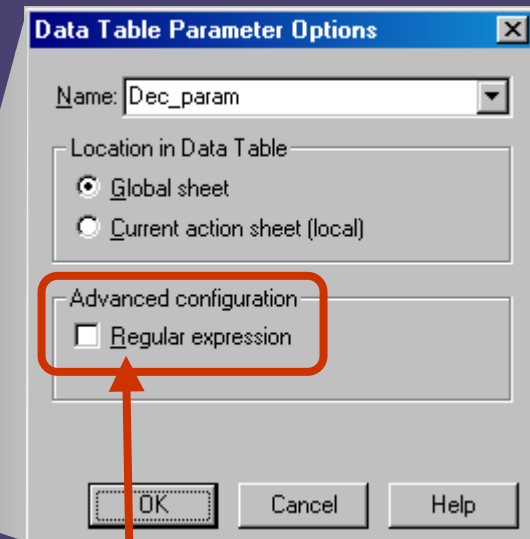
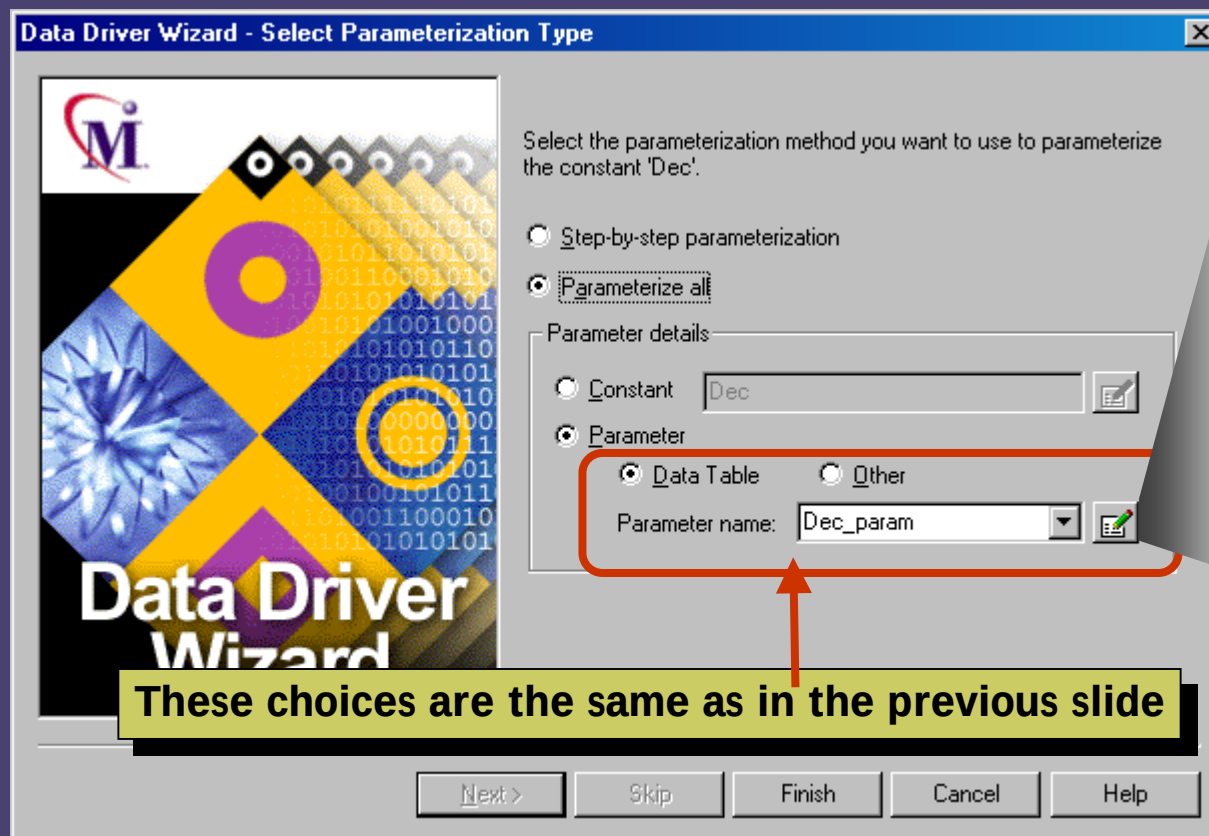
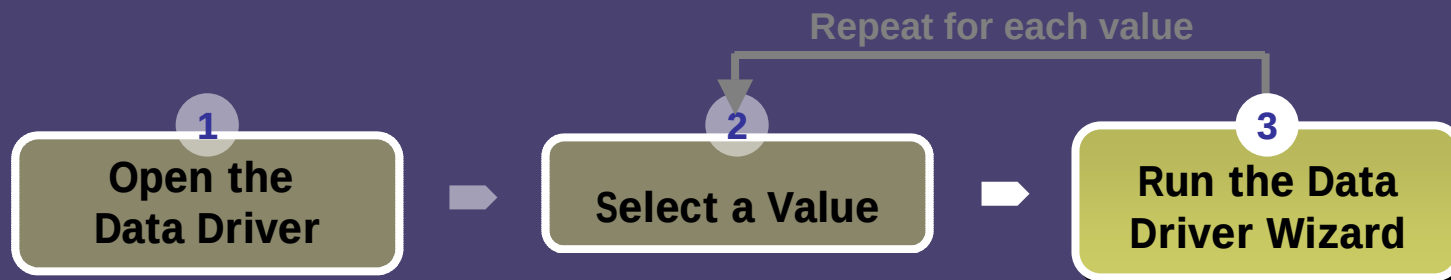
If Other selected



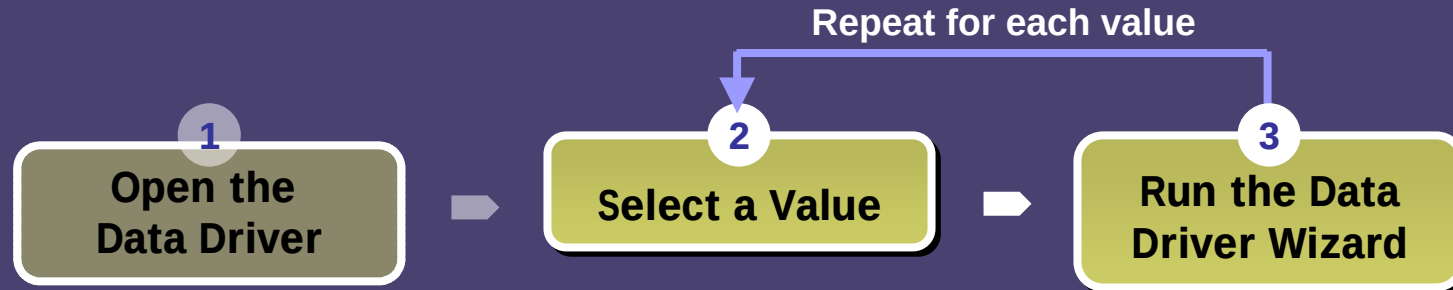
If Data Table selected

- 3.2 Choose Data Table or Other [data source]
- 3.3 Accept or modify Parameter name
- 3.4 Choose Parameter Options

Parameterize All



Finish, Then Choose Another Value



Data Driver Wizard - Select Parameterization Type

Select the parameterization method you want to use to parameterize the constant 'Dec'.

☐ Step-by-step parameterization

☒ **Parameterize all**

Parameter details:

☐ Constant

☒ **Parameter**

☒ **Data Table** ☐ Other

Parameter name:

Data Driver

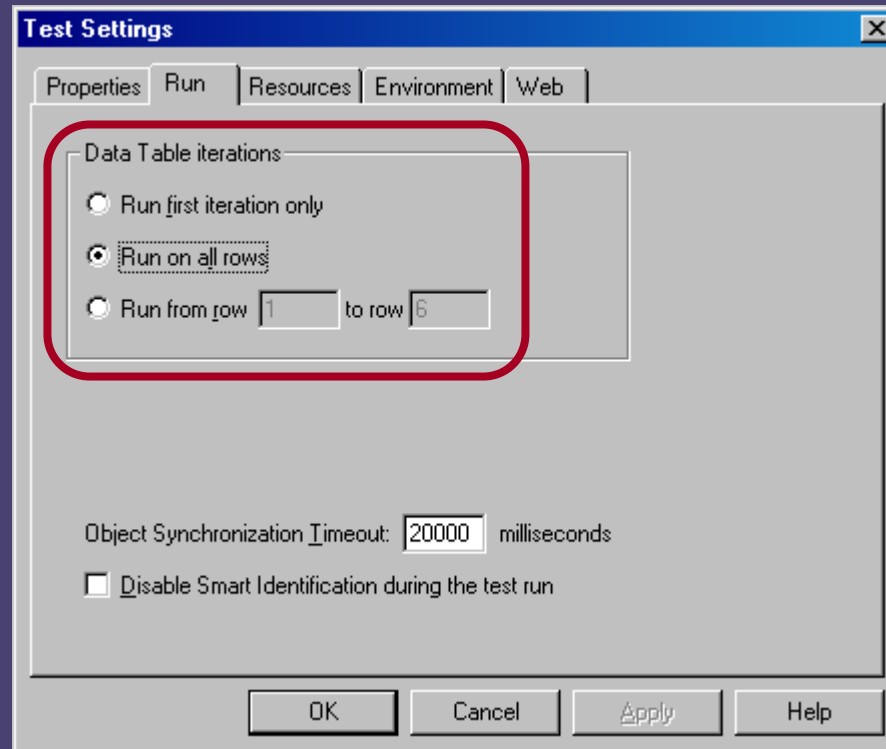
The table below displays the default constants that you can parameterize in the 'Action1' action. Select the constant value you want to parameterize and click Parameterize.

Value	Constants	Parameterized
2	2	0
30	1	0
3d99e3721e13e794	1	0
Dec	2	0
First	1	0

Click *Finish*, then choose another value to parameterize

Run Settings

- Run 1st iteration only (local sheet default)
- Run all rows (Global sheet default)
- Run from row *X* to row *Y* (consecutively)



Test Results

Data used for each iteration

The screenshot shows the 'WebVerif2 [Res4] - Test Results' window. The left pane displays a tree view with the following structure:

- Test WebVerif2 Summary
 - Run-Time Data
 - WebVerif2 Iteration 1 (Row 1)
 - WebVerif2 Iteration 2 (Row 2)
 - WebVerif2 Iteration 3 (Row 3)

The right pane displays a table with the following data:

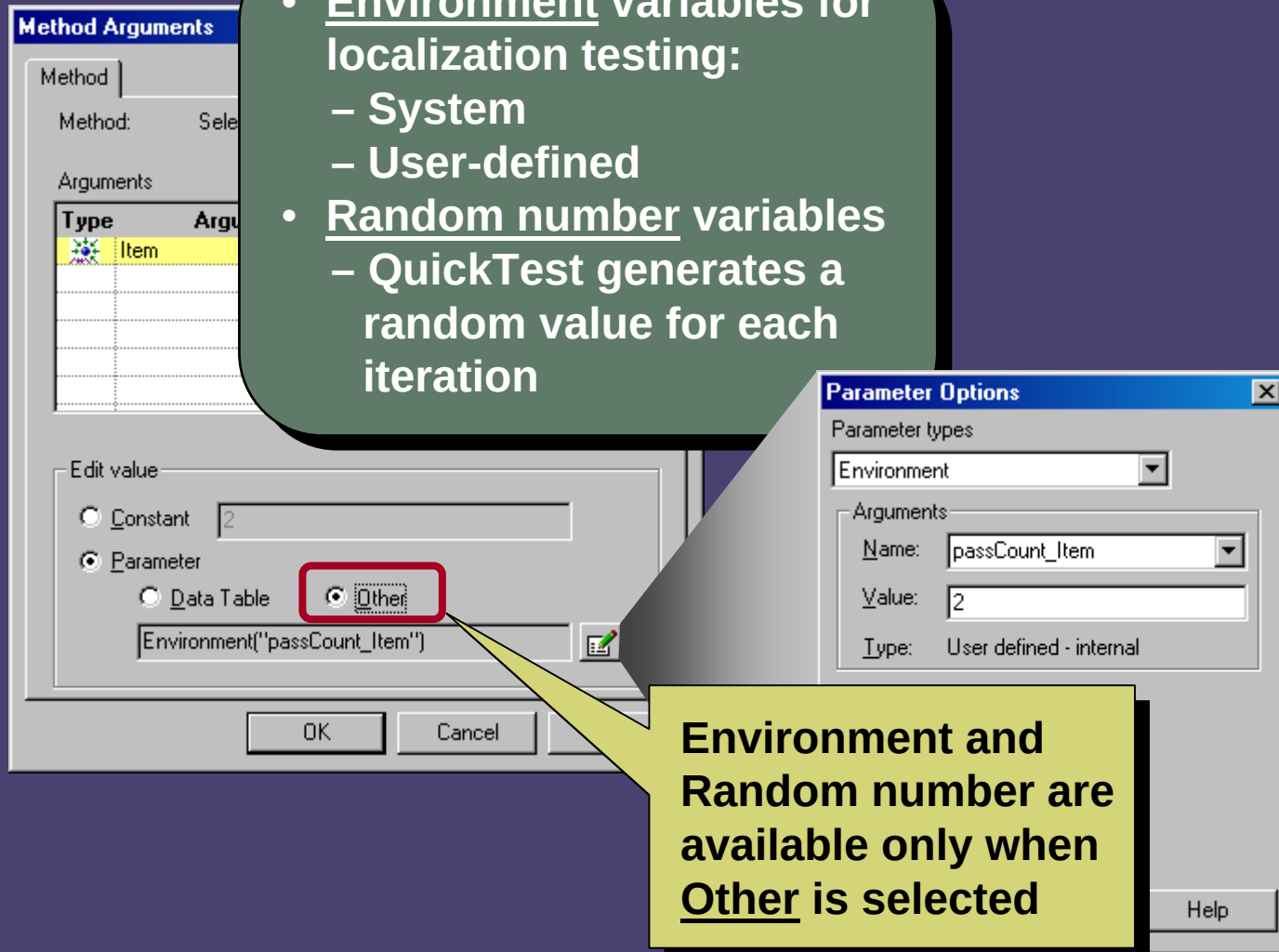
	fromCity	toCity	C
1	Zurich	London	
2	Paris	New York	
3	Acapulco	Sydney	
4			
5			
6			
7			
8			
9			
10			

An orange arrow points from the text 'Data used for each iteration' to the 'Run-Time Data' folder in the tree view. A yellow callout box points to the iteration items in the tree view.

Separate test results for each iteration

Other Types of Parameters

- Environment variables for localization testing:
 - System
 - User-defined
- Random number variables
 - QuickTest generates a random value for each iteration



Data Dependency Clues in Test Results

The screenshot shows a software window titled "WinDD2 [Res3] - Test Results". The window has a menu bar with "File", "View", "Tools", and "Help". Below the menu bar is a toolbar with various icons. The left pane shows a tree view with the following items:

- Test WinDD2 Summary (selected)
- Run-Time Data
 - WinDD2 Iteration 1 (Row 1) (passed)
 - WinDD2 Iteration 2 (Row 2) (failed)
 - WinDD2 Iteration 3 (Row 3) (failed)

The right pane displays the "WinDD2 Results Summary". It includes the following text:

Test: WinDD2
Results Name : Res3
Run started: 10/29/2002 - 13:12:42
Run ended: 10/29/2002 - 13:13:22

Below the text are two tables. The first table shows the results for each iteration:

Iteration #	Results
<u>1</u>	Passed
<u>2</u>	Failed
<u>3</u>	Failed

The second table shows the status and times for each iteration:

Status	Times
Passed	3
Failed	2
Warnings	2

At the bottom of the window, there is a status bar with the text "For Help, press F1" and a "Ready" label.

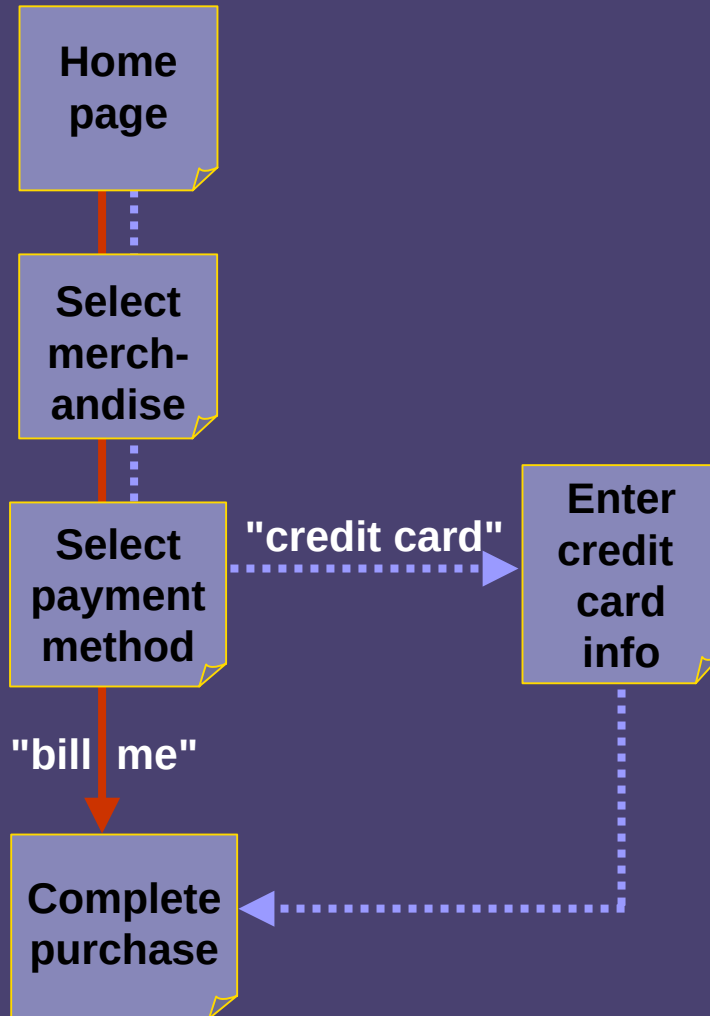
Multiple Paths

AUT

Path 1

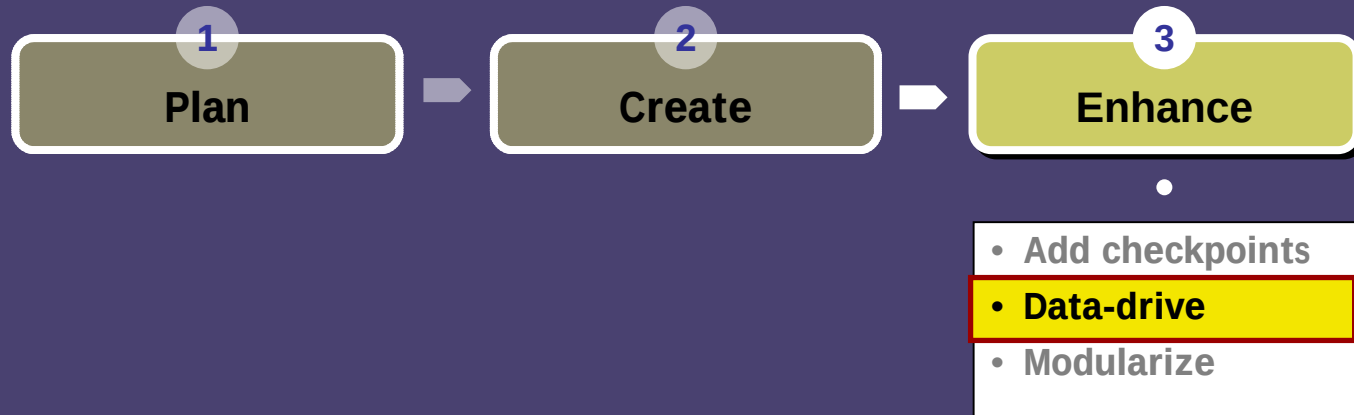


Path 2



Output & Correlation

QuickTest Workflow



Definition

correlated data

Data that is output from one object and used as input to another object.

Input and output objects need not belong to the same business process.

Benefits from Correlating Data

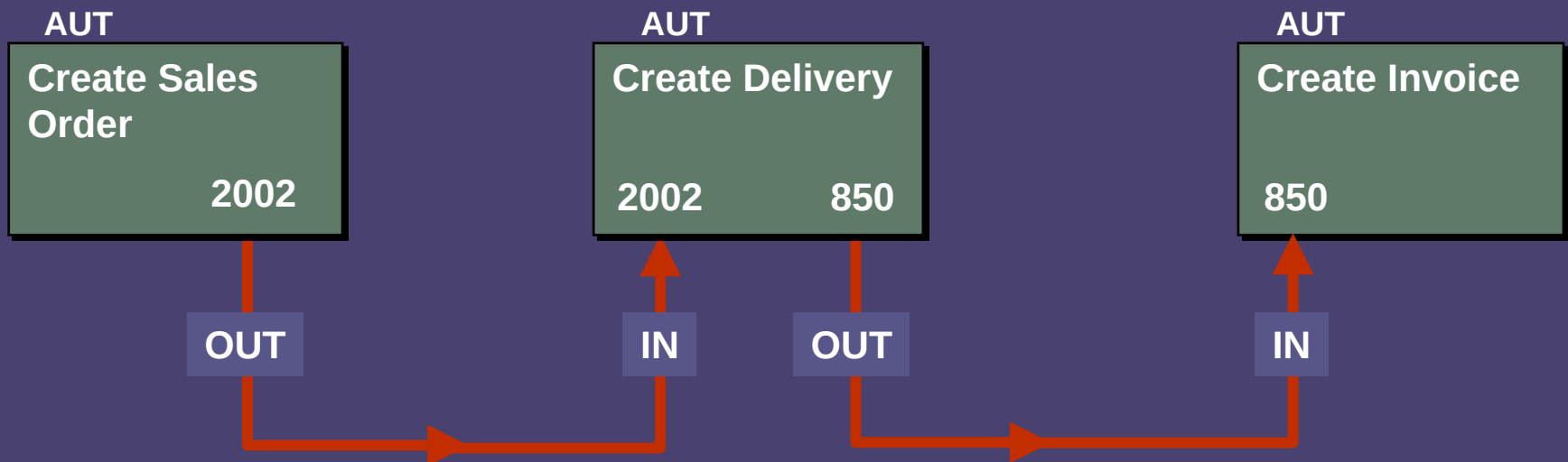
- Reduce run errors by preserving data dependencies
- Capture and reuse dynamic data that's unknown before run-time
- Link business processes together into business scenarios
- Make testcases more versatile and reusable

Reduce Run Errors

- Some applications/servers correlate data automatically
- In a recorded script, reused outputs can be hard-coded
- Output data is not captured automatically during a script run
- Script may not run correctly if data that should be correlated is not

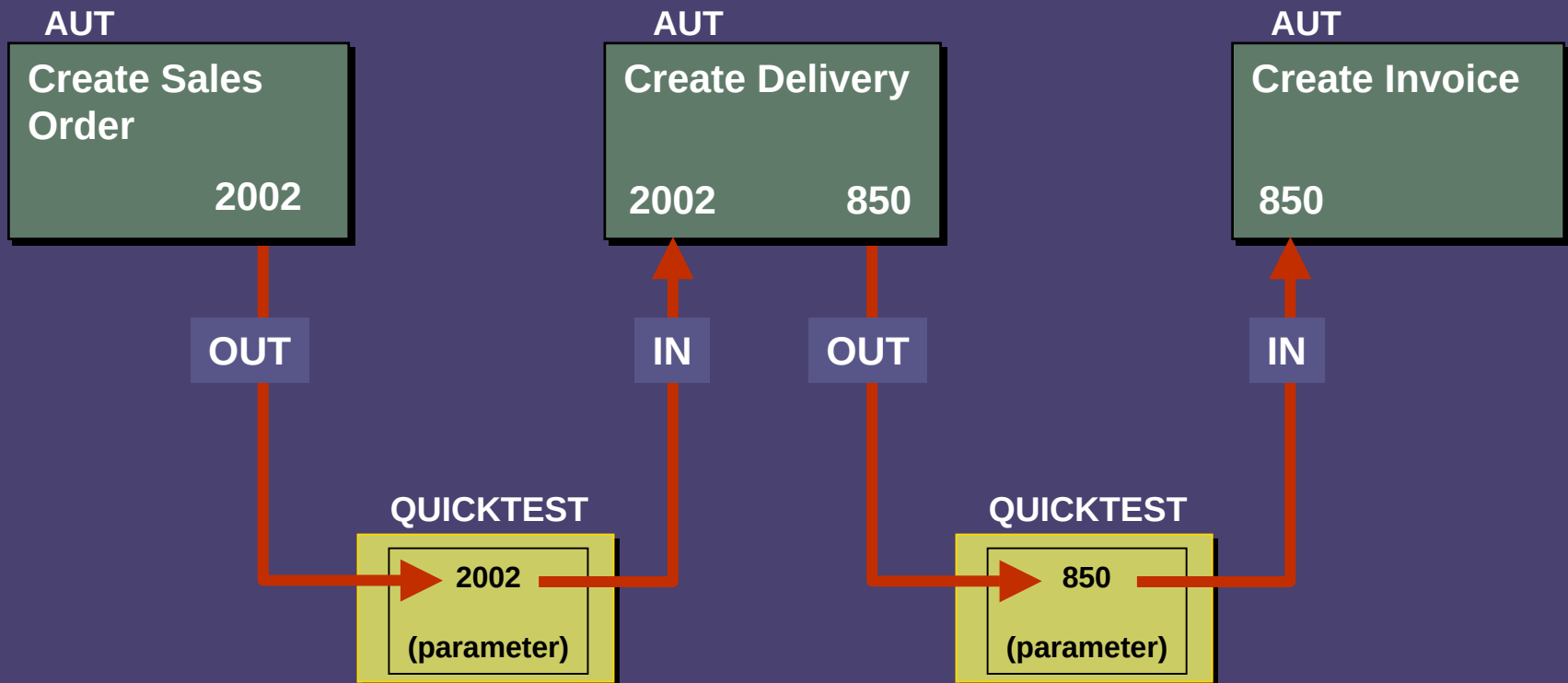
How Correlation Works—Example

- In some scenarios, a business process requires a system-generated output from a previous business process



How Correlation Works in QuickTest

- Add a step to capture the output to be reused and save it in a parameter
- Use the same parameter to input data when needed later

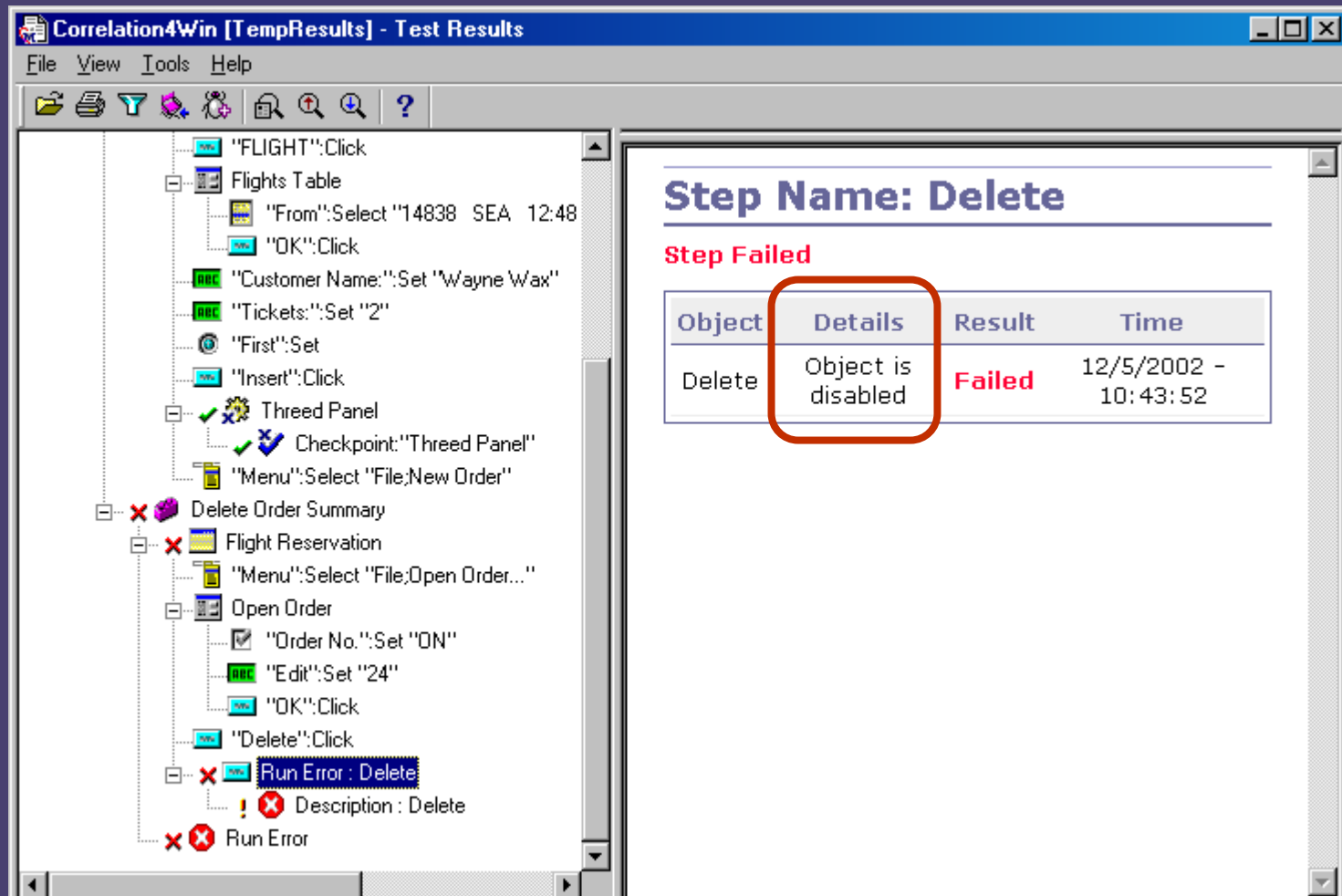


Finding and Fixing Broken Correlations

1. Run script and check errors in Test Results.
2. Find correlated input value(s) in the script.
3. Use a parameter to capture output data to be reused later.
4. Replace correlated input value(s) with output parameter(s).
5. Verify correct execution.

1. Check Errors in Test Results

- Run the script and look for errors in the Test Results
- Get developer / DBA / functional expert to examine the script if you need help



Correlation4Win [TempResults] - Test Results

File View Tools Help

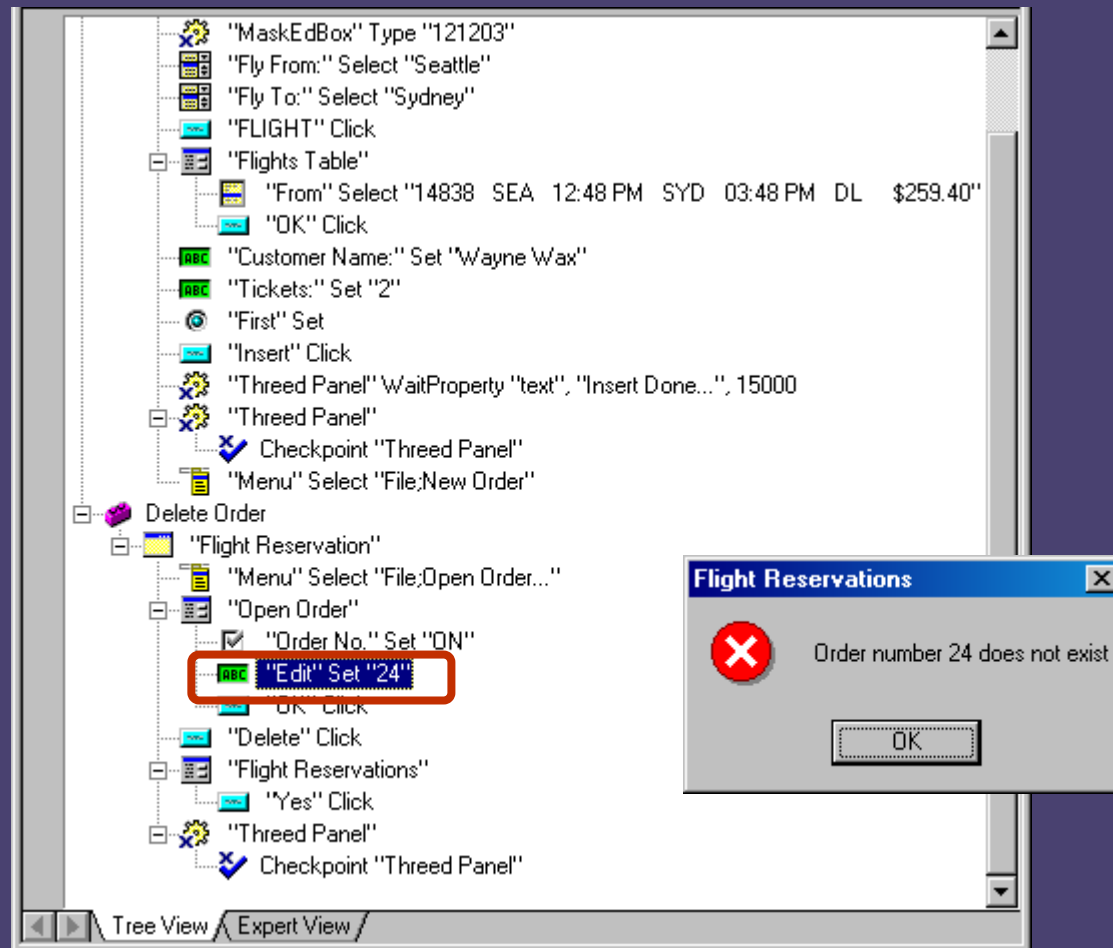
Step Name: Delete

Step Failed

Object	Details	Result	Time
Delete	Object is disabled	Failed	12/5/2002 - 10:43:52

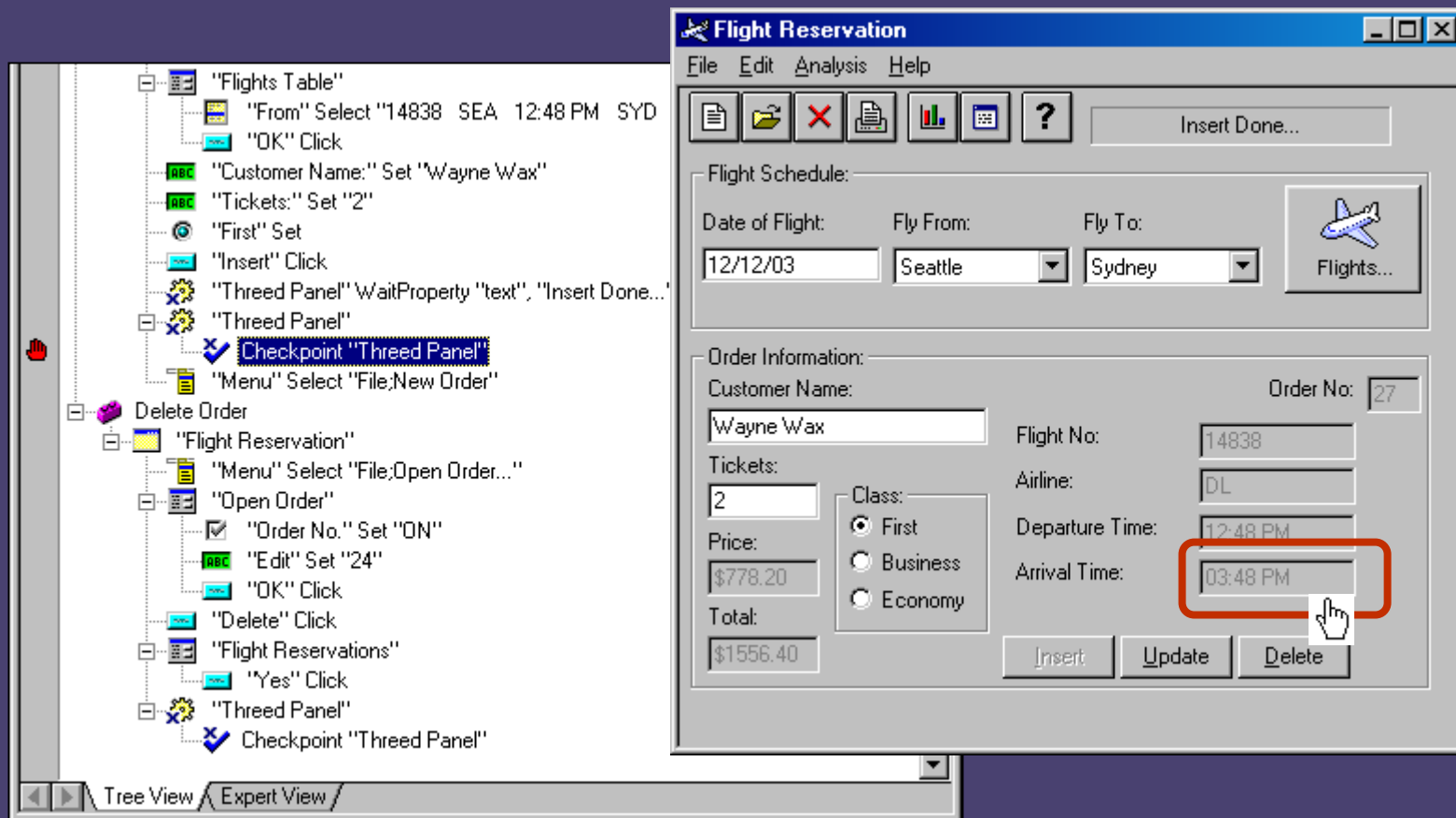
The screenshot shows a test results window for 'Correlation4Win [TempResults]'. The left pane displays a test script tree with various steps like 'FLIGHT':Click, 'From':Select, 'OK':Click, 'Customer Name':Set, 'Tickets':Set, 'First':Set, 'Insert':Click, 'Threed Panel', 'Checkpoint':Set, 'Menu':Select, 'Delete Order Summary', 'Flight Reservation', 'Menu':Select, 'Open Order', 'Order No.':Set, 'Edit':Set, 'OK':Click, 'Delete':Click, 'Run Error : Delete', and 'Description : Delete'. The 'Delete' step is highlighted. The right pane shows the details for the 'Delete' step, which has failed. A red box highlights the 'Details' column in the table, which contains the text 'Object is disabled'.

2. Find Value(s) to be Correlated



3. Capture Output Data to be Reused (1)

- Run test to point where output appears
- Insert a standard output step
- Click on the output to be captured



3. Capture Output Data to be Reused (2)

- Select the property with the value to be captured
- Type a descriptive name for the output parameter

Output Value Properties

Logical_name: Order No:
Class: WinEdit

Type	Property	Value
☐ ABC	focused	False
☐ ABC	height	20
☒ ABC	text	<Order_No_out>
☐ ABC	width	25
☐ ABC	x	373
☐ ABC	y	197

Edit value

Data Table column name
Order_No_out

Location in Data Table

☒ Global sheet
☐ Current action sheet (local)

OK Cancel Help

3. Capture Output Data to be Reused (3)

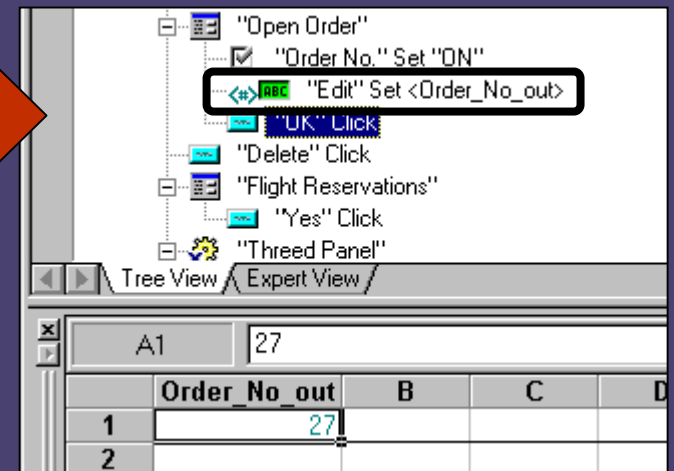
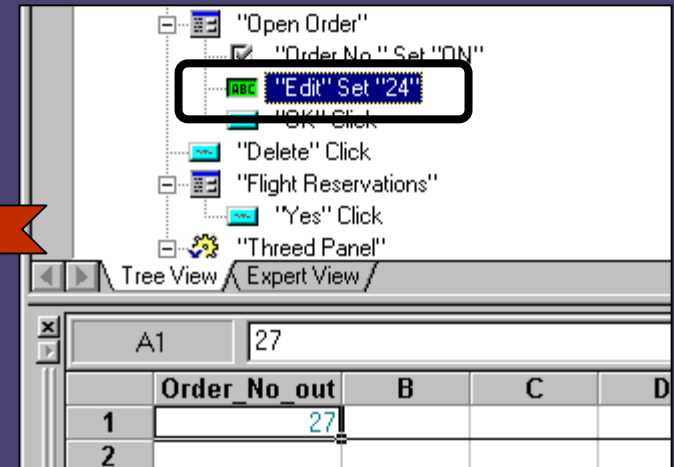
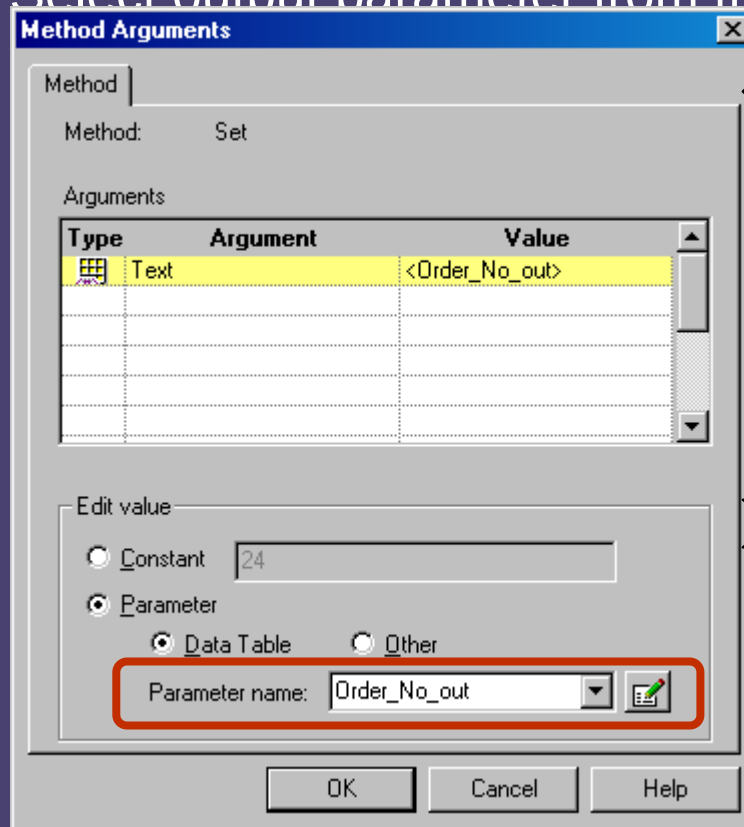
The screenshot displays a test script editor with a sequence of steps. A red box highlights the "Output "Order No."" step. Below the script, a data table is shown with a red box around the first row, which contains the value "27" in the "Order No_out" column.

	A1	B	C	D	E	F
1	Order No_out	27				
2						
3						
4						
5						

- Output step is added to the script
- Parameter is added to the Data Table
- Value at the time of recording is entered in Row 1

4. Replace Input Value with Output Parameter

- Parameterize the input value
- Select output parameter from the list



5. Run Script and Check Test Results

Correlation4Win [TempResults_1] - Test Results

File View Tools Help

Step Name: Output:Order No:

Step Done

Object	Details	Result	Time
Output:Order No:	Output text: captured "28" into data table column Order_No_out	Done	12/5/2002 - 11:28:17

Run the correlated script and check for correct parameter substitution

For Help, press F1

Ready

Text Output Value

- Available only during recording
- Opens Text Output Value Properties
 - See exactly what will be captured
 - Choose parameter name

FLIGHT CONFIRMATION

Your itinerary has been booked!

Please print a copy of this screen for your records. Thank you for choosing Mercury Tours.

Flight Confirmation # 2002-12-13148 2002-12-13 14:58:26.0

Departing

Acapulco to Zurich

12/13/2002 @ 5:03 w/ Blue Skies Airlines 190
Coach
\$706 each

Returning

Zurich to Acapulco

12/14/2002 @ 12:23 w/ Blue Skies Airlines 910
Coach
\$706 each

Passengers

✓ **Text Output Value Properties**

Output Value Summary:

Output the text that is displayed between **Departing** and **12/** into **<HP_itinerary_out>**. Captured text is **Acapulco to Zurich**

Output Text

Data Table column name
HP_itinerary_out

Location in Data Table

☒ Global sheet

☐ Current action sheet (local)

OK Cancel Help

Text Before Output Value

FLIGHT CONFIRMATION 

Your itinerary has been booked! 

Please print a copy of this screen for your records. Thank you for choosing Mercury Tours.

Flight Confirmation # 2002-12-13148 **2002-12-13 14:58:26.0**

Departing

Acapulco to Zurich

12/13/2002 @ 5:03 w/ Blue Skies Airlines 190
Coach
\$706 each

Returning

Zurich to Acapulco

12/14/2002 @ 12:23 w/ Blue Skies Airlines 910
Coach
\$706 each

Passengers

✓ Text Output Value Properties 

Output Value Summary:

Output the text that is displayed between **Departing** and **12/** into **<HP_itinerary_out>**. Captured text is **Acapulco to Zurich**

Text Before 

☒ Use the text before

Text to capture is displayed after occurrence **1** of

☒ Constant **Departing**

☐ Parameter

☒ Data Table ☐ Other

Parameter name: **_nullText_Before** 

OK **Cancel** **Help**

Text After Output Value

Best practice:

- Open the HTML source
- Do a "find" for the text
- Count how many "finds" until you find the one you want

FLIGHT CONFIRMATION

Your itinerary has been booked!

Please print a copy of this screen for your records. Thank you for choosing Mercury Tours.

Flight Confirmation # 2002-12-13148

2002-12-13 14:58:26.0

Departing

Acapulco to Zurich

12/13/2002 @ 5:03 w/ Blue Skies Airlines 190
Coach
\$706 each

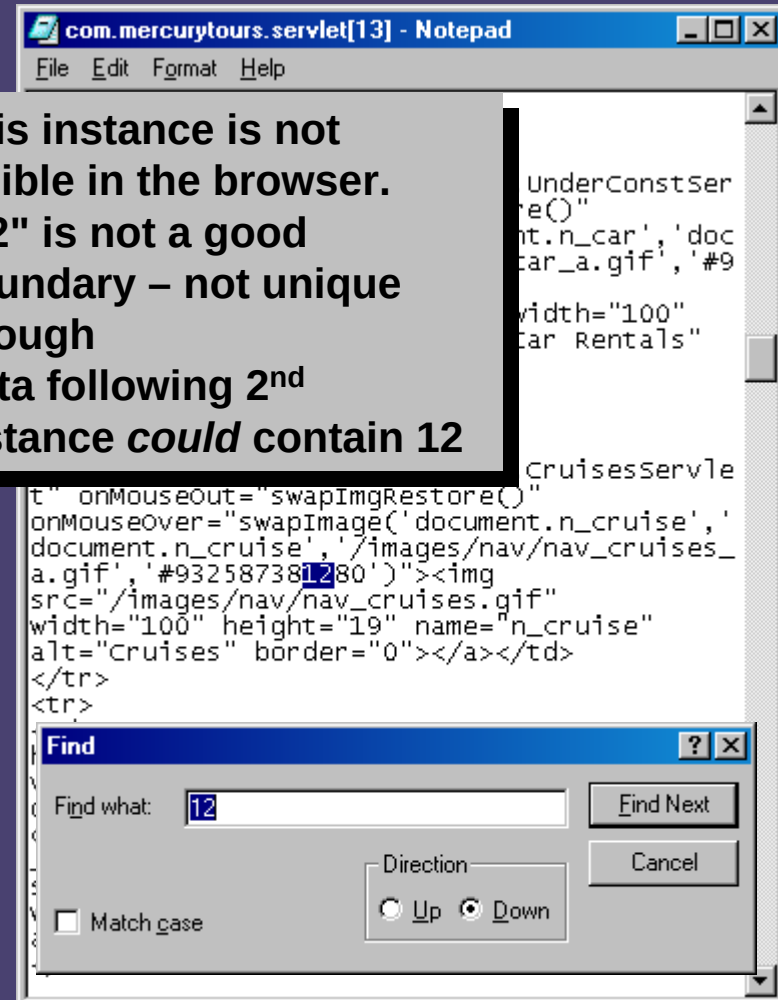
Returning

Zurich to Acapulco

12/14/2002 @ 12:23 w/ Blue Skies Airlines 910
Coach
\$706 each

Passengers

- This instance is not visible in the browser.
- "12" is not a good boundary – not unique enough
- Data following 2nd instance *could* contain 12



Text After Output Value

✓ **Text Output Value Properties** [X]

Output Value Summary:

Output the text that is displayed between **Departing** and **<month>** into **<HP_itinerary_out>**. Captured text is **Acapulco to Zurich**

Text After [v]

☒ Use the text after

Text to capture is displayed before occurrence of

☐ Constant

☒ Parameter

☒ Data Table ☐ Other

Parameter name: [icon]

[OK] [Cancel] [Help]

Data Table Parameter Options [X]

Name: [v]

Location in Data Table

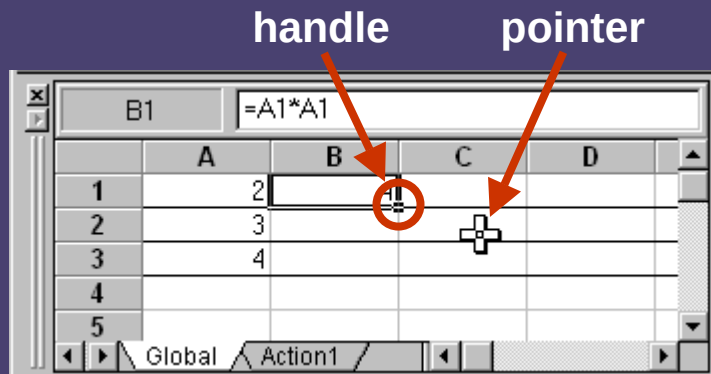
☒ Global sheet

☐ Current action sheet (local)

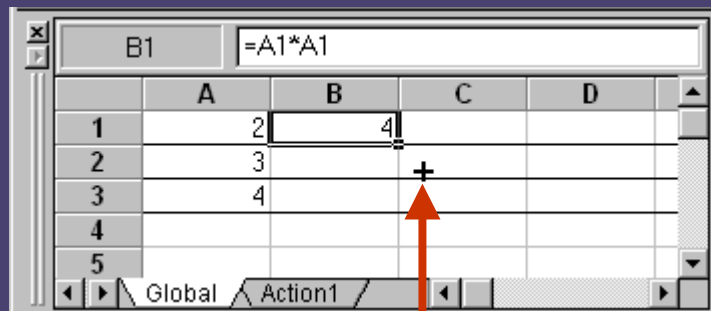
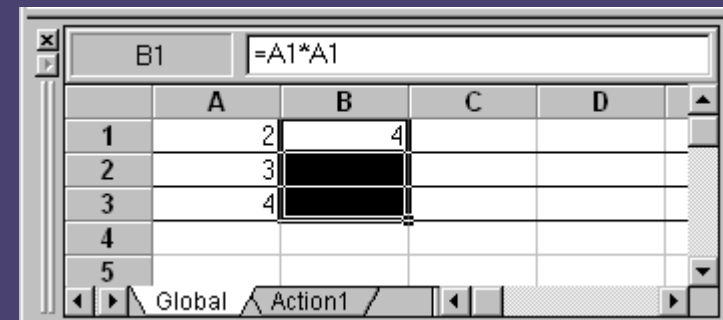
[OK] [Cancel] [Help]

Applying Data Table Formula to Subsequent Rows

- Click on the cell "handle," drag downward and release
- Formula is applied to all selected rows
- Cell references are updated automatically



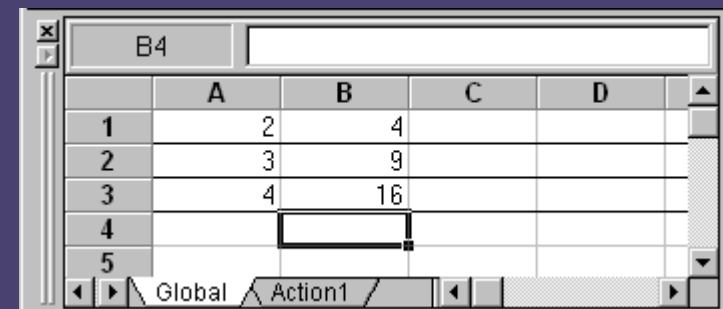
Release pointer when heavy outline surrounds all rows



pointer, when hovering over handle

Row 2 formula: $=B1*B1$

Row 3 formula: $=C1*C1$



Database Checkpoints

Definition

database checkpoint

A database checkpoint is a script step which verifies that one or more database tables contain expected data.

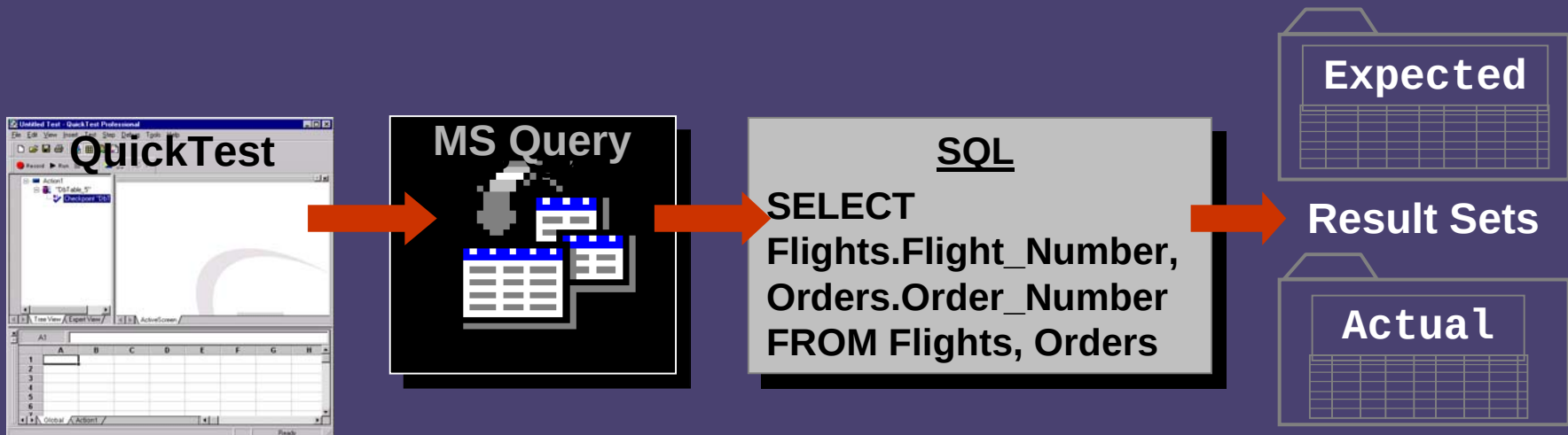
Benefits from Database Checkpoints

Database checkpoints can answer questions such as:

- Does the AUT insert, update and delete records correctly?
- Does the new version of the database function properly?
- Does the database maintain referential integrity?

How QuickTest Queries the Database

- QuickTest integrates with Microsoft Query
- Use Microsoft Query to create a query to extract data
- The Query is used by QuickTest to capture expected and actual data values from the table(s)



Test Specification - Example

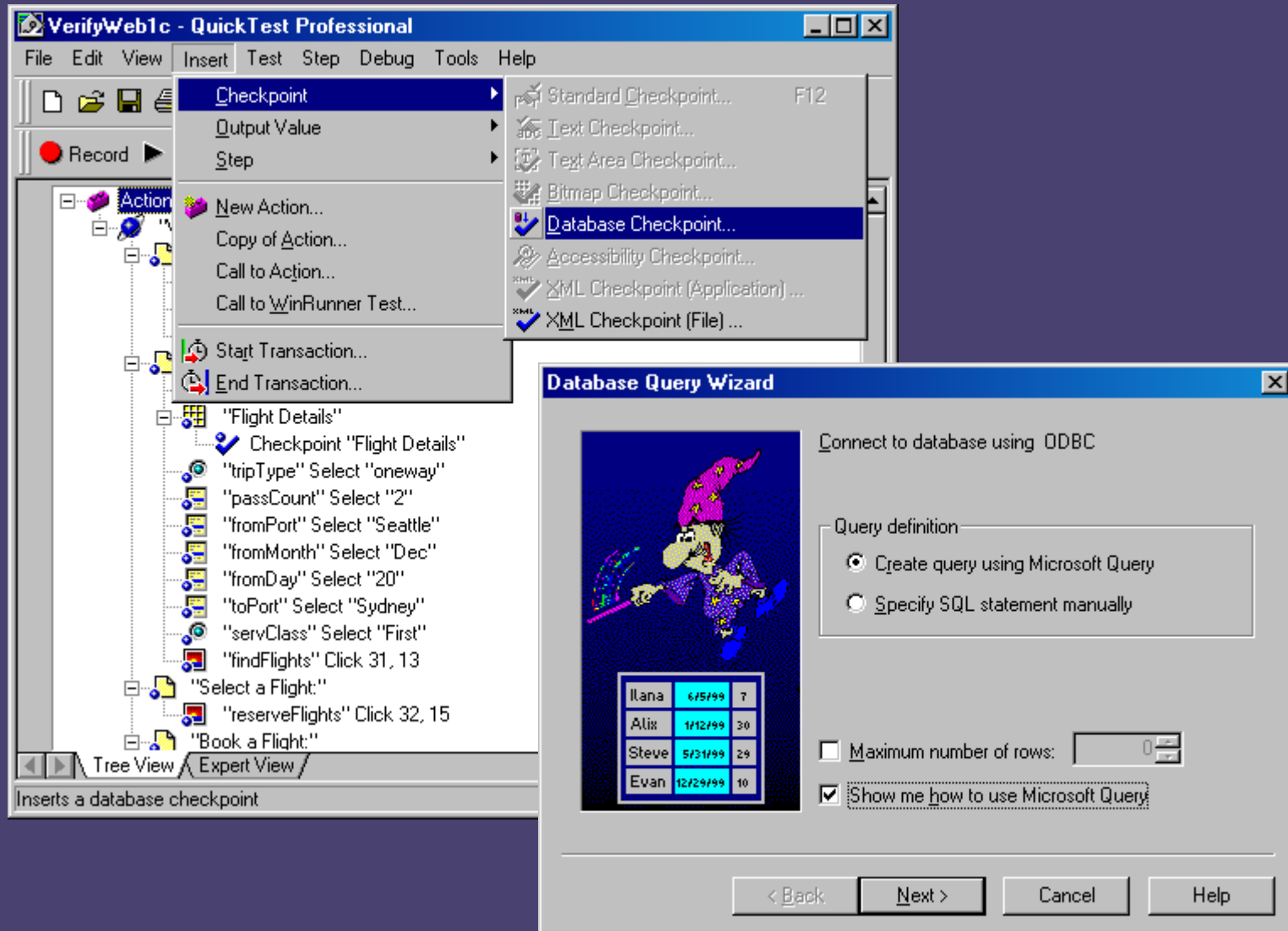
- This shows how a database checkpoint could be used in a test

Application	Mercury Web Tours
Business Process	Create a flight reservation
Description	Inserts a new reservation into the database
Starting Page	WelcomeServlet">http://localhost/servlets/com.mercurytours.servlet>WelcomeServlet
Test Objective	Verify that the User Name and Flight information are added correctly to the database
Test Requirements (partial list)	- User Name appears in Welcome page text - Checkout image button appears - Total cost is charged to credit card account
Browser Version	Internet Explorer 5.5 (or higher) Netscape Navigator 6

How to Add a Database Checkpoint

1. Invoke the Database Query Wizard.
2. Define a Query.
3. Specify the Checkpoint properties.

1. Invoke Database Query Wizard



2. Define a Query

Define a query with Microsoft Query

- use Query Wizard help
- write your own SQL statement

Database Query Wizard

Specify SQL statement

Connection string:

SQL statement:



< Back Finish Cancel Help

Database Query Wizard

Connect to database using ODBC

Query definition

☒ Create query using Microsoft Query

☐ Specify SQL statement manually

☐ Maximum number of rows:

☐ Show me how to use Microsoft Query

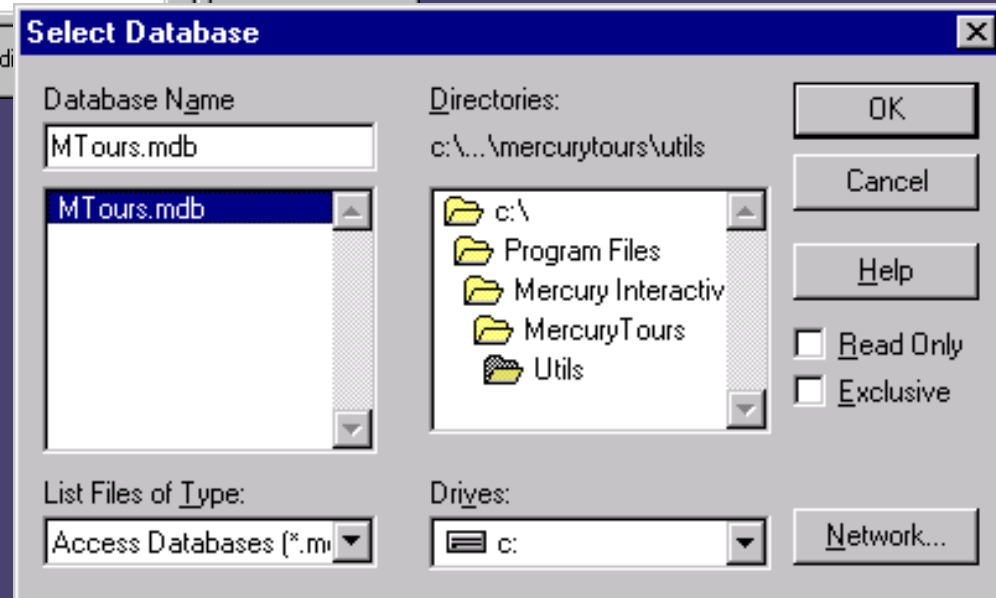
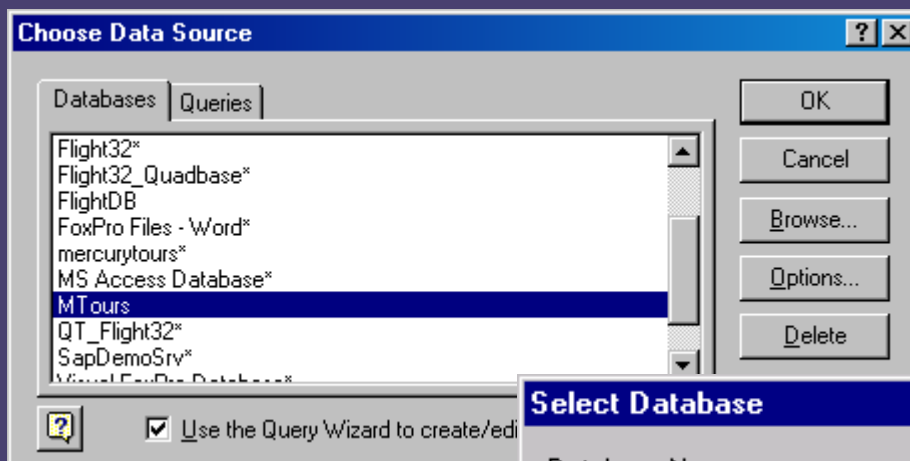


< Back Next > Cancel Help

Ilana	6/5/99	7
Alix	1/12/99	30
Steve	5/31/99	29
Evan	12/29/99	10

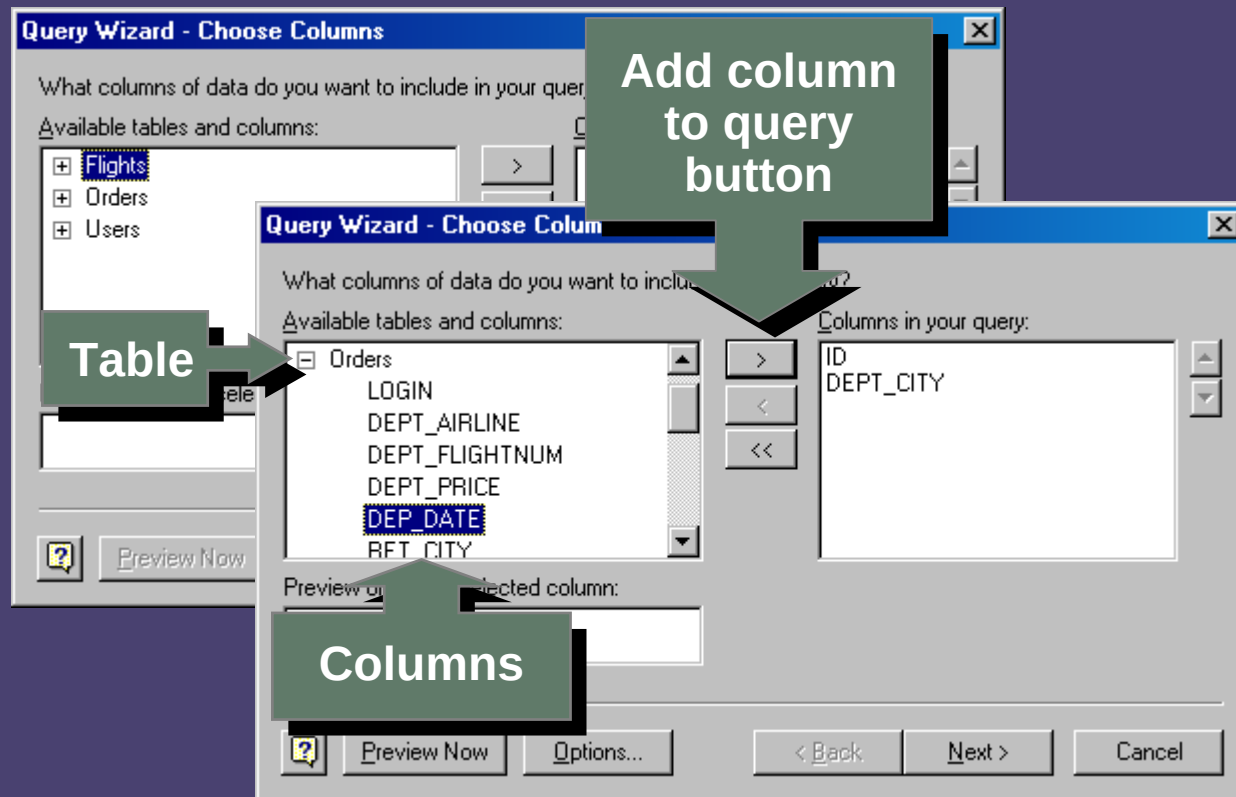
Select the Data Source

- Optionally, you might need to select the database



Choose Columns

- Select a table, then the columns
- Repeat as needed



Specify Expected Data

Database Checkpoint Properties

	ID	DEPT_CITY	DEP_DATE
1	✓ 102	Paris	✓ 12/20/2002
2	✓ 101	✓ London	✓ 12/20/2002
3	✓ 100	✓ Frankfurt	✓ 12/20/2002
4	✓ 99	✓ Paris	✓ 12/20/2002
5	✓ 98	✓ London	✓ 12/20/2002
6	✓ 97	✓ Frankfurt	✓ 12/20/2002
7	✓ 96	✓ Paris	✓ 12/20/2002
8	✓ 95	✓ London	✓ 12/20/2002
9	✓ 94	✓ Frankfurt	✓ 12/20/2002
10	✓ 93	✓ Paris	✓ 12/20/2002
11	✓ 92	✓ London	✓ 12/20/2002
12	✓ 91	✓ Frankfurt	✓ 12/20/2002
13	✓ 90	✓ Paris	✓ 12/22/2002
14	✓ 89	✓ Paris	✓ 12/22/2002

Expected Data | Settings

Selected cell: TbVerObj: Row 1 Col 2

Check value as:

☐ Constant Paris

☒ Parameter

☒ Data Table ☐ Other

Parameter name: FromCity

Insert Statement: ☐ Before current step ☒ After current step

OK Cancel Help

Selects all cells,
rows and columns

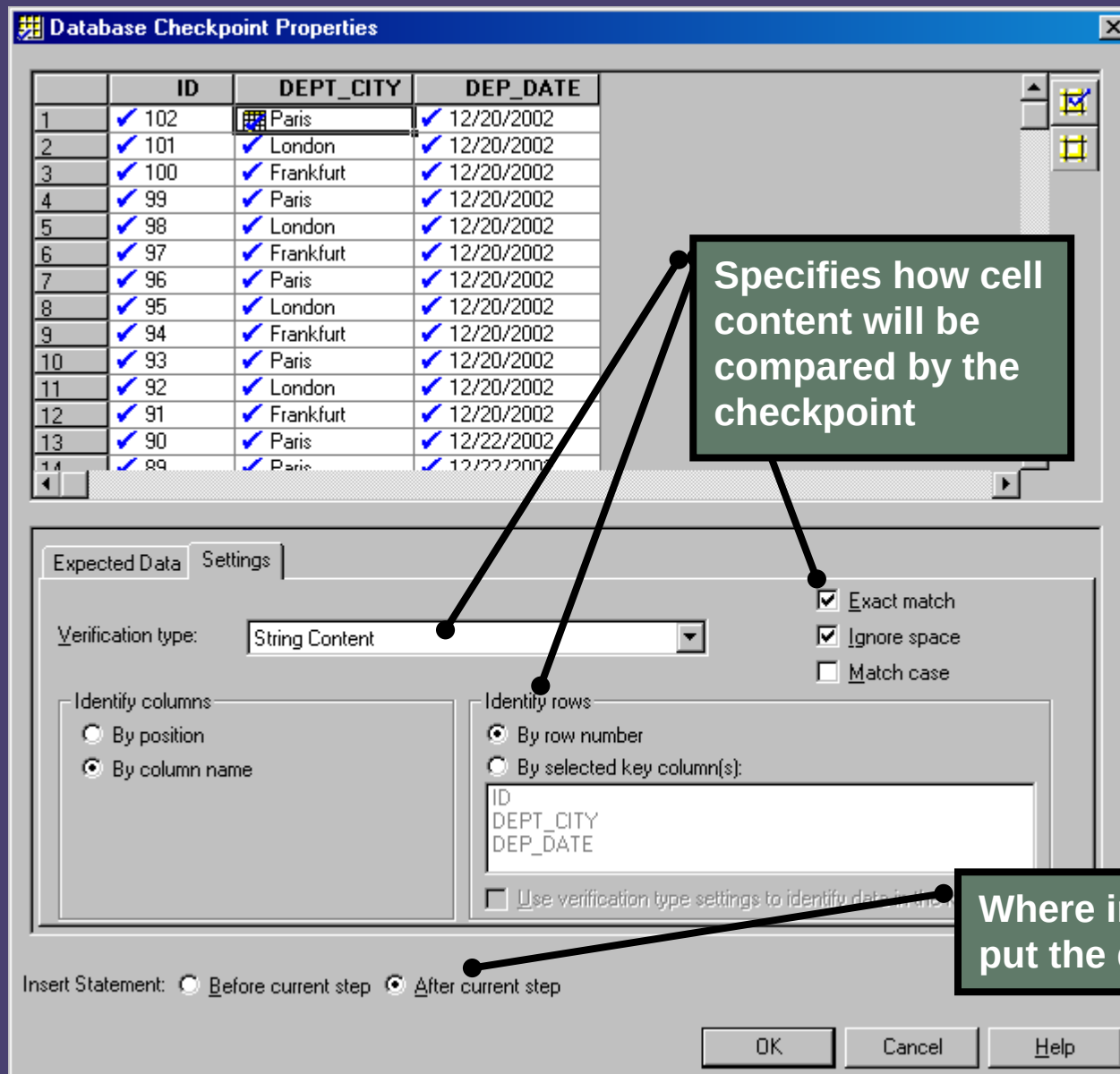
Checks
selected cell(s),
row or column

Unchecks
selected cell(s),
row or column

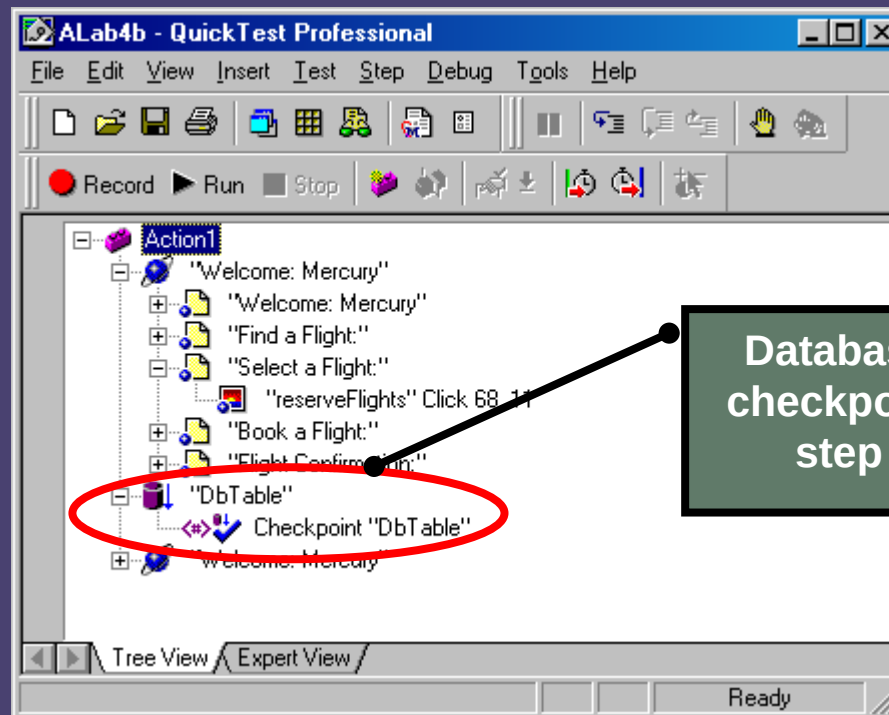
Indicates row and column
number of the
selected cell

Indicates that
multiple data values
are used for the
selected column

Specify Verification Type

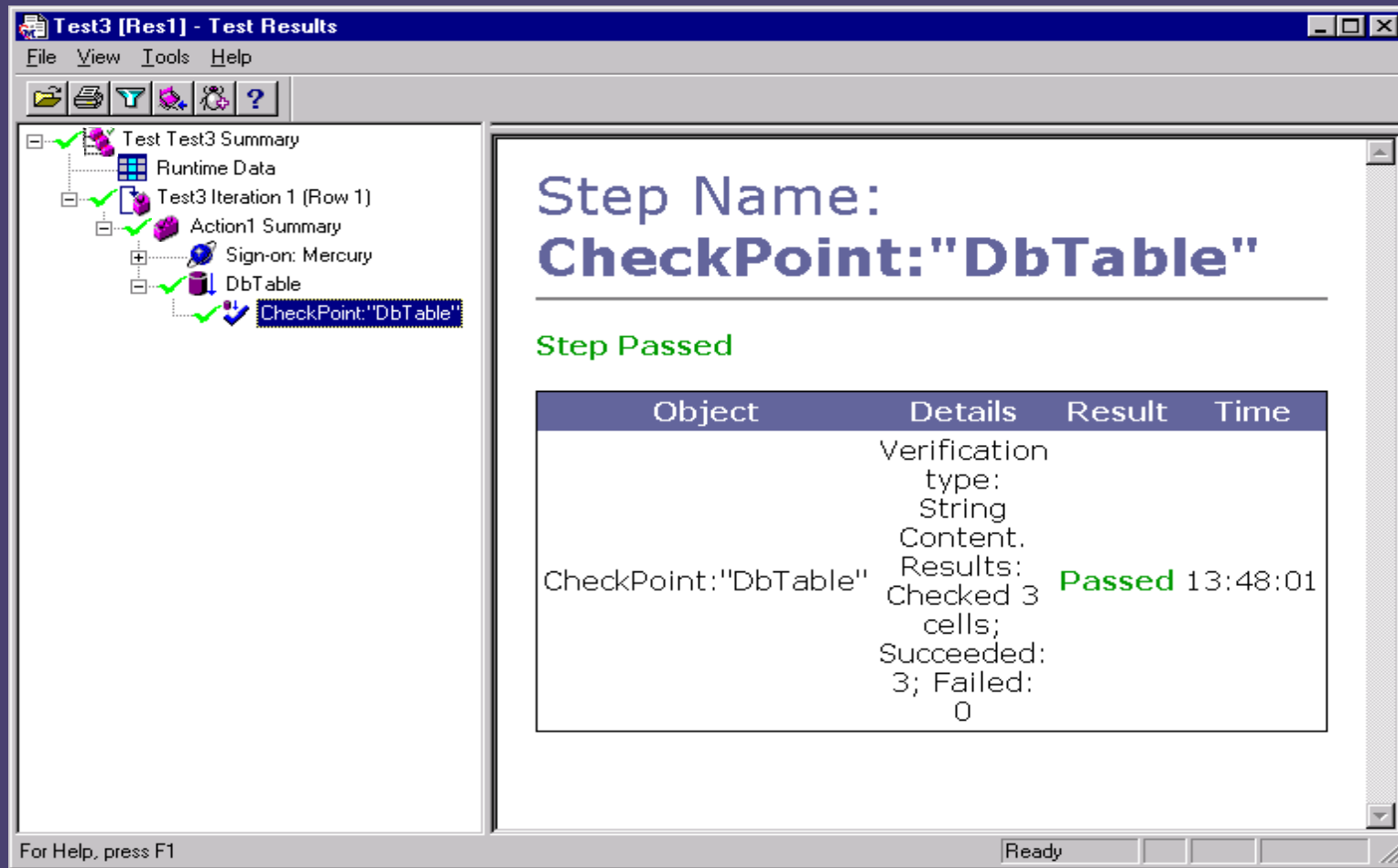


Database Checkpoint is Added



Database
checkpoint
step

Database Checkpoint Results—Passed



Database Checkpoint Results—Failed

DBCheckFail [Res2] - Test Results

File View Tools Help

Test DBCheckFail Summary

- Runtime Data
- DBCheckFail Iteration 1 (Row 1)
- DBCheckFail Iteration 2 (Row 2)
- StartUp Browser
- Action1 Summary
- Welcome: Mercury
- DbTable
- Checkpoint:"DbTable"
- Welcome: Mercury

Object	Details	Result	Time
	Verification type: String Content. Results:		
DEPT_CITY	DEP_DATE	RET_CITY	RET_DATE
34 New Yor	12/18/200	London	12/20/200
35 Frankfur	12/15/200	Acapulco	12/18/200
36 New Yor	12/18/200	London	12/20/200
37 Frankfur	12/17/200	San Fran	12/18/200
38 Frankfur	12/17/200	San Fran	12/18/200
39 Frankfur	12/17/200	San Fran	12/18/200
40 Acapulco	12/17/200		
41 Frankfur	12/18/200		
42 Acapulco	12/18/200		
43 Frankfur	12/19/200		
44 Frankfur	12/19/200		
45 Frankfur	12/19/200		
46 Frankfur	12/19/200		
47 New Yor	12/20/200		
48 Frankfur	12/19/200		
49 New Yor	12/20/200		
50 Frankfur	12/19/200	London	12/20/200
51 New Yor	12/20/200	Paris	12/22/200
52 Frankfur	12/19/200	London	12/20/200
53 New Yor	12/20/200	Paris	12/22/200
54 New Yor	12/22/20	Paris	12/25/20

Test Results

Expected Value: New York

Actual Value: Frankfurt

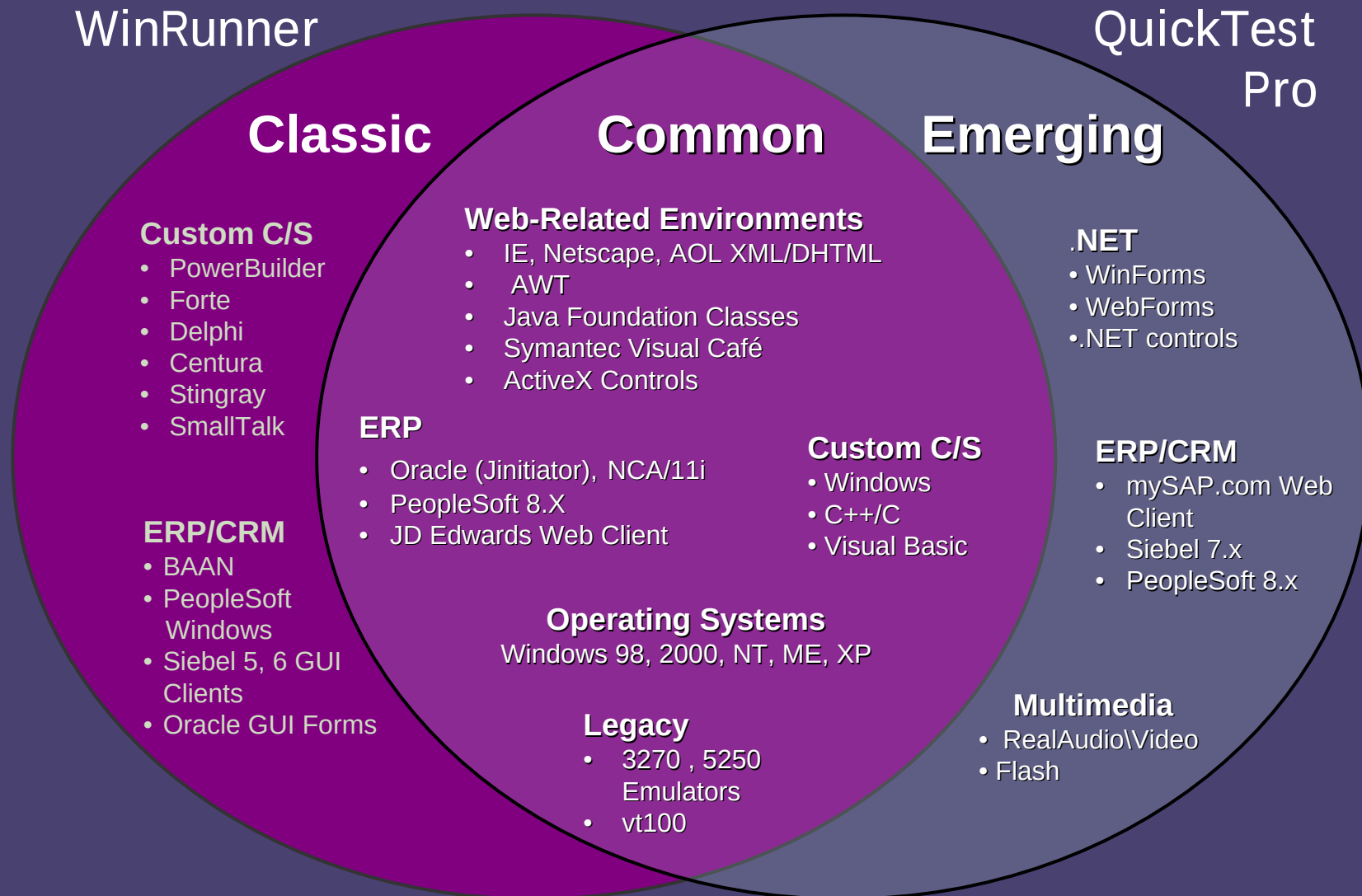
OK

For Help, press F1

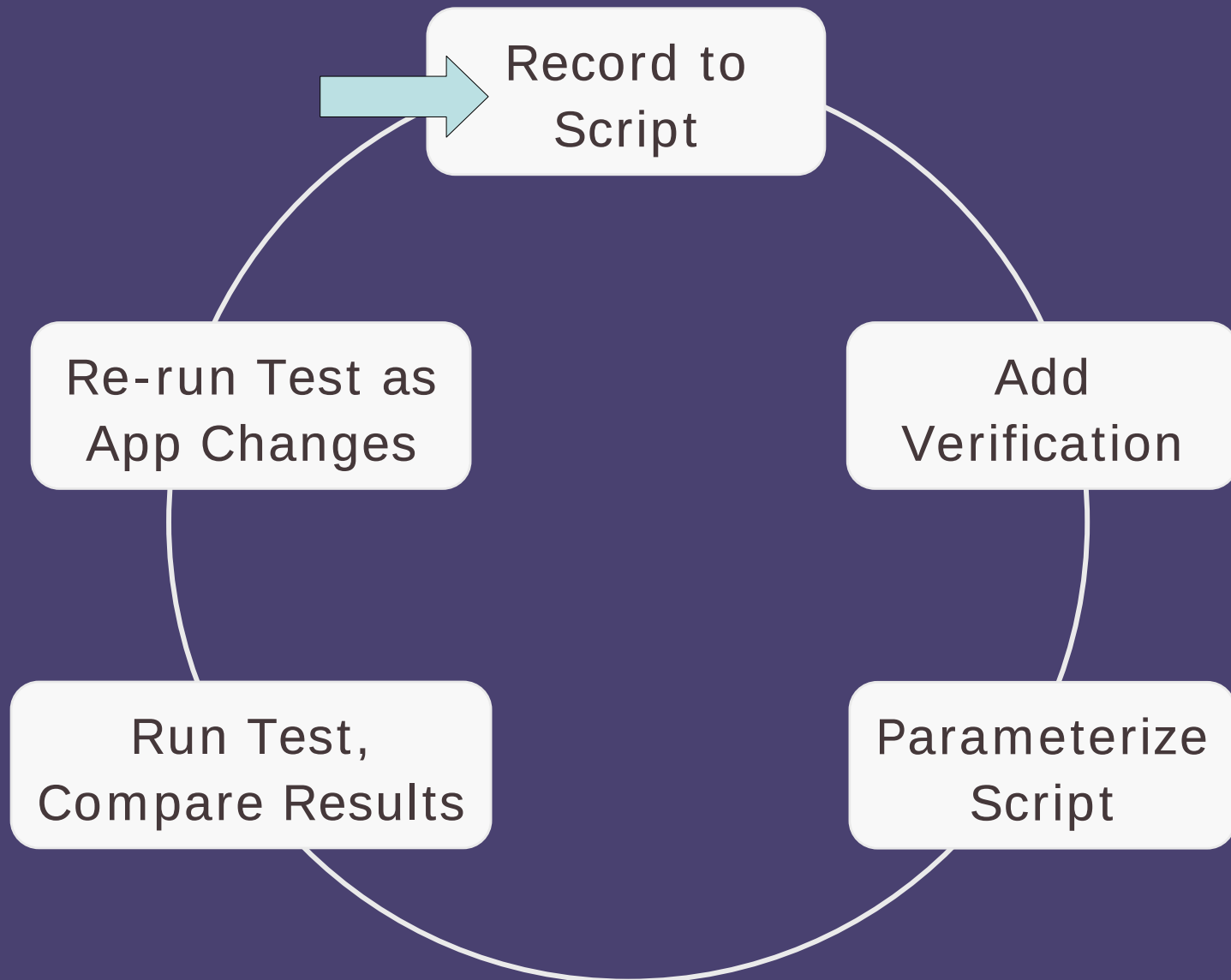
Ready

QuickTest for WinRunner Users

QuickTest and WinRunner Supported Environments



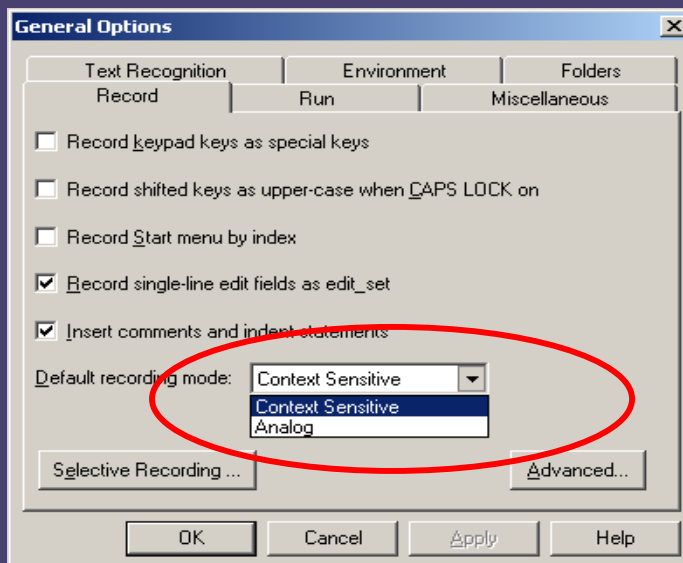
QTP and WR Features: Functional Testing Cycle



Script Recording: Two Modes

WinRunner

- *Context-Sensitive*
 - Identifies GUI objects
 - Flat object hierarchy
- *Analog*
Captures keyboard input,
mouse clicks, mouse path



QuickTest Pro

- *Context-Sensitive*
 - Identifies GUI objects
 - Multi-level object hierarchy
- *Low-level*
Uses mouse coordinates

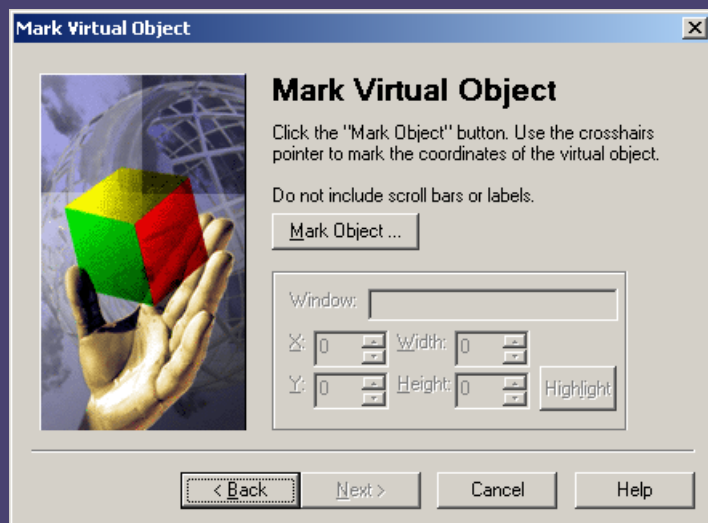


Virtual Objects

Map unknown object to a known object type

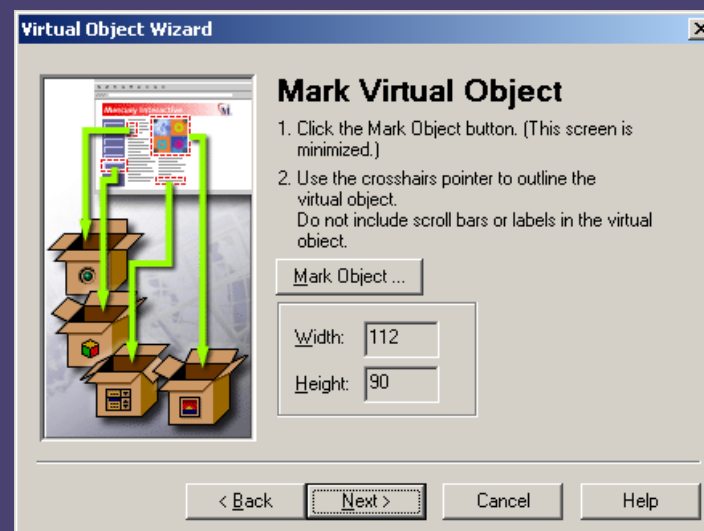
WinRunner

- Object identity based on parent window
- Object won't be recognized in any other windows



QuickTest Pro

- Object identity based on parent object or entire object hierarchy
- Benefit: virtual object identified across apps

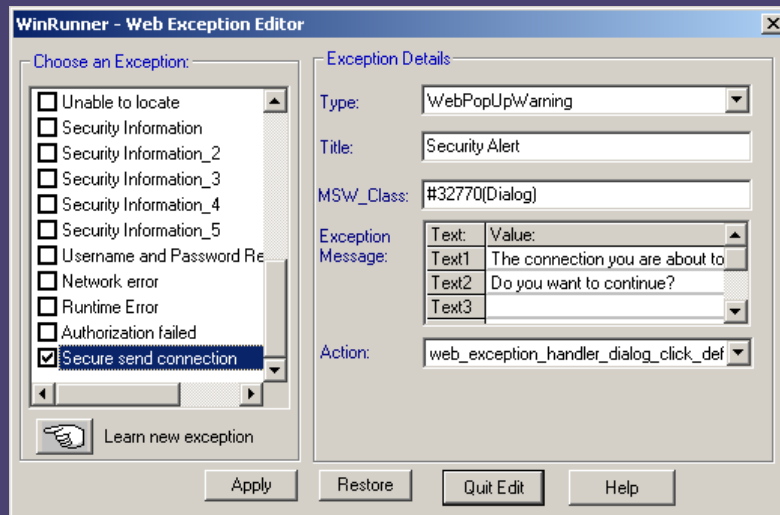


Handling Test Execution Issues

Drive browser error dialogs, if and when they appear

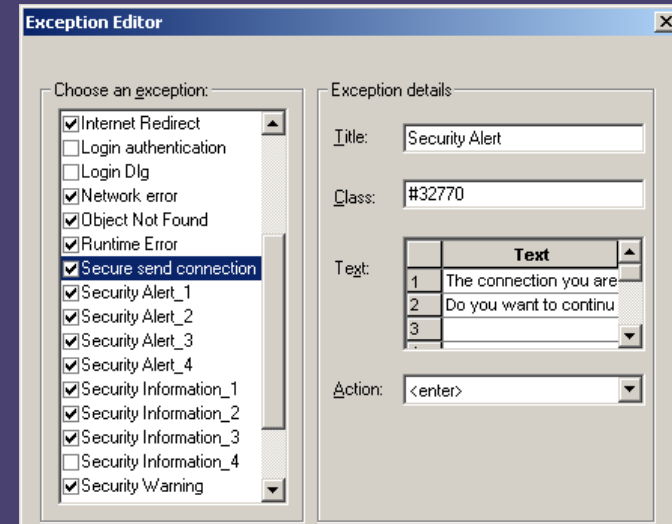
WinRunner

- *Web Exception Editor*
- Exception behavior must be specified for each dialog



QuickTest Pro

- *Exception Editor*
- Exception behavior pre-configured to “do the typical thing”
- Default behavior easily modified



Scripts

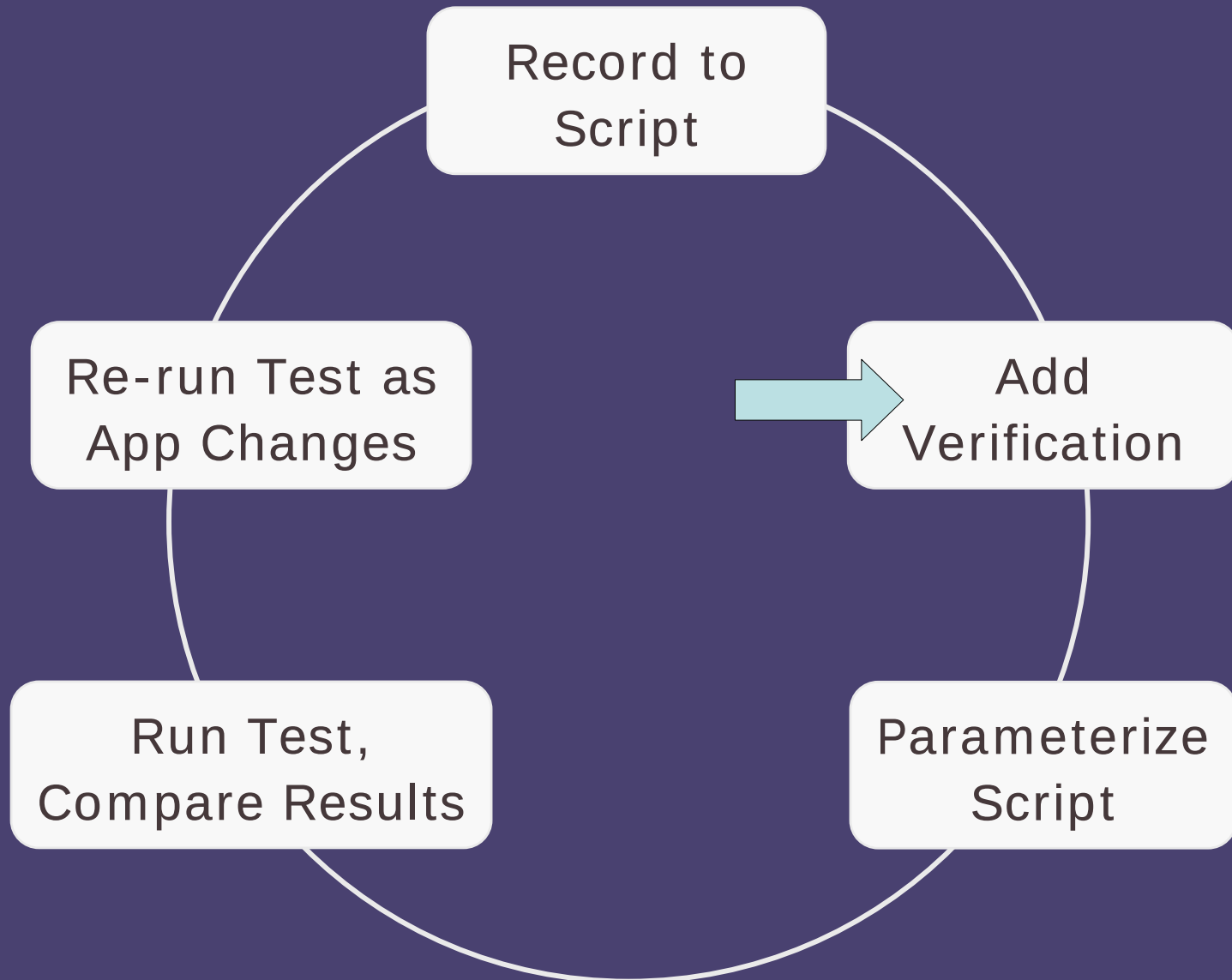
WinRunner *Test Script*

- Programmatic representation
- TSL, similar to C
- Procedural language
- Can call other scripts
- Creates temporary *GUI Map* for object-related information

QuickTest Pro *Action*

- Icon-based & programmatic representations
- VBScript, similar to VB
- Object-oriented language
- Can call other actions
- Automatically saves all object-related information into *Object Repository*

QTP and WR Features: Functional Testing Cycle



Verification Checkpoints

WinRunner

- Checkpoint on single property, object, or window
- Checkpoint for Multiple Objects
- Text
- Test Area
- HTML
- Table
- Database
- Bitmap
- Runtime Database Checkpoint
- Add via menu, wizard, or programmatically

QuickTest Pro

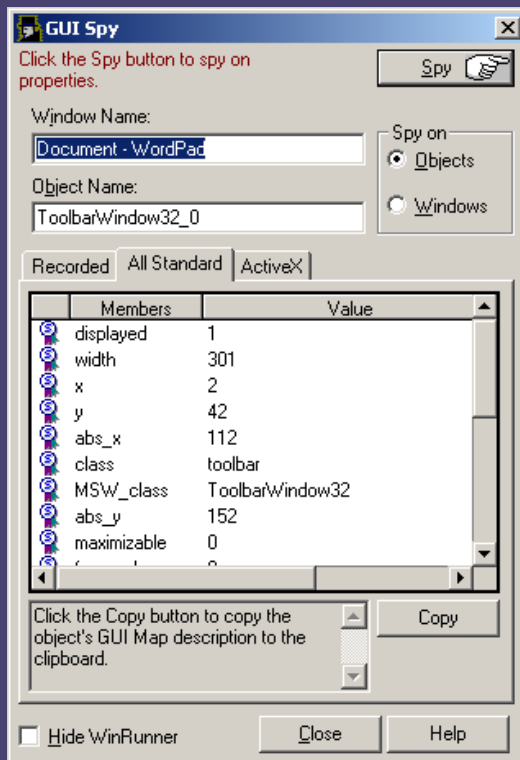
- Checkpoint on single property, object, or window
- Text (web-only)
- Text Area
- HTML
- Table
- Database
- Bitmap
- Image
- XML
- Accessibility
- Add via ActiveScreen, tree view, menu, or programmatically

GUI Spy v. Object Spy

View the properties of any object in AUT

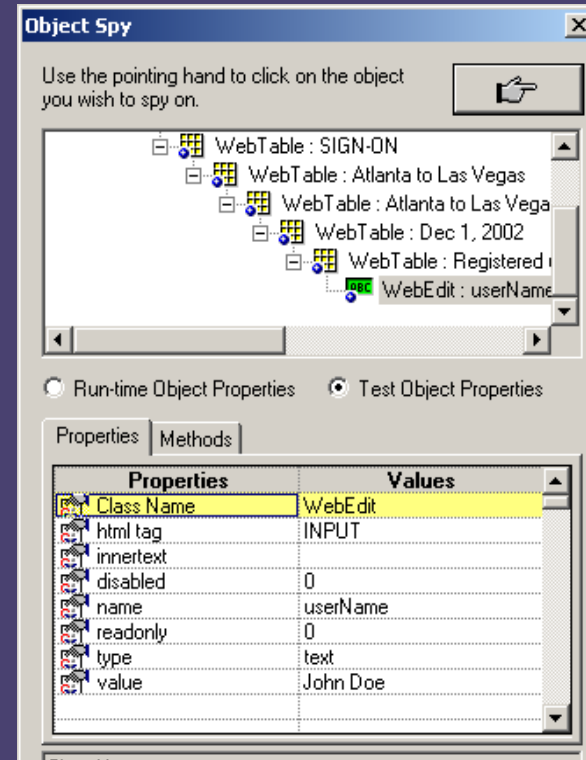
WinRunner: *GUI Spy*

- Flat object hierarchy



QTP: *Object Spy*

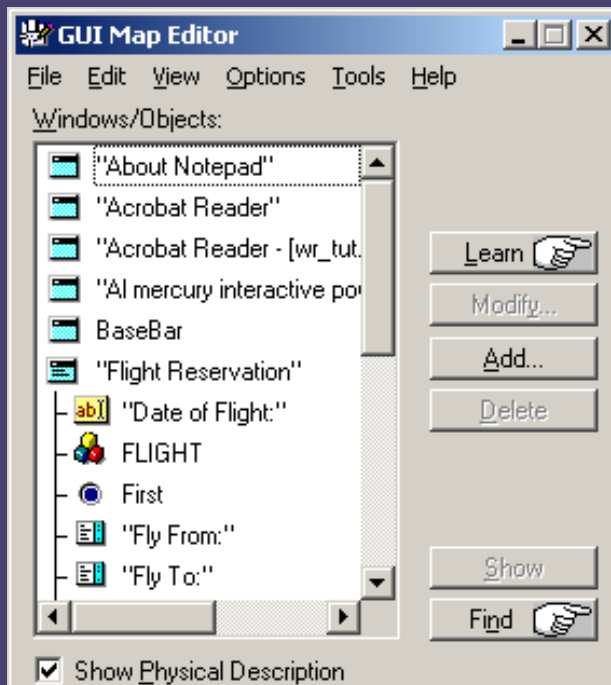
- Multi-level object hierarchy



Object Storage and Operations

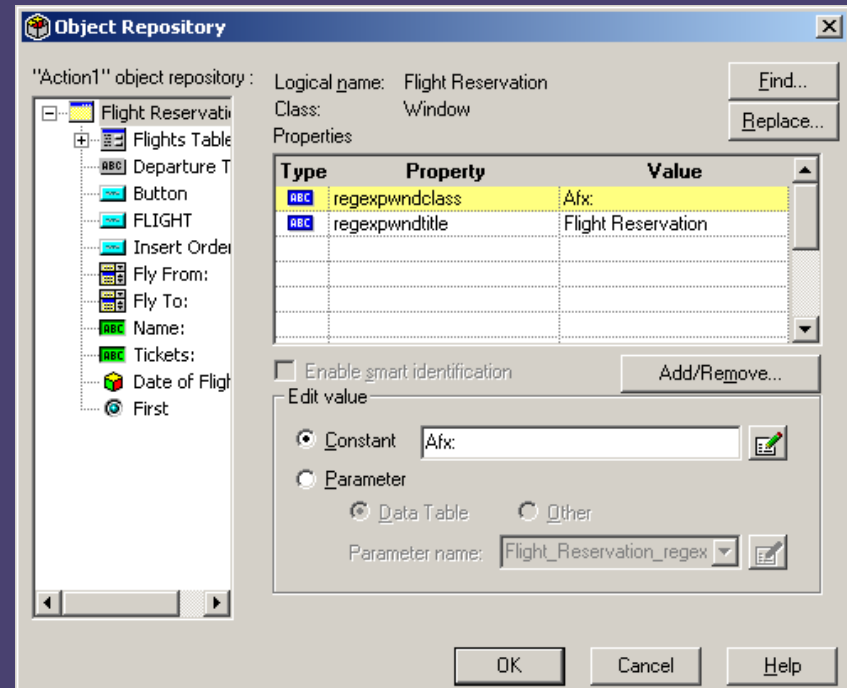
WR: GUI Map

- Two types
 - One GUI Map file per test
 - Global GUI Map file
- Creates temporary GUI Map, can be saved



QTP: Object Repository

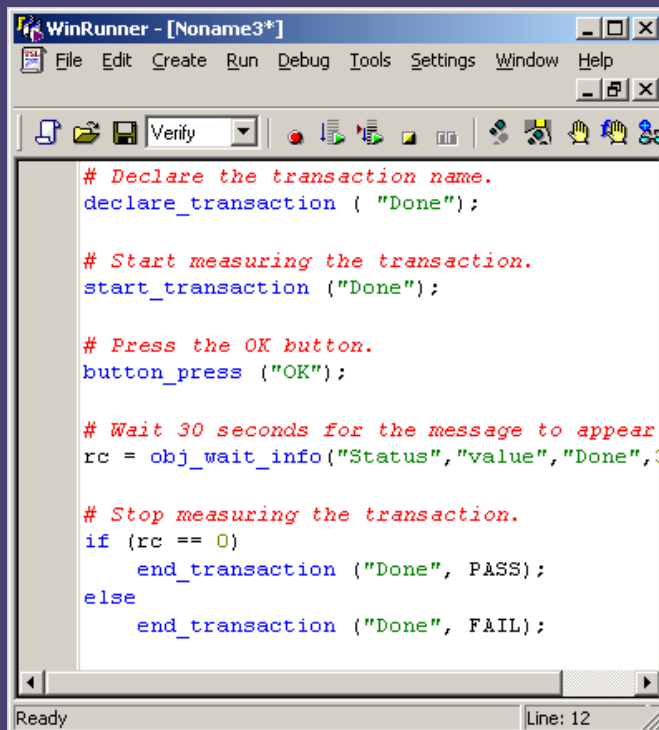
- Two Types
 - Per-action object repository
 - Shared object repository
- Automatically saves object repository



Transaction Measurement

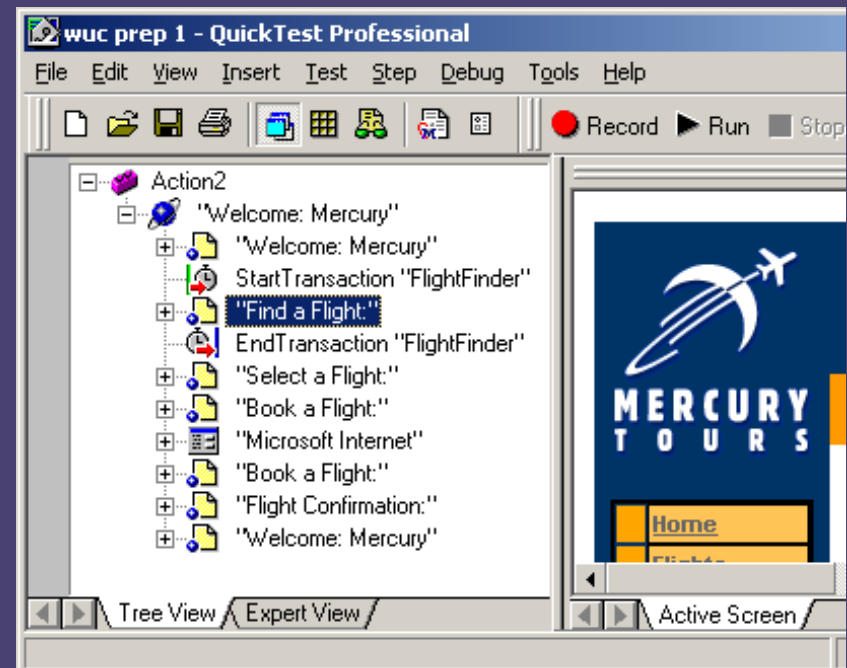
WinRunner: one method

- TSL programming

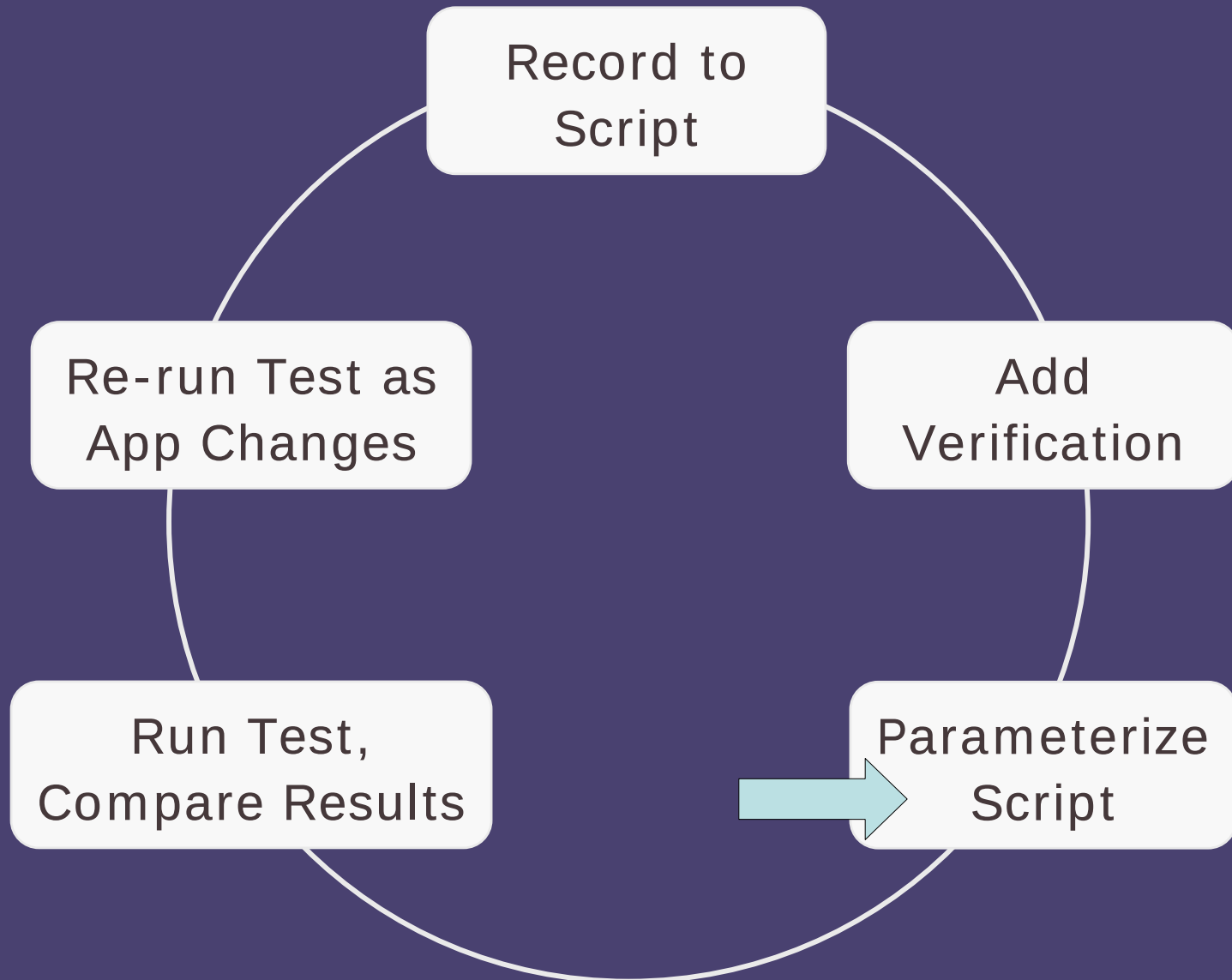


QuickTest: two methods

- Tree view (no coding)
- VBScript programming



QTP and WR Features: Functional Testing Cycle



Performing Data-Driven Operations

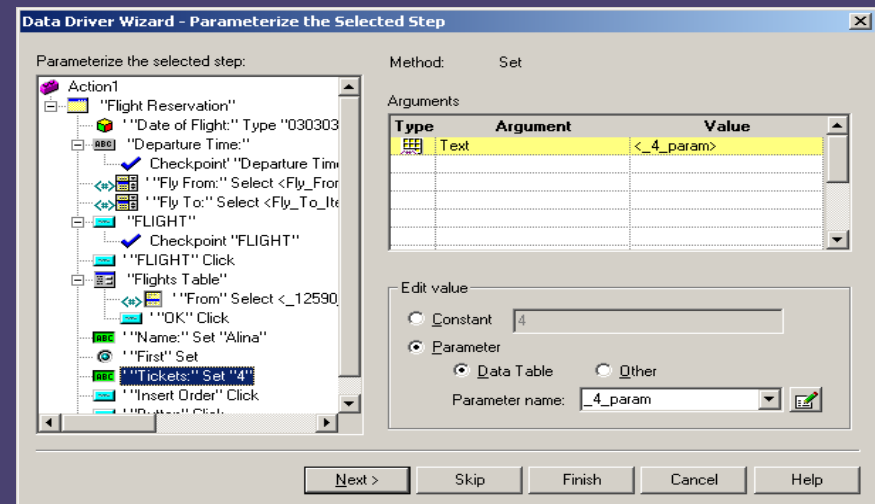
WinRunner:

- Parameterize:
 - Input
- Create iterations programmatically

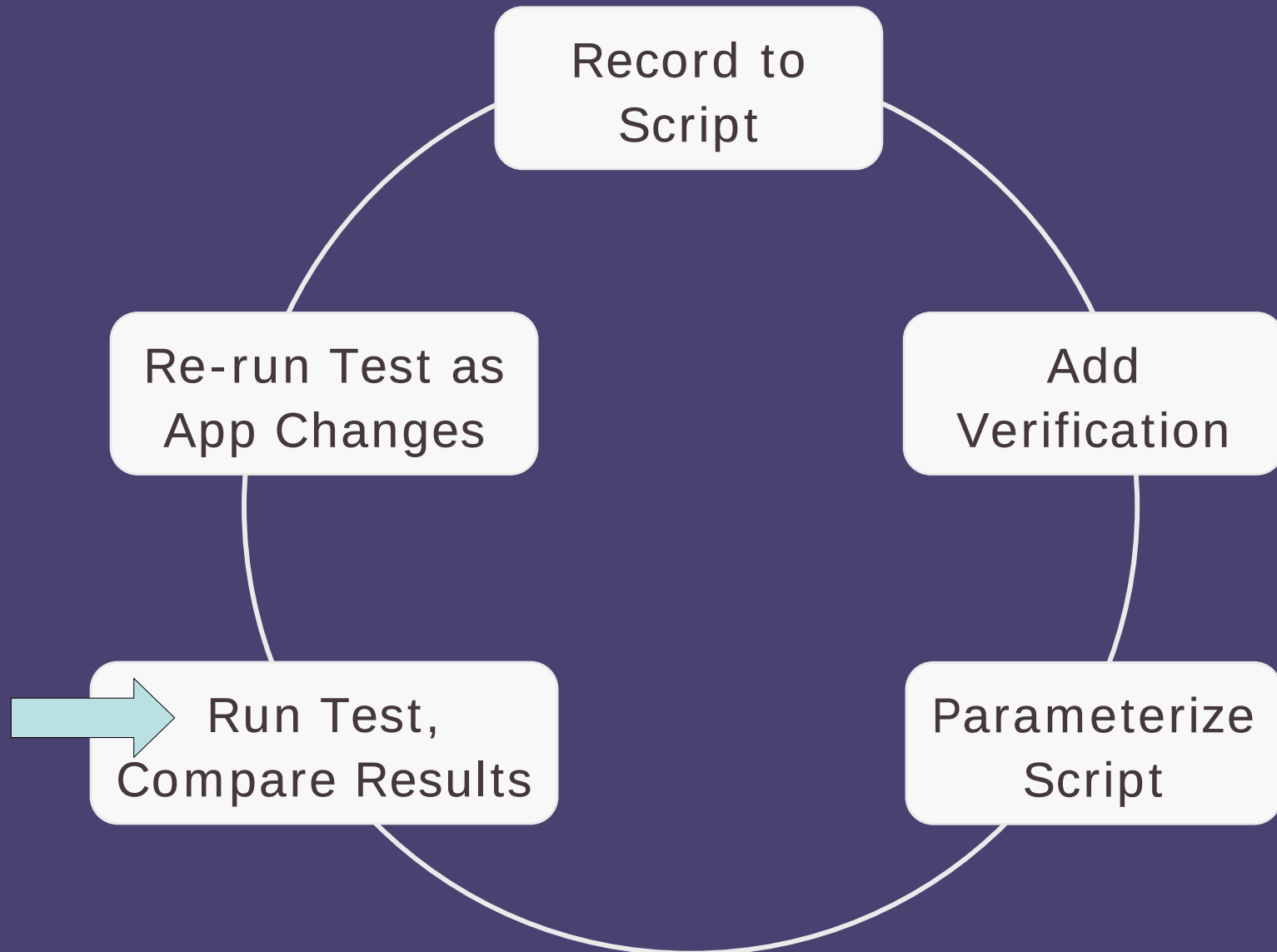


QuickTest Pro:

- Parameterize almost anything!
 - Input
 - Object Properties
 - Methods
- Create iterations automatically or programmatically



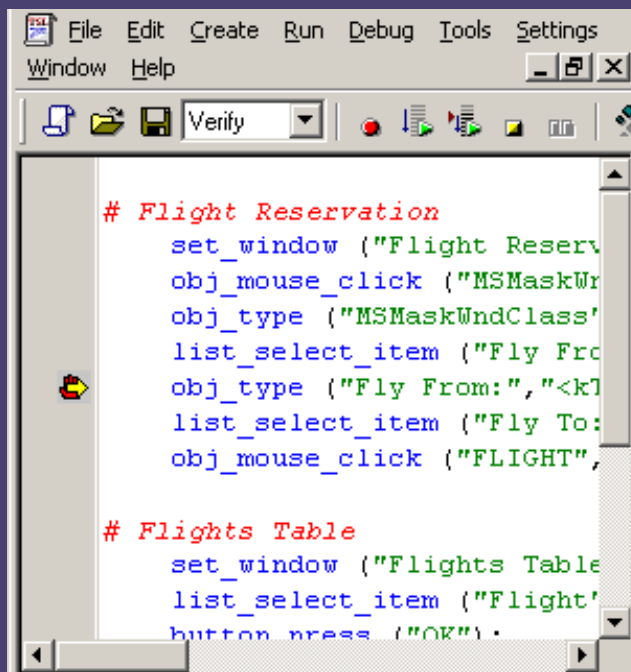
QTP and WR Features: Functional Testing Cycle



Debugging

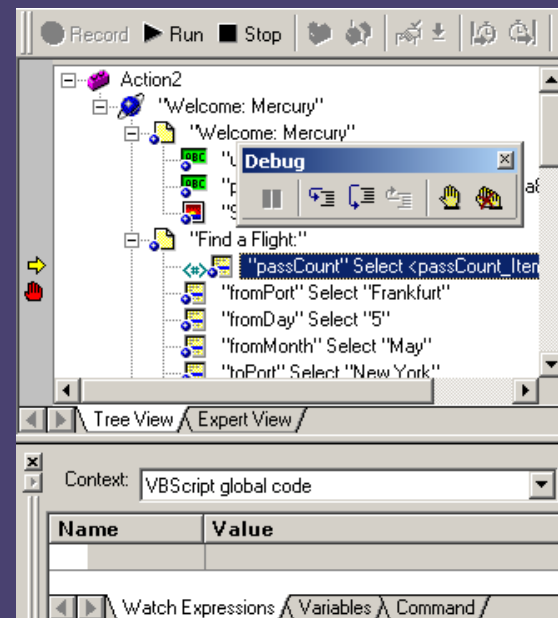
WinRunner

- Set breakpoints
- Step commands
- Pause test
- Watch expressions



QuickTest Pro

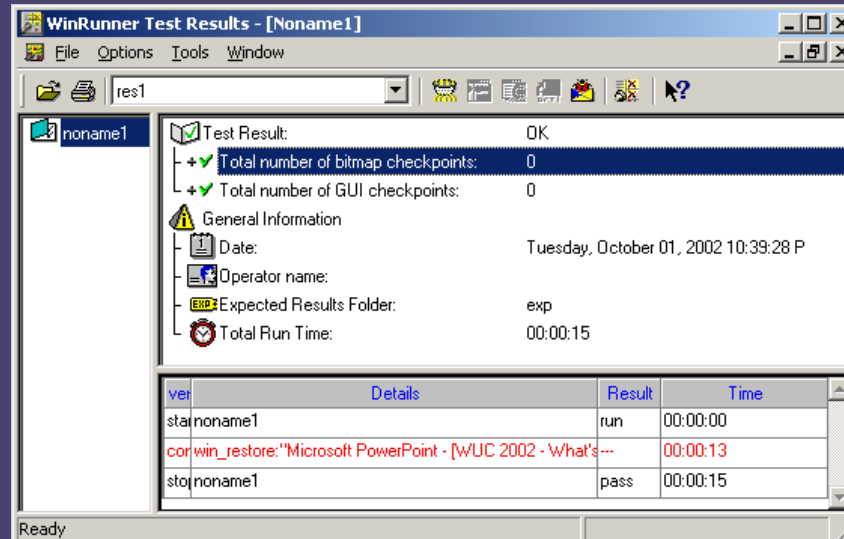
- Set breakpoints
- Step commands
- Pause test
- Watch expressions
- Watch & change variables



Reports

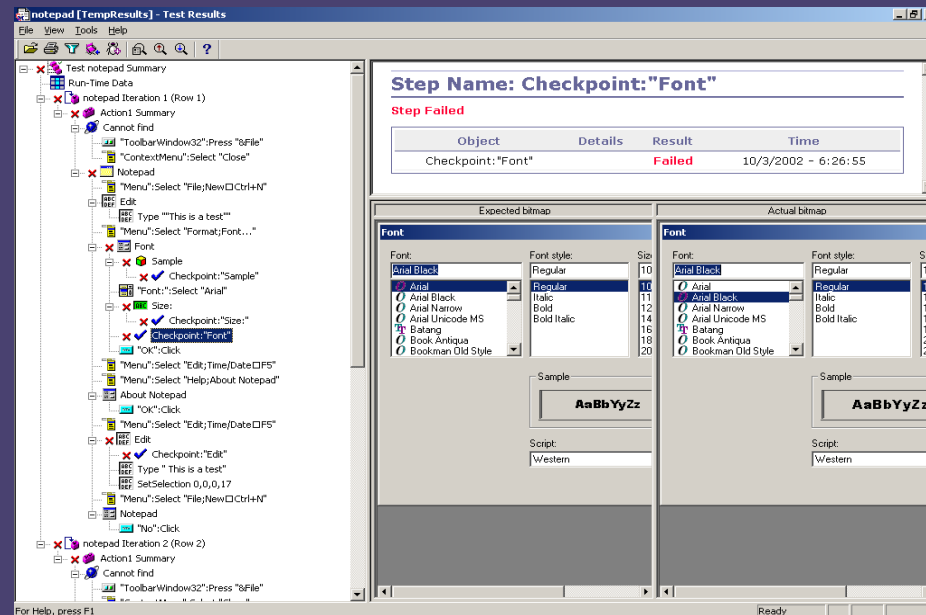
WinRunner

- Summary of checkpoints passed
- Failed checkpoints in grid view

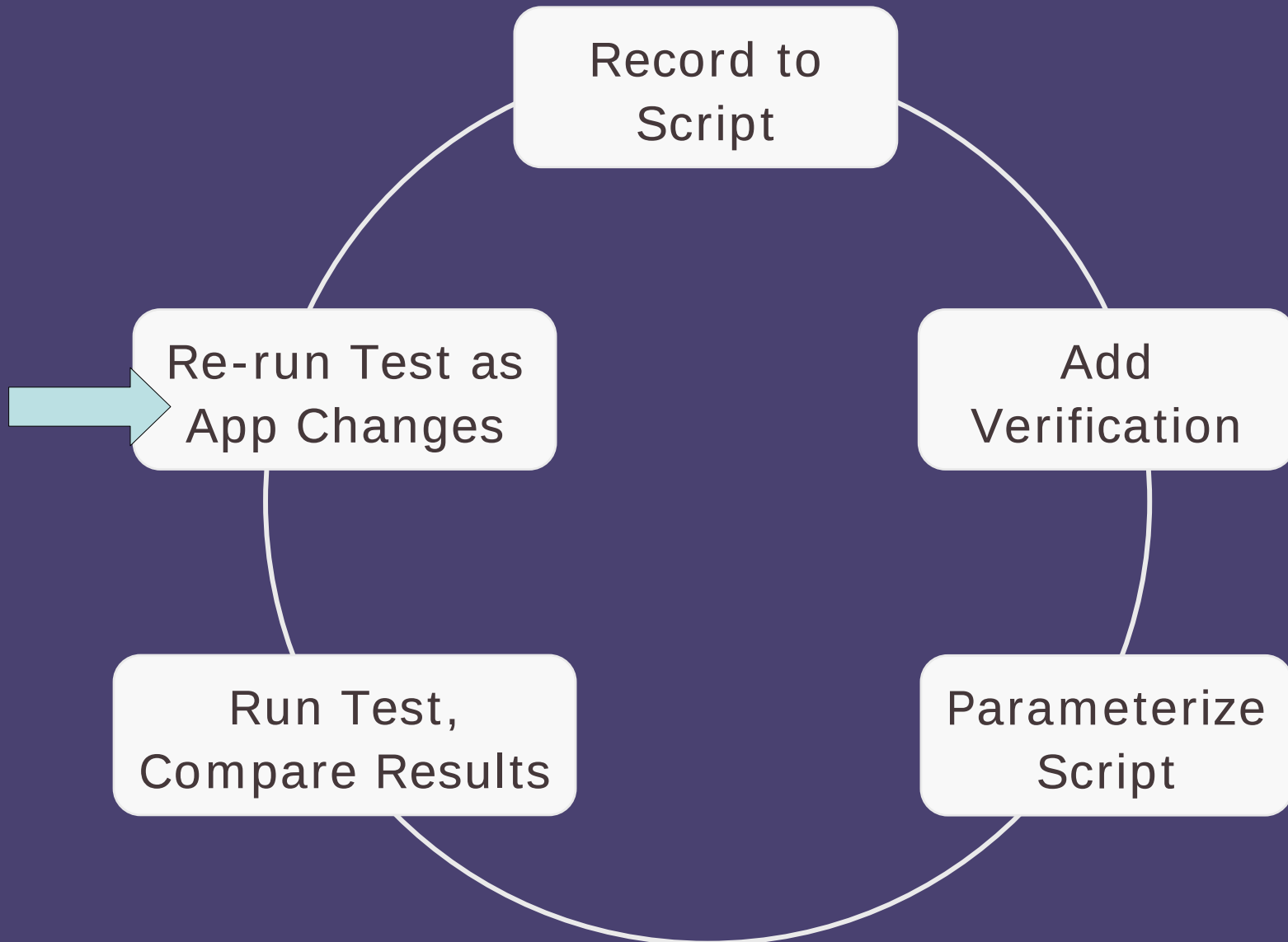


QuickTest Pro

- Tree view of test results mirrors test script
- Summary of actions passed
- Drill into exact failure point



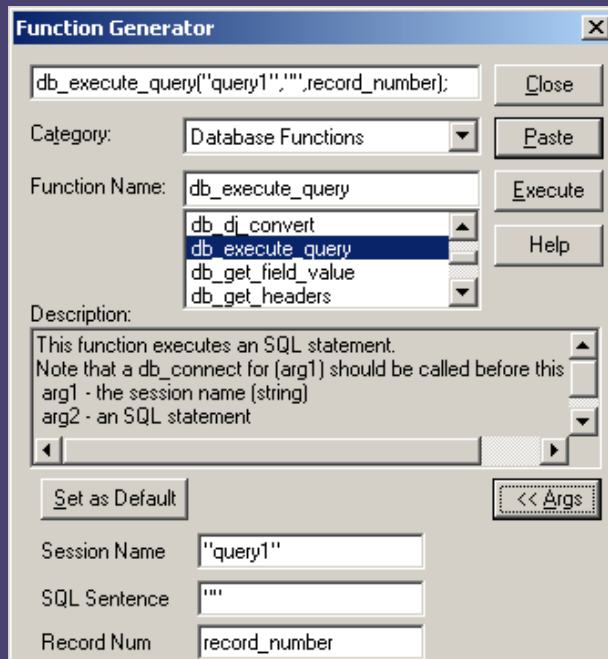
QTP and WR Features: Functional Testing Cycle



Working with Functions & Methods

WR: Function Generator

- TSL functions
 - Context-sensitive
 - Analog
 - Customization
- Add by pointing at object or via Function Generator



QTP: Method Wizard

- VBScript methods appropriate for selected object
- Recordable & non-recordable methods
- Add via tree view or Active Screen

